

Scientific Computing

Aditya Dendukuri

UC Santa Barbara

Contents

Preface	6
1 Fundamentals	7
1.1 Mathematical Notation and Proof Techniques	7
1.1.1 Logic and Set Notation	7
1.1.2 Proof Techniques	8
1.1.3 Notation Conventions	8
1.2 Vectors and Matrices	9
1.3 Matrix Arithmetic	13
1.3.1 Addition and Subtraction	14
1.3.2 Dot Product	16
1.3.3 Matrix Inverse	18
1.4 Introduction to Norms	20
1.5 Computational Complexity	22
1.5.1 Big-O Notation	23
1.5.2 Flop Counting	23
1.6 Floating Point Arithmetic	24
1.6.1 Floating Point Representation	24
1.6.2 IEEE 754 Special Values	26
1.6.3 Arithmetic Pitfalls	27
1.7 Quantifying Approximation Error	28
1.7.1 Absolute and Relative Errors	28
1.7.2 Error Analysis Frameworks	30
1.8 Python and NumPy Primer	31
1.8.1 Arrays and Basic Operations	31
1.8.2 Linear Algebra Operations	31
1.8.3 Key Idioms	32

2	Direct Methods for Linear Systems	34
2.1	Linearity	34
2.1.1	Linear Systems	34
2.1.2	Rows, Columns, and Geometry	35
2.1.3	Invertibility and Existence	35
2.1.4	Residuals and Approximate Solutions	36
2.1.5	Linear Combinations and Bases	37
2.1.6	Rank	37
2.1.7	The Four Fundamental Subspaces	38
2.2	LU Factorization	39
2.2.1	Gaussian Elimination as Matrix Factorization	39
2.2.2	Triangular Solves	40
2.2.3	Pivoting and Numerical Stability	41
2.2.4	Computational Complexity and Efficiency	43
2.3	Symmetric Positive Definite Matrices	44
2.3.1	Definitions and Characterizations	44
2.3.2	Geometry of the Energy Norm	46
2.3.3	The Cholesky Factorization Algorithm	46
2.3.4	Stability and Complexity Analysis	48
3	Orthogonality and Least Squares	49
3.1	Orthogonal Transformations and the QR Factorization	49
3.1.1	Orthogonality, Isometry, and Invariance	50
3.1.2	The QR Factorization	50
3.1.3	Householder Triangularization	51
3.1.4	Gram-Schmidt Orthogonalization	52
3.1.5	Orthogonal Projectors	52
3.2	Vector and Matrix Norms	53
3.2.1	Vector Norms	53
3.2.2	Matrix Norms	54
3.3	Conditioning and Numerical Stability	55
3.3.1	Residual, Forward Error, and Backward Error	55
3.3.2	Conditioning and Backward Stability	56
3.3.3	The Solver Selection Hierarchy	57
3.3.4	Iterative Refinement	57
3.4	Linear Least Squares and Overdetermined Systems	58
3.4.1	The Least-Squares Problem and the Normal Equations	58
3.4.2	Solving Least Squares via QR Factorization	59
3.4.3	Rank Deficiency, SVD, and Regularization	60
3.5	Chebyshev Polynomials	61
3.5.1	Failure of the Monomial Basis	61
3.5.2	Chebyshev Polynomials (T_k)	62
3.5.3	Zeros and Nodes	63
3.5.4	The Minimax Property	64
3.5.5	Chebyshev Series	65

4	Eigenvalues and the SVD	66
4.1	Eigenvalue Problems and the QR Algorithm	66
4.1.1	Theoretical Foundations: Multiplicity and Schur Form	66
4.1.2	Power Iteration and Shifted Inverse Iteration	67
4.1.3	The Practical QR Algorithm	68
4.2	The Singular Value Decomposition	69
4.2.1	The SVD Theorem and Subspace Characterization	70
4.2.2	Low-Rank Matrix Approximation	71
4.2.3	The Moore-Penrose Pseudoinverse	71
4.3	The Lanczos Algorithm	72
4.3.1	The Three-Term Lanczos Recurrence	72
4.3.2	Ritz Values, Ritz Vectors, and Error Bounds	73
4.3.3	Loss of Orthogonality and Ghost Eigenvalues	74
4.4	The Jacobi Eigenvalue Algorithm	75
4.4.1	Givens Plane Rotations and the Zeroing Angle	75
4.4.2	Monotone Convergence of the Off-Diagonal Norm	76
5	Iterative Methods for Linear Systems	77
5.1	Iterative Methods for Linear Systems	77
5.1.1	Matrix Splittings and Stationary Iterations	78
5.1.2	Convergence Theory and the Spectral Radius	78
5.1.3	Krylov Subspaces and Matrix-Free Solvers	79
5.2	Jacobi, Gauss-Seidel, and SOR Methods	80
5.2.1	The Jacobi Method	81
5.2.2	The Gauss-Seidel Method	81
5.2.3	Convergence Theorems	82
5.2.4	Successive Over-Relaxation (SOR)	83
5.3	The Conjugate Gradient Method	84
5.3.1	Quadratic Optimization and Steepest Descent	85
5.3.2	Conjugate Directions and Subspace Decoupling	85
5.3.3	The Conjugate Gradient Algorithm	86
5.3.4	Convergence Rate and Preconditioning	88
5.4	The Generalized Minimal Residual (GMRES) Method	89
5.4.1	The Arnoldi Process	89
5.4.2	Reduction to Hessenberg Least Squares	90
5.4.3	Solution via Givens Rotations	90
5.4.4	Restarted GMRES and Practical Considerations	91
6	Numerical Analysis	92
6.1	Polynomial Interpolation	92
6.1.1	Existence and Uniqueness	92
6.1.2	Lagrange Form	92
6.1.3	Barycentric Interpolation	93
6.1.4	Interpolation Error	93
6.1.5	Runge's Phenomenon	94

6.1.6	Cubic Splines	94
6.1.7	Exercises	94
6.2	Numerical Quadrature	95
6.2.1	Newton-Cotes Rules	95
6.2.2	Gaussian Quadrature	96
6.2.3	Richardson Extrapolation and Romberg Integration	97
6.2.4	Adaptive Quadrature	98
6.2.5	Periodic Functions and Spectral Accuracy	98
6.2.6	Exercises	98
6.3	Root Finding	99
6.3.1	Bracketing Methods	99
6.3.2	Open Methods	99
6.3.3	Brent's Method	101
6.3.4	Exercises	102
7	Ordinary Differential Equations	102
7.1	Numerical Methods for ODEs	102
7.1.1	Initial Value Problems	102
7.1.2	Error and Convergence	103
7.1.3	Euler Methods	104
7.1.4	Absolute Stability and Stiffness	106
7.1.5	Runge-Kutta Methods	108
7.1.6	Adaptive Time Stepping	109
7.1.7	Linear Multistep Methods	110
7.1.8	Systems and Eigenvalues	111
7.1.9	Hamiltonian and Symplectic Methods	112
7.1.10	Method Choice	113
8	Applications	113
8.1	Principal Component Analysis (PCA)	113
8.1.1	Definitions and Preprocessing	113
8.1.2	PCA via SVD	114
8.1.3	Variance and Dimensionality Reduction	115
8.1.4	Implementation Details	115
8.1.5	Exercises	115
8.2	Spectral Graph Theory and PageRank	116
8.2.1	Graph Matrices	116
8.2.2	Spectral Clustering	117
8.2.3	Random Walks and PageRank	118
8.2.4	Exercises	118
8.3	Linear Operators and the Poisson Equation	119
8.3.1	Differential Operators and Discretization	119
8.3.2	Operator Norms and Conditioning	121
8.3.3	Exercises	121
8.4	Fourier Analysis as Change of Basis	121

8.4.1	Discrete Fourier Transform (DFT)	121
8.4.2	Convolution and Circulant Matrices	123
8.4.3	The Fast Fourier Transform (FFT)	123
8.4.4	Exercises	124
8.5	Markov Chains and Stochastic Matrices	124
8.5.1	Definitions	124
8.5.2	Convergence and Mixing	126
8.5.3	Properties and Design	127
8.5.4	Exercises	128
8.6	Koopman Theory, DMD, and SINDy	128
8.6.1	Koopman Operator Theory	128
8.6.2	Dynamic Mode Decomposition (DMD)	129
8.6.3	SINDy: Sparse Identification	130
8.6.4	Exercises	130

Preface

These notes started as a study guide for CS 111 at UC Santa Barbara and grew through my PhD as teaching and research kept exposing things I thought I understood until I tried to explain them: why LU factorization is numerically stable, why the QR algorithm converges, what the Koopman operator actually is. Writing forces the gaps to surface. The result is a reference written by a graduate student for himself. Treat it accordingly.

For a rigorous treatment of this material, see [Principles of Scientific Computing](#) by Bindel and Goodman, and G. W. Stewart’s [Afternotes on Numerical Analysis](#). For further reading, the [CS 6241 reading list](#) at Cornell covers the literature well.

Computers were built to do numerical computation. The word “computer” originally meant a person whose job was to evaluate functions by hand. The U.S. Army employed hundreds of them during the Second World War to compute artillery firing tables, a task so large it directly motivated the construction of [ENIAC](#) (1945). The machine’s first classified application was thermonuclear weapon simulation, directed by John von Neumann, whose [1947 paper with Goldstine](#) laid the foundations of numerical stability analysis. In 1950, Jule Charney used ENIAC to run the [first machine weather forecast](#): [Lewis Fry Richardson](#) had estimated the same computation would require 64,000 human computers working in parallel. The [Apollo guidance computer](#) navigated to the Moon in real time with 4 kilobytes of memory. The [Cooley-Tukey FFT](#) (1965) reduced a core signal processing computation from $O(n^2)$ to $O(n \log n)$. The demand for numerical computation at scale was the original motivation for programmable machines.

Almost every numerical algorithm reduces a nonlinear problem to a sequence of linear ones, because linear systems are the only class of problems we understand completely: their solution structure, their sensitivity to perturbations, their reliable algorithms. Newton’s method, finite element discretization, PCA, gradient-based optimization, each is, at its core, a repeated linearization. Linear algebra is the substrate of scientific computing.

Much of the perspective here came from **CS 210A: Matrix Analysis**, taught by Professor Shivkumar Chandrasekaran in the ECE department at UC Santa Barbara. If you have the chance to take that course, take it.

Aditya Dendukuri
UC Santa Barbara, 2025

1 Fundamentals

1.1 Mathematical Notation and Proof Techniques

Standard vocabulary for proofs and algorithms.

1.1.1 Logic and Set Notation

Def 1.1 (Logical Quantifiers)

- **Universal ($\forall$):** “For all.”
- **Existential ($\exists$):** “There exists.”
- **Example:** “ $\forall \varepsilon > 0, \exists \delta > 0$ ” reads as “for every positive ε there is a positive δ .”

Def 1.2 (Implication and Equivalence)

- **Implication ($P \Rightarrow Q$):** P implies Q .
- **Equivalence ($P \Leftrightarrow Q$):** P if and only if Q (iff).
- **Contrapositive:** $\neg Q \Rightarrow \neg P$. Logically equivalent to $P \Rightarrow Q$.

Def 1.3 (Set Notation)

- $x \in S$: x is an element of S .
- $S \subseteq T$: S is a subset of T .
- $S \cup T$: Union; $S \cap T$: Intersection; $S \setminus T$: Set difference.
- $\{x \in S : P(x)\}$: Set builder notation.
- **Number Sets:** $\mathbb{Z}$ (integers), $\mathbb{R}$ (reals), $\mathbb{C}$ (complex), $\mathbb{R}^n$ (vectors), $\mathbb{R}^{m \times n}$ (matrices).

Ex 1.1

1. Write “every invertible matrix has a nonzero determinant” using quantifiers and implication.
2. State the contrapositive of: “if $\mathbf{A}$ is invertible, then $\mathbf{A}\mathbf{x} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$.”
3. Let $S = \{1, 2, 3, 4\}$ and $T = \{3, 4, 5, 6\}$. Compute $S \cup T, S \cap T, S \setminus T$.

1.1.2 Proof Techniques

Def 1.4 (Direct Proof) Assume P is true and derive Q through a chain of valid logical steps.

Def 1.5 (Proof by Contradiction) Assume $\neg P$ and derive a logical contradiction $(R \wedge \neg R)$. Proves P must hold.

Def 1.6 (Proof by Induction) To prove $P(n)$ holds for all integers $n \geq n_0$:

1. **Base case:** Verify $P(n_0)$ directly.
2. **Inductive step:** Assume $P(k)$ and prove $P(k+1)$.

Remark 1.1 Numerical Proofs: Often validate that an approximation is stable or that an error term converges. Frequently involves showing $|f(x) - \hat{f}(x)| \leq \varepsilon$ for small ε .

Ex 1.2

1. Prove: if $\mathbf{A}$ and $\mathbf{B}$ are symmetric, then $\mathbf{A} + \mathbf{B}$ is symmetric.
2. Prove by contradiction: if $\mathbf{A}\mathbf{x} = \mathbf{0} \Rightarrow \mathbf{x} = \mathbf{0}$, then $\mathbf{A}$ has no zero eigenvalue.
3. By induction, prove $\sum_{k=1}^n k = \frac{n(n+1)}{2}$.

1.1.3 Notation Conventions

Remark 1.2 (Scalars, Vectors, and Matrices)

- **Scalars:** Lowercase italic (a, b, x).
- **Vectors:** Lowercase bold ($\mathbf{x}, \mathbf{y}, \mathbf{a}$). Default is column vector.
- **Matrices:** Uppercase bold ($\mathbf{A}, \mathbf{B}, \mathbf{Q}$).
- **Indexing:** 1-based (unlike Python). x_i is i -th entry; a_{ij} is (i, j) entry.

Remark 1.3 (Transpose and Inverse)

- $\mathbf{A}^T$: Transpose.
- $\mathbf{A}^{-1}$: Matrix inverse (square, nonsingular matrices).
- $\mathbf{A}^{-T}$: Transpose of the inverse $(\mathbf{A}^{-1})^T$.
- $\mathbf{A}^k$: k -th power. $\mathbf{A}^0 = \mathbf{I}$.

Remark 1.4 (Sums and Products)

$$\sum_{i=1}^n a_i = a_1 + \cdots + a_n, \quad \prod_{i=1}^n a_i = a_1 \cdots a_n.$$

Remark 1.5 (Functions)

- $f : \mathbb{R}^n \rightarrow \mathbb{R}^m$: Maps n -vectors to m -vectors.
- $f \circ g$: Function composition $(f \circ g)(\mathbf{x}) = f(g(\mathbf{x}))$.

Remark 1.6 (Asymptotic Notation) Scaling of algorithms is described using Big-O notation ($O(n^2), O(n^3)$). Details in the Computational Complexity section.

1.2 Vectors and Matrices

Vectors and matrices serve as the primary data structures for representing numerical data and linear operators. Before introducing these formally, we fix notation for the standard number systems that provide the raw material for all numerical computation.

(**Standard Number Systems**) Throughout these notes, $\mathbb{R}$ denotes the set of all real numbers, $\mathbb{Z}$ the set of all integers, and $\mathbb{C}$ the set of all complex numbers. Our primary focus is real-valued computation; complex numbers arise naturally in eigenvalue analysis and signal processing.

Def 1.7 (**Scalar**) An element of a number system ($\mathbb{R}$, $\mathbb{Z}$, or $\mathbb{C}$) is called a **scalar**. Scalars represent individual numerical quantities and serve as the atomic building blocks from which vectors and matrices are constructed.

Ex 1.3 Refer to Def 1.7.

1. Classify each of the following as a real scalar, an integer, a complex (non-real) scalar, or *not* a scalar: 3.14 ; -7 ; $2 + 3i$; $\begin{pmatrix} 1 \\ 0 \end{pmatrix}$.
2. In Python, evaluate `type(3.14)`, `type(3)`, and `type(3+2j)`. Which NumPy dtype would you use to store each?

Def 1.8 (**Vectors**) A **vector** is a one-dimensional ordered array of scalars. Vectors can represent points, directions, or more abstract quantities within a vector space, and are fundamental in encoding data and linear relationships.

Remark 1.7 Vectors may be written as row vectors or column vectors. Although both forms carry the same information, the distinction matters when applying linear transformations and

computing inner products. **In these notes, we assume vectors are column vectors by default.**

Example

$$(1 \ 2 \ 3), \quad \begin{pmatrix} 1 \\ 2 \\ 3 \end{pmatrix}, \quad \begin{pmatrix} 1+2i \\ 7i \\ 3 \end{pmatrix}, \dots$$

Ex 1.4 Refer to Def 1.8.

1. Write a column vector $\mathbf{x} \in \mathbb{R}^4$ whose i -th entry equals 2^i for $i = 1, 2, 3, 4$.
2. In NumPy, construct this vector with `np.array([...])` and confirm `x.shape == (4,)`. **Why does NumPy report shape (4,) rather than (4,1)?** This is known as a *rank-1 array*; it is neither a row nor a column vector, and its broadcasting behavior differs from 2D arrays.
3. Can a vector in $\mathbb{R}^3$ be added to a vector in $\mathbb{R}^4$? Justify in one sentence.

Just as scalars belong to number systems like $\mathbb{R}$ or $\mathbb{C}$, vectors inhabit **vector spaces**. While these notes treat vector spaces informally, subsequent chapters provide more rigorous definitions and properties.

Def 1.9 ($\mathbb{R}^n$) The set of all real vectors of size n . When we write $\mathbf{x} \in \mathbb{R}^n$, it means $\mathbf{x}$ has n real entries. Vectors in $\mathbb{R}^n$ take one of the following forms:

$$\mathbf{x} \in \mathbb{R}^{n \times 1} = \begin{pmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{pmatrix} \quad (\text{column vector, default convention})$$

$$\mathbf{x} \in \mathbb{R}^{1 \times n} = (x_1 \ x_2 \ \cdots \ x_n) \quad (\text{row vector})$$

Remark 1.8 These notes focus on finite-dimensional spaces. Infinite-dimensional vector spaces, central to functional analysis, arise in differential equations and quantum mechanics but are beyond our scope here.

Ex 1.5 Refer to Def 1.9.

1. Let $\mathbf{x} = (1, -2, 3)^T \in \mathbb{R}^3$. Identify the value of entry x_2 . What is the dimension of $\mathbb{R}^3$?
2. In NumPy, create a column vector (shape (3,1)) from the list `[1, -2, 3]` in two ways: using `np.reshape` and using `[:, np.newaxis]`.

3. What does `x.T` return when `x.shape == (3,)`? How does the result differ when `x.shape == (3,1)`?

Def 1.10 (Matrices) A **matrix** is a two-dimensional array of scalars. A matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$ is written as

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{pmatrix}.$$

Matrices are used to represent linear transformations, systems of linear equations, and more complex data structures.

Example Two-dimensional matrices include:

$$\begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix} \in \mathbb{R}^{2 \times 3}, \quad \begin{pmatrix} 1 & 2 \\ 4 & 5 \\ 6 & 7 \end{pmatrix} \in \mathbb{R}^{3 \times 2}.$$

Remark 1.9 (Structural Intuition: Matrix Columns) Matrices can be interpreted as vectors whose entries are themselves vectors:

$$\mathbf{A} = \begin{pmatrix} \text{---} \mathbf{r}_1 \text{---} \\ \vdots \\ \text{---} \mathbf{r}_n \text{---} \end{pmatrix} = \begin{pmatrix} \left| \right. & \cdots & \left| \right. \\ \mathbf{c}_1 & \cdots & \mathbf{c}_n \\ \left| \right. & & \left| \right. \end{pmatrix},$$

where $\mathbf{r}_i = \mathbf{A}[i, :]$ is the i -th row and $\mathbf{c}_i = \mathbf{A}[:, i]$ is the i -th column. This viewpoint is vital: the product $\mathbf{A}\mathbf{x}$ is simply a **linear combination of the columns of $\mathbf{A}$** weighted by the entries of $\mathbf{x}$.

Ex 1.6 Refer to Def 1.10.

1. State the dimensions (m, n) of $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$. What is the shape of its second column as a vector?
2. In NumPy (0-based indexing), write the expression to access entry a_{12} (first row, second column in 1-based notation). Then extract the second column of $\mathbf{A}$ and verify its shape is $(2,)$.
3. True or False: a vector in $\mathbb{R}^n$ is a special case of a matrix in $\mathbb{R}^{n \times 1}$. How does NumPy handle this distinction in practice?

Def 1.11 (Matrix Transpose) The **transpose** of $\mathbf{A} \in \mathbb{R}^{m \times n}$, denoted $\mathbf{A}^T$, is the $n \times m$ matrix obtained by interchanging rows and columns:

$$(\mathbf{A}^T)_{ij} = a_{ji}, \quad 1 \leq i \leq n, \quad 1 \leq j \leq m.$$

For a column vector $\mathbf{x} \in \mathbb{R}^n$, the transpose $\mathbf{x}^T$ is its corresponding row vector and vice versa.

Def 1.12 (Symmetric and Skew-Symmetric Matrices) A square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is:

- **symmetric** if $\mathbf{A}^T = \mathbf{A}$, equivalently $a_{ij} = a_{ji}$ for all i, j .
- **skew-symmetric** if $\mathbf{A}^T = -\mathbf{A}$, equivalently $a_{ij} = -a_{ji}$ for all i, j .

Remark 1.10 Every square matrix $\mathbf{A}$ decomposes uniquely into a symmetric part and a skew-symmetric part:

$$\mathbf{A} = \underbrace{\frac{1}{2}(\mathbf{A} + \mathbf{A}^T)}_{\text{symmetric}} + \underbrace{\frac{1}{2}(\mathbf{A} - \mathbf{A}^T)}_{\text{skew-symmetric}}.$$

Symmetric matrices arise constantly in scientific computing: covariance matrices in statistics, graph Laplacians in network problems, and Hessians of quadratic objectives in optimization are all symmetric.

Thm 1.1 (Properties of the Transpose) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$, $\mathbf{B} \in \mathbb{R}^{n \times p}$, and $\alpha \in \mathbb{R}$. Then:

1. $(\mathbf{A}^T)^T = \mathbf{A}$
2. $(\mathbf{A} + \mathbf{B})^T = \mathbf{A}^T + \mathbf{B}^T$ (for conformable $\mathbf{A}, \mathbf{B}$)
3. $(\alpha \mathbf{A})^T = \alpha \mathbf{A}^T$
4. $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$ (**Reversal Law**)

Proof *Proof.* Properties (1)–(3) follow directly from the entry-wise definition $(\mathbf{A}^T)_{ij} = a_{ji}$. For (2):

$$\begin{aligned} (\mathbf{A} + \mathbf{B})_{ij}^T &= (\mathbf{A} + \mathbf{B})_{ji} \\ &= a_{ji} + b_{ji} \\ &= (\mathbf{A}^T)_{ij} + (\mathbf{B}^T)_{ij} \\ &= (\mathbf{A}^T + \mathbf{B}^T)_{ij}. \end{aligned}$$

For (4), let $\mathbf{C} = \mathbf{AB} \in \mathbb{R}^{m \times p}$. By the definition of matrix product:

$$(\mathbf{C}^T)_{ij} = c_{ji} = \sum_{k=1}^n a_{jk} b_{ki}.$$

Expanding the (i, j) entry of $\mathbf{B}^T \mathbf{A}^T$ using $(\mathbf{B}^T)_{ik} = b_{ki}$ and $(\mathbf{A}^T)_{kj} = a_{jk}$:

$$\begin{aligned} (\mathbf{B}^T \mathbf{A}^T)_{ij} &= \sum_{k=1}^n (\mathbf{B}^T)_{ik} (\mathbf{A}^T)_{kj} \\ &= \sum_{k=1}^n b_{ki} a_{jk}. \end{aligned}$$

Both summations are identical, so $(\mathbf{AB})^T = \mathbf{B}^T \mathbf{A}^T$. □

Cor 1.2 (Reversal Law for Multiple Factors) By induction on Thm 1.1(4):

$$(\mathbf{A}_1 \mathbf{A}_2 \cdots \mathbf{A}_k)^T = \mathbf{A}_k^T \cdots \mathbf{A}_2^T \mathbf{A}_1^T.$$

Ex 1.7 Refer to Def 1.11 and Thm 1.1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \end{pmatrix}$.

1. State the dimensions of $\mathbf{A}$, $\mathbf{A}^T$, $\mathbf{AA}^T$, and $\mathbf{A}^T \mathbf{A}$.
2. Compute $\mathbf{A}^T$ explicitly. Then compute $\mathbf{AA}^T$ and verify it is symmetric using Def 1.12.
3. In NumPy (0-based indexing), write the expression to retrieve element a_{12} (first row, second column in 1-based notation). Then write a one-liner for $\mathbf{AA}^T$.
4. Is $\mathbf{AA}^T = \mathbf{A}^T \mathbf{A}$ in general? What can you say about each product individually?

Cor 1.3 (Outer Product is Symmetric of Rank 1) For any nonzero $\mathbf{x} \in \mathbb{R}^n$, the outer product $\mathbf{xx}^T \in \mathbb{R}^{n \times n}$ is symmetric and has rank 1. The inner product $\mathbf{x}^T \mathbf{x} \in \mathbb{R}$ satisfies $\mathbf{x}^T \mathbf{x} \geq 0$ with equality iff $\mathbf{x} = \mathbf{0}$.

Ex 1.8 In this exercise, we will prove Cor 1.3. Refer to Def 1.11 and Thm 1.1. Let $\mathbf{x} = (x_1, \dots, x_n)^T \in \mathbb{R}^n$ be a nonzero column vector.

1. The **outer product** is $\mathbf{xx}^T \in \mathbb{R}^{n \times n}$. Write out entry (i, j) explicitly.
2. Use the Reversal Law (Thm 1.1(4)) to show $\mathbf{xx}^T$ is symmetric.
3. Show that $\mathbf{xx}^T$ has rank 1 by arguing about the column space. (*Hint: every column of $\mathbf{xx}^T$ is a scalar multiple of $\mathbf{x}$.*)
4. Express $\mathbf{x}^T \mathbf{x}$ as a sum of squares. Conclude $\mathbf{x}^T \mathbf{x} \geq 0$ with equality iff $\mathbf{x} = \mathbf{0}$, and identify the geometric quantity $\sqrt{\mathbf{x}^T \mathbf{x}}$.

1.3 Matrix Arithmetic

We are all familiar with scalar arithmetic: addition, subtraction, multiplication, and properties such as associativity and distributivity. In the following sections we extend these operations to vectors and matrices, building the algebraic machinery used throughout scientific computing.

1.3.1 Addition and Subtraction

Matrix addition generalizes scalar addition by operating entry-wise. The key constraint is dimensional compatibility: two matrices can be added only when they share the same shape.

Def 1.13 (Matrix Sum) Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{m \times n}$. Their sum is defined as

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} + \begin{pmatrix} b_{11} & \cdots & b_{1n} \\ \vdots & \ddots & \vdots \\ b_{n1} & \cdots & b_{nn} \end{pmatrix} = \begin{pmatrix} (a+b)_{11} & \cdots & (a+b)_{1n} \\ \vdots & \ddots & \vdots \\ (a+b)_{n1} & \cdots & (a+b)_{nn} \end{pmatrix},$$

where each entry satisfies $(a+b)_{ij} = a_{ij} + b_{ij}$ for $1 \leq i \leq m$ and $1 \leq j \leq n$.

Remark 1.11 For $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{p \times q}$, the sum $\mathbf{A} + \mathbf{B}$ is defined if and only if $m = p$ and $n = q$.

Ex 1.9 Refer to Def 1.13.

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 5 & 6 \\ 7 & 8 \end{pmatrix}$. Compute $\mathbf{A} + \mathbf{B}$ and $2\mathbf{A} - \mathbf{B}$ by hand.
2. In NumPy, verify your answers using `np.array`. Then check whether `np.array([1,2,3]) + np.array([1,2])` raises an error and explain why.

Commutativity and Associativity of Matrix Addition

Let

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\mathbf{A} + \mathbf{B} = \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} \quad \text{and} \quad \mathbf{B} + \mathbf{A} = \begin{pmatrix} e+a & f+b \\ g+c & h+d \end{pmatrix}.$$

Since addition of real numbers is commutative, $\mathbf{A} + \mathbf{B} = \mathbf{B} + \mathbf{A}$. Furthermore, for any third matrix $\mathbf{C} = \begin{pmatrix} i & j \\ k & l \end{pmatrix}$,

$$(\mathbf{A} + \mathbf{B}) + \mathbf{C} = \begin{pmatrix} (a+e)+i & (b+f)+j \\ (c+g)+k & (d+h)+l \end{pmatrix},$$

and

$$\mathbf{A} + (\mathbf{B} + \mathbf{C}) = \begin{pmatrix} a+(e+i) & b+(f+j) \\ c+(g+k) & d+(h+l) \end{pmatrix}.$$

By associativity of real addition, these are identical. Hence matrix addition is both commutative and associative.

Thm 1.4 ($\mathbb{R}^{m \times n}$ as a Vector Space) The set $\mathbb{R}^{m \times n}$ equipped with matrix addition and scalar multiplication forms a real vector space of dimension mn .

Proof *Proof.* Each axiom follows from the corresponding property of $\mathbb{R}$ applied entry-wise. For **commutativity**:

$$\begin{aligned} (\mathbf{A} + \mathbf{B})_{ij} &= a_{ij} + b_{ij} \\ &= b_{ij} + a_{ij} \\ &= (\mathbf{B} + \mathbf{A})_{ij}. \end{aligned}$$

The **additive identity** is $\mathbf{0}$ since $(\mathbf{A} + \mathbf{0})_{ij} = a_{ij} + 0 = a_{ij}$. The **additive inverse** of $\mathbf{A}$ is $-\mathbf{A}$, satisfying $(\mathbf{A} + (-\mathbf{A}))_{ij} = a_{ij} - a_{ij} = 0$.

For the **dimension**: let $\mathbf{E}^{(ij)}$ be matrices with a 1 in position (i, j) and zeros elsewhere. Every $\mathbf{A} \in \mathbb{R}^{m \times n}$ decomposes uniquely as:

$$\mathbf{A} = \sum_{i,j} a_{ij} \mathbf{E}^{(ij)}.$$

Since these mn matrices are linearly independent and span $\mathbb{R}^{m \times n}$, they form a basis of size mn . $\square$

Def 1.14 (**Scalar Multiplication**) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $c \in \mathbb{R}$. Then scalar multiplication is defined as

$$c\mathbf{A} = c \begin{pmatrix} a_{11} & \cdots & a_{1n} \\ \vdots & \ddots & \vdots \\ a_{n1} & \cdots & a_{nn} \end{pmatrix} = \begin{pmatrix} ca_{11} & \cdots & ca_{1n} \\ \vdots & \ddots & \vdots \\ ca_{n1} & \cdots & ca_{nn} \end{pmatrix},$$

where each entry satisfies $(ca)_{ij} = c \cdot a_{ij}$.

Distributivity of Scalar Multiplication over Matrix Addition

Let $\alpha \in \mathbb{R}$ and

$$\mathbf{A} = \begin{pmatrix} a & b \\ c & d \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} e & f \\ g & h \end{pmatrix}.$$

Then,

$$\alpha(\mathbf{A} + \mathbf{B}) = \alpha \begin{pmatrix} a+e & b+f \\ c+g & d+h \end{pmatrix} = \begin{pmatrix} \alpha(a+e) & \alpha(b+f) \\ \alpha(c+g) & \alpha(d+h) \end{pmatrix}.$$

Since scalar multiplication distributes over addition for real numbers, this equals

$$\begin{pmatrix} \alpha a + \alpha e & \alpha b + \alpha f \\ \alpha c + \alpha g & \alpha d + \alpha h \end{pmatrix} = \alpha \mathbf{A} + \alpha \mathbf{B}.$$

Ex 1.10 Refer to Def 1.14 and Thm 1.4.

1. Compute $3\mathbf{A} - 2\mathbf{B}$ for $\mathbf{A} = \begin{pmatrix} 1 & 0 \\ -1 & 2 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 2 & 1 \\ 0 & -1 \end{pmatrix}$.
2. The standard basis for $\mathbb{R}^{2 \times 2}$ consists of four matrices $\mathbf{E}^{(ij)}$. Write them out explicitly and verify that $\mathbf{A} = \sum_{i,j} a_{ij} \mathbf{E}^{(ij)}$ holds for your $\mathbf{A}$ above.

1.3.2 Dot Product

Before defining the general matrix product, we introduce the dot product of two vectors. The matrix product will then be understood entry-wise as a collection of dot products.

Def 1.15 (Dot Product) For vectors $\mathbf{a}, \mathbf{b} \in \mathbb{R}^n$, the **dot product** (or inner product) is defined as

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{a}^T \mathbf{b} = \begin{pmatrix} a_1 & \cdots & a_n \end{pmatrix} \begin{pmatrix} b_1 \\ \vdots \\ b_n \end{pmatrix} = \sum_{i=1}^n a_i b_i. \quad (1)$$

Ex 1.11 Refer to Def 1.15.

1. Compute $\mathbf{a} \cdot \mathbf{b}$ for $\mathbf{a} = (1, 2, 3)^T$ and $\mathbf{b} = (-1, 0, 2)^T$ by hand.
2. In NumPy, evaluate this using `np.dot(a, b)` and also with `a @ b`. Confirm both give the same result.
3. Show that $\mathbf{a} \cdot \mathbf{a} \geq 0$ with equality iff $\mathbf{a} = \mathbf{0}$. What geometric quantity does $\sqrt{\mathbf{a} \cdot \mathbf{a}}$ represent?

Def 1.16 (Matrix Product) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$. Their product $\mathbf{AB}$ is an $m \times p$ matrix defined by

$$(\mathbf{AB})_{ij} = \sum_{k=1}^n a_{ik} b_{kj},$$

or equivalently, if $\mathbf{a}_i$ denotes the i -th row of $\mathbf{A}$ and $\mathbf{b}_j$ the j -th column of $\mathbf{B}$:

$$\mathbf{AB} = \begin{pmatrix} \mathbf{a}_1 \cdot \mathbf{b}_1 & \cdots & \mathbf{a}_1 \cdot \mathbf{b}_p \\ \vdots & \ddots & \vdots \\ \mathbf{a}_m \cdot \mathbf{b}_1 & \cdots & \mathbf{a}_m \cdot \mathbf{b}_p \end{pmatrix}.$$

Remark 1.12 Matrix multiplication is defined only when the number of columns of $\mathbf{A}$ equals the number of rows of $\mathbf{B}$. It is associative and distributive over addition, but in general not commutative: $\mathbf{AB} \neq \mathbf{BA}$ for most matrices.

Lem 1.5 (Computational Cost of Matrix Multiplication) Computing $\mathbf{AB}$ where $\mathbf{A} \in \mathbb{R}^{m \times n}$ and $\mathbf{B} \in \mathbb{R}^{n \times p}$ requires exactly mnp multiplications and $m(n-1)p$ additions using the standard algorithm, giving $O(mnp)$ floating-point operations (flops). For two square $n \times n$ matrices this reduces to $O(n^3)$ flops.

Remark 1.13 The $O(n^3)$ cost of naive matrix multiplication is a central bottleneck in scientific computing. Strassen's algorithm reduces this to $O(n^{\log_2 7}) \approx O(n^{2.807})$, but is rarely used in

practice due to numerical stability concerns and cache-unfriendly memory access. Modern BLAS/LAPACK implementations approach peak hardware throughput via blocking strategies, but the asymptotic cost remains $O(n^3)$. For many applications, avoiding explicit matrix-matrix products entirely (e.g., exploiting sparsity or factored structure) is more impactful than algorithmic improvements.

Compatibility for Matrix Multiplication

Let $\mathbf{A}$ be a 3×4 matrix and $\mathbf{B}$ a 4×2 matrix. Since the number of columns of $\mathbf{A}$ (4) equals the number of rows of $\mathbf{B}$ (4), the product $\mathbf{AB}$ is defined and results in a 3×2 matrix. Conversely, if $\mathbf{B}$ were a 5×2 matrix, the product would be undefined.

Distributive Property of Matrix Multiplication

Let

$$\mathbf{A} = \begin{pmatrix} a_{11} & a_{12} \\ a_{21} & a_{22} \end{pmatrix}, \quad \mathbf{B} = \begin{pmatrix} b_{11} & b_{12} \\ b_{21} & b_{22} \end{pmatrix}, \quad \mathbf{C} = \begin{pmatrix} c_{11} & c_{12} \\ c_{21} & c_{22} \end{pmatrix}.$$

Then,

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{A} \begin{pmatrix} b_{11} + c_{11} & b_{12} + c_{12} \\ b_{21} + c_{21} & b_{22} + c_{22} \end{pmatrix},$$

and by computing the entries, one can verify that

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{AB} + \mathbf{AC}.$$

Non-Commutativity of Matrix Multiplication

Consider

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix} \quad \text{and} \quad \mathbf{B} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}.$$

Then,

$$\mathbf{AB} = \begin{pmatrix} 0 & 1 \\ 2 & 0 \end{pmatrix} \quad \text{and} \quad \mathbf{BA} = \begin{pmatrix} 0 & 2 \\ 1 & 0 \end{pmatrix}.$$

Since $\mathbf{AB} \neq \mathbf{BA}$, matrix multiplication is not commutative.

Ex 1.12 Refer to Def 1.16 and Lem 1.5. Let $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 0 & 3 \end{pmatrix}$ and $\mathbf{B} = \begin{pmatrix} 1 & -1 \\ 2 & 4 \end{pmatrix}$.

1. Compute $\mathbf{AB}$ and $\mathbf{BA}$ by hand. Confirm $\mathbf{AB} \neq \mathbf{BA}$.
2. Find a nonzero matrix $\mathbf{C} \neq \mathbf{I}$ that commutes with $\mathbf{A}$ (i.e., $\mathbf{AC} = \mathbf{CA}$). *Hint: try $\mathbf{C} = \alpha\mathbf{A} + \beta\mathbf{I}$.*

3. Verify associativity: compute $(\mathbf{AB})\mathbf{c}$ and $\mathbf{A}(\mathbf{Bc})$ for $\mathbf{c} = (1, 0)^T$ and confirm they are equal.
4. Compare the flop count of $(\mathbf{AB})\mathbf{c}$ versus $\mathbf{A}(\mathbf{Bc})$ for general $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\mathbf{c} \in \mathbb{R}^n$. Which order is cheaper and by what factor?

Many properties of scalar arithmetic extend naturally to vectors and matrices. In fact, matrices can be viewed as elements of a higher-dimensional vector space, and matrix multiplication can be interpreted as the composition of linear transformations. While the notations for vectors and matrices are similar, their roles differ:

- Vectors typically represent points or directions in space.
- Matrices often represent linear maps or transformations between vector spaces.

This conceptual distinction becomes crucial when discussing eigenvalues and linear transformations.

Thm 1.6 (Associativity as an Optimization) For $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\mathbf{x} \in \mathbb{R}^n$, computing $\mathbf{A}(\mathbf{Bx})$ via two matrix-vector products costs $O(n^2)$ flops, whereas forming $(\mathbf{AB})\mathbf{x}$ via a matrix-matrix product costs $O(n^3)$ flops.

Ex 1.13 Suppose $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ and $\mathbf{x} \in \mathbb{R}^n$.

1. *Approach 1:* Compute $\mathbf{M} = \mathbf{AB}$ first (cost?), then $\mathbf{Mx}$ (cost?). Write the total flop count.
2. *Approach 2:* Compute $\mathbf{y} = \mathbf{Bx}$ first (cost?), then $\mathbf{Ay}$ (cost?). Write the total flop count.
3. Conclude the asymptotic improvement factor as $n \rightarrow \infty$.
4. Write NumPy code for both approaches using `@`. Does NumPy automatically choose the optimal evaluation order for a chain like `A @ B @ x`?

1.3.3 Matrix Inverse

We now turn to a fundamental question: under what conditions can a matrix operation be defined that is analogous to scalar division? The matrix inverse plays the role of the reciprocal in scalar arithmetic, but it exists only under specific conditions.

Def 1.17 (Matrix Inverse) For a square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the **inverse** of $\mathbf{A}$, denoted $\mathbf{A}^{-1}$, is the unique matrix satisfying

$$\mathbf{AA}^{-1} = \mathbf{A}^{-1}\mathbf{A} = \mathbf{I},$$

where $\mathbf{I}$ is the $n \times n$ identity matrix. A matrix that has an inverse is called *invertible* or *nonsingular*.

Remark 1.14 A square matrix $\mathbf{A}$ is invertible if and only if $\det(\mathbf{A}) \neq 0$, which also implies that $\mathbf{A}$ has full rank.

Uniqueness of the Inverse

Suppose $\mathbf{A}$ is invertible and both $\mathbf{B}$ and $\mathbf{C}$ satisfy the conditions of Def 1.17. Then,

$$\mathbf{B} = \mathbf{B}\mathbf{I} = \mathbf{B}(\mathbf{A}\mathbf{C}) = (\mathbf{B}\mathbf{A})\mathbf{C} = \mathbf{I}\mathbf{C} = \mathbf{C}.$$

Thus the inverse of $\mathbf{A}$ is unique.

Thm 1.7 (Properties of the Matrix Inverse) Let $\mathbf{A}, \mathbf{B} \in \mathbb{R}^{n \times n}$ be invertible and $\alpha \in \mathbb{R} \setminus \{0\}$. Then:

1. $(\mathbf{A}^{-1})^{-1} = \mathbf{A}$
2. $(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ **(Reversal Law)**
3. $(\alpha\mathbf{A})^{-1} = \frac{1}{\alpha}\mathbf{A}^{-1}$
4. $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$

Proof *Proof. (1):* The conditions $\mathbf{A}^{-1}\mathbf{A} = \mathbf{I}$ and $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ show that $\mathbf{A}$ is both a left and right inverse of $\mathbf{A}^{-1}$. By the uniqueness of the inverse:

$$(\mathbf{A}^{-1})^{-1} = \mathbf{A}.$$

(2): Check the right-inverse condition for $\mathbf{B}^{-1}\mathbf{A}^{-1}$:

$$\begin{aligned} (\mathbf{A}\mathbf{B})(\mathbf{B}^{-1}\mathbf{A}^{-1}) &= \mathbf{A}(\mathbf{B}\mathbf{B}^{-1})\mathbf{A}^{-1} \\ &= \mathbf{A}\mathbf{I}\mathbf{A}^{-1} \\ &= \mathbf{A}\mathbf{A}^{-1} = \mathbf{I}. \end{aligned}$$

The left-inverse condition $(\mathbf{B}^{-1}\mathbf{A}^{-1})(\mathbf{A}\mathbf{B}) = \mathbf{I}$ follows similarly. Thus $(\mathbf{A}\mathbf{B})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$.

(3): For any $\alpha \neq 0$:

$$(\alpha\mathbf{A})\left(\frac{1}{\alpha}\mathbf{A}^{-1}\right) = \frac{\alpha}{\alpha}(\mathbf{A}\mathbf{A}^{-1}) = \mathbf{I}.$$

(4): Transpose both sides of $\mathbf{A}\mathbf{A}^{-1} = \mathbf{I}$ using the Reversal Law for transposes:

$$\begin{aligned} (\mathbf{A}\mathbf{A}^{-1})^T &= \mathbf{I}^T \\ (\mathbf{A}^{-1})^T\mathbf{A}^T &= \mathbf{I}. \end{aligned}$$

This shows $(\mathbf{A}^{-1})^T$ is the inverse of $\mathbf{A}^T$, so $(\mathbf{A}^T)^{-1} = (\mathbf{A}^{-1})^T$. □

Remark 1.15 (The Decomposition Principle) In scientific computing, we almost never compute $\mathbf{A}^{-1}$ explicitly. To solve $\mathbf{A}\mathbf{x} = \mathbf{b}$, we decompose $\mathbf{A}$ into simpler factors (like LU or QR) and

solve a sequence of simpler systems. This is more numerically stable and computationally efficient. The Reversal Law $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$ reflects this: to solve $(\mathbf{AB})\mathbf{x} = \mathbf{b}$, we first solve $\mathbf{Ay} = \mathbf{b}$ for $\mathbf{y}$, then $\mathbf{Bx} = \mathbf{y}$.

Ex 1.14 Refer to Def 1.17 and Thm 1.7.

1. For $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 5 & 3 \end{pmatrix}$, compute $\mathbf{A}^{-1}$ by hand using the 2×2 formula $\mathbf{A}^{-1} = \frac{1}{\det \mathbf{A}} \begin{pmatrix} d & -b \\ -c & a \end{pmatrix}$. Verify $\mathbf{AA}^{-1} = \mathbf{I}$.
2. Given $\mathbf{B} = \begin{pmatrix} 1 & 0 \\ -1 & 1 \end{pmatrix}$, use the Reversal Law to compute $(\mathbf{AB})^{-1}$ without inverting a new 2×2 system.
3. Explain why `np.linalg.solve(A, b)` is preferred over `np.linalg.inv(A) @ b` for solving $\mathbf{Ax} = \mathbf{b}$. Address both numerical accuracy and computational cost.

Def 1.18 (**Involutory Matrix**) A square matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is called **involutory** if $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.15 In this exercise, we will prove that a matrix is involutory (Def 1.18) if and only if it is its own inverse. Refer to Def 1.17 and Thm 1.7.

1. Suppose $\mathbf{A}^2 = \mathbf{I}$. Multiply both sides of $\mathbf{AA} = \mathbf{I}$ on the right by $\mathbf{A}^{-1}$ to deduce $\mathbf{A}^{-1} = \mathbf{A}$.
2. Conversely, suppose $\mathbf{A}^{-1} = \mathbf{A}$. Show $\mathbf{A}^2 = \mathbf{I}$.
3. Give a concrete 2×2 example of an involutory matrix other than $\pm \mathbf{I}$. Verify it satisfies $\mathbf{A}^2 = \mathbf{I}$.

Ex 1.16 (**Identifying Invertibility**) For each matrix below, determine without computing the inverse whether it is invertible. Give a one-sentence justification.

$$(a) \begin{pmatrix} 1 & 2 \\ 2 & 4 \end{pmatrix} \quad (b) \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad (c) \begin{pmatrix} 3 & 0 & 0 \\ 0 & 5 & 0 \\ 0 & 0 & -1 \end{pmatrix} \quad (d) \begin{pmatrix} 1 & 1 & 0 \\ 0 & 1 & 1 \\ 1 & 2 & 1 \end{pmatrix}$$

1.4 Introduction to Norms

A *norm* is a function that assigns a non-negative scalar “size” to a vector or matrix. Norms are the fundamental tool for measuring errors, bounding perturbations, and defining convergence throughout scientific computing. This chapter introduces the key vector norm definitions and the families most used in practice. We will study norms more rigorously in later chapters, where we develop matrix norms, operator inequalities, Cauchy-Schwarz, Hölder’s inequality, and the condition number.

Def 1.19 (**Vector Norm**) A function $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ is a *vector norm* if for all $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and $c \in \mathbb{R}$:

1. $\|\mathbf{x}\| \geq 0$ and $\|\mathbf{x}\| = 0 \Leftrightarrow \mathbf{x} = \mathbf{0}$ (positive definiteness)
2. $\|c\mathbf{x}\| = |c|\|\mathbf{x}\|$ (absolute homogeneity)
3. $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$ (triangle inequality)

Def 1.20 (ℓ^p Norms) For $\mathbf{x} \in \mathbb{R}^n$ and $p \geq 1$, the ℓ^p norm is

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p \right)^{1/p}.$$

The three most common cases are:

- $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$ (*Manhattan norm*, the sum of absolute values)
- $\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2} = \sqrt{\mathbf{x}^T \mathbf{x}}$ (*Euclidean norm*)
- $\|\mathbf{x}\|_\infty = \max_{1 \leq i \leq n} |x_i|$ (*max norm*, the limit as $p \rightarrow \infty$)

Remark 1.16 (**Physical Intuition**) In scientific computing, the choice of norm depends on what you want to measure:

- ℓ_2 (**Energy**): The standard measure of magnitude; corresponds to physical energy or distance.
- ℓ_∞ (**Worst-case**): Measures the single largest entry. Useful for “tolerance” checks (e.g., “every entry must be below 10^{-6} ”).
- ℓ_1 (**Sparsity**): Encourages or measures sparsity; often used in optimization and compressed sensing.

Example For $\mathbf{x} = (3, -4, 0)^T$:

$$\|\mathbf{x}\|_1 = 3 + 4 + 0 = 7, \quad \|\mathbf{x}\|_2 = \sqrt{9 + 16 + 0} = 5, \quad \|\mathbf{x}\|_\infty = 4.$$

The unit balls $\{\mathbf{x} : \|\mathbf{x}\|_p \leq 1\}$ in $\mathbb{R}^2$ are: a diamond ($p = 1$), a circle ($p = 2$), and a square ($p = \infty$). As p increases, the unit ball grows toward the ℓ^∞ square.

Remark 1.17 Throughout these notes, $\|\cdot\|$ without a subscript denotes the Euclidean norm $\|\cdot\|_2$ unless stated otherwise. Errors and residuals are almost always measured in $\|\cdot\|_2$. In NumPy: `np.linalg.norm(x)` computes $\|\mathbf{x}\|_2$; `np.linalg.norm(x, ord=p)` computes $\|\mathbf{x}\|_p$.

Remark 1.18 (**Numerical Zero**) In floating-point arithmetic, the condition $\|\mathbf{x}\| = 0$ is rarely met exactly due to rounding. We instead use a **tolerance** ε : we say $\mathbf{x}$ is “numerically zero” if $\|\mathbf{x}\| \leq \varepsilon \cdot \|\mathbf{x}_0\|$, where $\mathbf{x}_0$ is some initial reference vector.

Thm 1.8 (All Norms are Equivalent on $\mathbb{R}^n$) For any two norms $\|\cdot\|_\alpha$ and $\|\cdot\|_\beta$ on $\mathbb{R}^n$, there exist constants $c_1, c_2 > 0$ such that

$$c_1 \|\mathbf{x}\|_\beta \leq \|\mathbf{x}\|_\alpha \leq c_2 \|\mathbf{x}\|_\beta \quad \text{for all } \mathbf{x} \in \mathbb{R}^n.$$

In particular: $\|\mathbf{x}\|_2 \leq \|\mathbf{x}\|_1 \leq \sqrt{n} \|\mathbf{x}\|_2$ and $\|\mathbf{x}\|_\infty \leq \|\mathbf{x}\|_2 \leq \sqrt{n} \|\mathbf{x}\|_\infty$.

Proof *Proof.* Since $\mathbb{R}^n$ is finite-dimensional, the unit sphere $S_\beta = \{\mathbf{x} : \|\mathbf{x}\|_\beta = 1\}$ is compact. Any norm $\|\cdot\|_\alpha$ is a continuous function on $\mathbb{R}^n$, and by the Extreme Value Theorem, it attains its minimum c_1 and maximum c_2 on S_β :

$$c_1 = \min_{\mathbf{x} \in S_\beta} \|\mathbf{x}\|_\alpha, \quad c_2 = \max_{\mathbf{x} \in S_\beta} \|\mathbf{x}\|_\alpha.$$

For any nonzero $\mathbf{y} \in \mathbb{R}^n$, let $\mathbf{x} = \mathbf{y}/\|\mathbf{y}\|_\beta \in S_\beta$. Then:

$$c_1 \leq \left\| \frac{\mathbf{y}}{\|\mathbf{y}\|_\beta} \right\|_\alpha \leq c_2 \quad \Rightarrow \quad c_1 \|\mathbf{y}\|_\beta \leq \|\mathbf{y}\|_\alpha \leq c_2 \|\mathbf{y}\|_\beta.$$

The explicit inequality $\|\mathbf{x}\|_1 \leq \sqrt{n} \|\mathbf{x}\|_2$ follows from Cauchy-Schwarz applied to $|\mathbf{x}|$ and the all-ones vector $\mathbf{1}$:

$$\begin{aligned} \|\mathbf{x}\|_1 &= \sum_{i=1}^n |x_i| \cdot 1 \\ &\leq \left(\sum_{i=1}^n x_i^2 \right)^{1/2} \left(\sum_{i=1}^n 1^2 \right)^{1/2} \\ &= \sqrt{n} \|\mathbf{x}\|_2. \end{aligned}$$

□

Ex 1.17 Refer to Defs 1.19–1.20 and Thm 1.8.

1. For $\mathbf{x} = (-1, 2, -3, 4)^T$, compute $\|\mathbf{x}\|_1$, $\|\mathbf{x}\|_2$, $\|\mathbf{x}\|_\infty$ by hand. Verify all three norm-equivalence inequalities hold.
2. Verify the triangle inequality $\|\mathbf{x} + \mathbf{y}\|_2 \leq \|\mathbf{x}\|_2 + \|\mathbf{y}\|_2$ for $\mathbf{x} = (1, 2)^T$, $\mathbf{y} = (-3, 1)^T$ by computing both sides.
3. Using NumPy, confirm your hand calculations with `np.linalg.norm(x, 1)`, `np.linalg.norm(x, 2)` and `np.linalg.norm(x, np.inf)`.
4. Show that $\|\mathbf{x}\|_\infty = \lim_{p \rightarrow \infty} \|\mathbf{x}\|_p$. (*Hint: let $M = \max_i |x_i|$; factor M out of $\|\mathbf{x}\|_p$ and take the limit.*)

1.5 Computational Complexity

Algorithmic cost measured in arithmetic operations (flops). Determines scalability and feasibility.

1.5.1 Big-O Notation

Def 1.21 (**Big-O Notation**) $f(n) = O(g(n))$ if $\exists C, n_0 > 0$ s.t. $|f(n)| \leq C|g(n)|$ for $n \geq n_0$.

- $\Omega(g)$: Grows at least as fast.
- $\Theta(g)$: Grows at the same rate.
- $o(g)$: Grows strictly slower.

Remark 1.19 (**Typical Rates**) $O(1) \ll O(\log n) \ll O(n) \ll O(n \log n) \ll O(n^2) \ll O(n^3) \ll O(2^n)$.
For $n = 1000$: $O(n^2)$ is $\sim 10^6$ ops (ms); $O(n^3)$ is $\sim 10^9$ ops (s).

Remark 1.20 (**Properties**) 1. **Transitivity**: $f = O(g), g = O(h) \Rightarrow f = O(h)$. 2. **Sum Rule**: $O(f) + O(g) = O(\max(f, g))$. 3. **Product Rule**: $O(f) \cdot O(g) = O(fg)$. 4. **Constant Factors**: $O(cf) = O(f)$.

Ex 1.18

1. Show $3n^2 + 100n + 5 = O(n^2)$.
2. Compare growth: $n \log n$ vs n^2 .
3. Rank by cost: $5n^3 + n, 100n^2, n^2 \log n, 2^{10}n^2$.
4. Use Master Theorem: $T(n) = 2T(n/2) + n$.

1.5.2 Flop Counting

Hardware-independent measure of work.

Def 1.22 (**Flop**) A single floating-point operation $(+, -, \times, \div)$.

Thm 1.9 (**Basic Costs**) For $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ and $\mathbf{A} \in \mathbb{R}^{m \times n}, \mathbf{B} \in \mathbb{R}^{n \times p}$:

- **Dot product**: $2n$ flops.
- **Matrix-vector product**: $2mn$ flops.
- **Matrix-matrix product**: $2mnp$ flops ($2n^3$ for square).
- **Triangular solve**: n^2 flops.

Remark 1.21 **Data Locality and Performance**: On modern architectures, computational throughput often significantly exceeds memory bandwidth. Consequently, algorithms with high cache locality, such as blocked matrix multiplication (GEMM), may achieve superior performance compared to algorithms with lower theoretical operation counts but poorer data reuse.

Remark 1.22 (**Benchmarks**)

- **LU Factorization:** $\frac{2}{3}n^3$.
- **Cholesky:** $\frac{1}{3}n^3$.
- **QR (Householder):** $\frac{4}{3}n^3$.

Ex 1.19

1. Verify $2mn$ flops for $\mathbf{Ax}$ by counting multiply-adds.
2. Estimate wall-clock time for 10^9 flops on a 10^9 flop/s CPU.
3. Python: Plot `np.dot(A, B)` time vs n (log-log). Verify exponent ≈ 3 .
4. Why might the empirical exponent be less than 3? (Cache hierarchy, vectorization).

1.6 Floating Point Arithmetic

Digital computers represent real numbers using a finite number of bits. The result is not the real line, but a finite, non-uniform grid of representable numbers. Numerical analysis begins with this mismatch: the mathematical algorithm acts on real numbers, while the computer executes a nearby algorithm on rounded numbers.

Floating-point arithmetic is usually accurate locally: one elementary operation is rounded to a nearby representable number. Instability appears when an algorithm repeatedly amplifies these tiny local errors, or when it subtracts nearly equal quantities and exposes digits that were already contaminated by rounding.

1.6.1 Floating Point Representation

Most modern systems adhere to the IEEE 754 standard, which represents numbers in a scientific-like notation. Normalized numbers take the form $x = (-1)^s \times (1.f) \times 2^e$.

- **s:** Sign bit, determining whether the number is positive or negative.
- **f:** Fractional significand (mantissa); the leading 1 is implicit to maximize precision.
- **e:** Biased exponent, allowing both very large and very small numbers to be represented.

The exponent controls scale and the significand controls precision. This means floating-point numbers have roughly fixed **relative** spacing, not fixed absolute spacing. There are many representable numbers near zero and far fewer, in absolute terms, near 10^{100} . The gap between adjacent numbers near x is proportional to $|x|$.

Remark 1.23 (IEEE 754 Standards)

- **Single (32-bit):** 1 sign, 8 exponent, 23 fraction bits.
- **Double (64-bit):** 1 sign, 11 exponent, 52 fraction bits.

Example (16-bit float) 1 sign, 5 exp (bias 15), 10 frac. Represent -3.25 :

1. $s = 1$.
2. $3.25_{10} = 11.01_2 = 1.101_2 \times 2^1$.
3. $e = 1 + 15 = 16 = 10000_2$.
4. $f = 1010000000_2$.

Result: 1 10000 1010000000.

Ex 1.20

1. Encode $+5.5$ in 16-bit format.
2. How many distinct normalized numbers for 5 exp bits, 10 frac bits?

Def 1.23 (**Machine Epsilon** ($\varepsilon_{\text{mach}}$)) Machine epsilon is the distance from 1 to the next larger floating-point number. Equivalently, it is the smallest positive floating-point number ε such that $\text{fl}(1 + \varepsilon) > 1$. It characterizes the spacing of representable numbers near 1 and is given by $\varepsilon_{\text{mach}} = 2^{1-p}$, where p is the number of bits in the significand.

- For double precision (64-bit), $\varepsilon_{\text{mach}} \approx 2.22 \times 10^{-16}$.

Remark 1.24 (**Spacing rule**) Near a normalized number x , the distance to the next representable number is approximately $\varepsilon_{\text{mach}}|x|$. Near 10^{16} , the gap is on the order of 1; near 1, the gap is on the order of 10^{-16} . This is why adding a small number to a huge number may do nothing.

Thm 1.10 (**Absorption Criterion**) In round-to-nearest arithmetic, if $|y|$ is less than half the spacing between adjacent floating-point numbers near x , then

$$\text{fl}(x + y) = x.$$

Equivalently, small increments disappear when they are below the local resolution of the floating-point grid.

Ex 1.21

1. Use Def 1.23 to predict whether $1.0 + 1\text{e-}16$ and $1.0 + 2\text{e-}16$ differ from 1.0; then check in Python.
2. Find $\varepsilon_{\text{mach}}$ empirically via loop.

3. Use Thm 1.10 to predict the smallest power of 10 for which `1e16 + 10**k` changes `1e16`.
4. Explain why the answer changes if `1e16` is replaced by `1.0`.

Def 1.24 (Unit Roundoff (u)) Unit roundoff is the maximum relative error caused by rounding one real number to the nearest floating-point number:

$$\frac{|\text{fl}(x) - x|}{|x|} \leq u.$$

For round-to-nearest arithmetic, $u = \varepsilon_{\text{mach}}/2 = 2^{-p}$.

Remark 1.25 (Machine epsilon vs. unit roundoff) $\varepsilon_{\text{mach}}$ is a spacing near 1. The unit roundoff u is an error bound for rounding to nearest. Many texts use “machine epsilon” for both quantities, so always check the convention. In these notes, elementary rounding errors are bounded by u .

Thm 1.11 (Fundamental Axiom) IEEE 754 guarantees $\text{fl}(a \circ b) = (a \circ b)(1 + \delta)$ for $\circ \in \{+, -, \times, \div\}$ and $|\delta| \leq u$. Individual operations are as accurate as the representation allows.

Proof *Proof.* The real line near x is partitioned into intervals, each containing exactly one floating point number. In round-to-nearest mode, $\text{fl}(x)$ is the closer endpoint, so:

$$|\text{fl}(x) - x| \leq \frac{1}{2} \text{gap} = \frac{1}{2}(\varepsilon_{\text{mach}}|x|) = u|x|.$$

Dividing by $|x|$ gives the relative error bound $|\delta| \leq u$. For any operation $\circ$, IEEE 754 mandates computing the exact result first, then rounding, ensuring:

$$\text{fl}(a \circ b) = \text{round}(a \circ b) = (a \circ b)(1 + \delta).$$

□

Remark 1.26 Warning: After n operations, error often grows like $O(nu)$ when errors do not systematically amplify. This is a rule of thumb, not a guarantee. Operation order matters because each intermediate result is rounded.

Ex 1.22

1. Let $a = 1, b = -0.999\dots$. Is error in $\text{fl}(a + b)$ bounded by u ?
2. Estimate worst-case relative error for 10^6 sequential additions.

1.6.2 IEEE 754 Special Values

- **Signed Zero:** $+0, -0$. $1/(+0) = \infty$, $1/(-0) = -\infty$.

- **Subnormals:** Fill the gap between 0 and the smallest normalized number. Reduced precision, allows gradual underflow.
- **Infinities:** $\pm\infty$. Result of overflow or $x/0$.
- **NaN:** Not a Number ($0/0$, $\infty - \infty$). Unequal to everything, including itself.

Remark 1.27 (**NaN propagation**) A NaN usually contaminates all later arithmetic. This is useful for debugging: it marks an invalid operation that happened earlier, even if the failure is observed much later.

Ex 1.23

1. Python: `1.0/0.0` vs `1.0/-0.0`.
2. Result of `float('inf') - float('inf')` and `float('nan') == float('nan')`.

1.6.3 Arithmetic Pitfalls

Because floating-point numbers are discrete, the standard laws of algebra do not always hold. In particular, operations are neither associative nor distributive in general.

- **Non-Associativity:** The order of operations can change the result, i.e., $(a+b)+c \neq a+(b+c)$.
- **Non-Distributivity:** $a(b+c) \neq ab+ac$.
- **Absorption:** If $|a|$ is much larger than $|b|$, then $a+b$ may equal a exactly because b is smaller than the gap between representable numbers near a .

Example (**Absorption**) In double precision, `1e16 + 1.0 == 1e16` is true. The number 1 is smaller than the spacing between adjacent representable numbers near 10^{16} , so it is rounded away. Consequently `(1e16 + 1.0) - 1e16` returns 0.0, not 1.0.

Example (**Non-associativity**) Let $a = 10^{16}$, $b = -10^{16}$, and $c = 1$. Then

$$(a + b) + c = 1, \quad a + (b + c) = 0$$

in double precision. The second expression first rounds $b + c$ back to -10^{16} , so the 1 is lost before the final addition.

Remark 1.28 (**Summation rule**) Add numbers of similar magnitude when possible. Summing from smallest to largest, pairwise summation, or compensated summation reduces the loss of small terms. This is why library reductions may give different answers than a hand-written loop while still being more accurate.

Ex 1.24

1. Before running Python, use Thm 1.10 to predict $(1e16 + 1.0) - 1e16$.
2. `a=1e16, b=1`. Find `(a+b)-a` in Python. Explain result.
3. Find a, b, c where `a*(b+c) != a*b + a*c`.
4. Before running Python, predict the results of summing $[10^{16}, 1, -10^{16}]$ in the listed order and in the order $[10^{16}, -10^{16}, 1]$.
5. Compare naive summation, pairwise summation, and Kahan summation on a list containing one large number and many small numbers. Which method best preserves the small terms?

1.7 Quantifying Approximation Error

1.7.1 Absolute and Relative Errors

- **Absolute Error:** $|\hat{x} - x|$.
- **Relative Error:** $\varepsilon = \frac{|\hat{x} - x|}{|x|}$. This is the standard measure for precision in numerical computing.

Absolute error measures the size of the mistake in the original units. Relative error measures the size of the mistake compared to the quantity being computed. In numerical computing, relative error is usually more informative: an error of 10^{-6} is excellent for a number of size 10^3 and terrible for a number of size 10^{-9} .

Remark 1.29 (Comparing floating-point numbers) Avoid testing computed real numbers with exact equality. Use a tolerance such as

$$|\hat{x} - x| \leq \text{atol} + \text{rtol}|x|.$$

This is the logic behind `np.isclose`. Absolute tolerance handles values near zero; relative tolerance handles scale.

Ex 1.25

1. $\sqrt{2} \approx 1.414$. Errors?
2. Relative error of product $\hat{x}\hat{y}$? (Sum of individual relative errors).

Def 1.25 (Catastrophic Cancellation) The most dangerous error in numerical computing is **catastrophic cancellation**. This occurs when two nearly equal numbers are subtracted, causing the most significant digits to cancel out and leaving only the rounding errors as the "leading" digits.

- Relative error bound: $\varepsilon_{\text{rel}} \leq u \cdot \frac{x+y}{x-y}$.
- As $x \rightarrow y$, the amplification factor $\frac{x+y}{x-y}$ tends to infinity.

Proof *Proof.* Let $\hat{x} = x(1 + \varepsilon_x)$ and $\hat{y} = y(1 + \varepsilon_y)$ with $|\varepsilon_x|, |\varepsilon_y| \leq u$. The error in the subtraction is:

$$\begin{aligned} (\hat{x} - \hat{y}) - (x - y) &= x(1 + \varepsilon_x) - y(1 + \varepsilon_y) - x + y \\ &= x\varepsilon_x - y\varepsilon_y. \end{aligned}$$

The relative error is bounded by:

$$\begin{aligned} \frac{|x\varepsilon_x - y\varepsilon_y|}{x - y} &\leq \frac{x|\varepsilon_x| + y|\varepsilon_y|}{x - y} \\ &\leq \frac{xu + yu}{x - y} \\ &= u \cdot \frac{x + y}{x - y}. \end{aligned}$$

As $x \rightarrow y$, the denominator $x - y \rightarrow 0$, causing the relative error to explode. $\square$

Remark 1.30 **Numerical Stability Principle:** Rounding error is an inevitable consequence of finite precision. However, catastrophic cancellation is an algorithmic flaw that can often be avoided by mathematically rearranging the problem.

Example **(Stabilizing Cancellation)** $f(x) = \sqrt{x+1} - \sqrt{x}$ for $x = 10^8$.

- **Direct:** $10000.000000005 - 10000 \approx 0$ (Catastrophic).
- **Stable:** $\frac{1}{\sqrt{x+1} + \sqrt{x}} \approx 5 \times 10^{-5}$.

Example **(Quadratic formula)** The roots of $ax^2 + bx + c = 0$ are

$$x_{\pm} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}.$$

If $b^2 \gg 4ac$, one numerator subtracts nearly equal numbers. A stable strategy computes the root with no cancellation first, then uses $x_1 x_2 = c/a$ to recover the other root.

Remark 1.31 **(Use stable library functions)** Functions such as `log1p(x)`, `expm1(x)`, and `hypot(x, y)` implement stable rearrangements for common cancellation and overflow patterns. Prefer them over hand-expanded formulas when the inputs may be small, large, or nearly cancelling.

Ex 1.26 Stabilize:

1. Use Def 1.25 to explain why $(1 - \cos x)/x^2$ is unstable for small x , then derive a stable form.
2. Use Def 1.25 to identify which root of $x^2 - 10^8x + 1 = 0$ is unstable in the quadratic formula, then compute it using $x_1x_2 = c/a$.
3. Use the Taylor series of e^x to explain why $e^x - 1$ loses relative accuracy for small x . Compare with `np.expm1`.

1.7.2 Error Analysis Frameworks

- **Forward Error Analysis:** Tracks the accumulation of error from inputs to outputs.
- **Backward Error Analysis:** Determines the input perturbation that would make the computed result exact.
- **Backward Stability:** An algorithm is backward stable if the backward error is $O(u)$, where u is the unit roundoff.

Forward error asks whether the computed answer is close to the exact answer. Backward error asks whether the computed answer is the exact answer to a nearby problem. Backward error is often easier to prove and more useful in practice: if the data are uncertain at the level of the backward error, the algorithm has not introduced a meaningful additional error.

Def 1.26 (**Condition Number (κ)**) Inherent sensitivity of the problem. Forward Error $\leq \kappa \cdot$ Backward Error.

Remark 1.32 (**Stability vs. conditioning**) Conditioning is a property of the mathematical problem. Stability is a property of the algorithm. A stable algorithm can still return a poor answer on an ill-conditioned problem, because the problem itself amplifies small perturbations.

Ex 1.27

1. $\kappa(A) = 10^8$, Backward Error = 10^{-15} . Max forward error?
2. Prove BE for linear systems: $(\mathbf{A} + \mathbf{E})\hat{\mathbf{x}} = \mathbf{b}$ for $\mathbf{E} = -\mathbf{r}\hat{\mathbf{x}}^T/\|\hat{\mathbf{x}}\|^2$.

Proof *Proof.* (1): Substituting $\mathbf{E}$ into $(\mathbf{A} + \mathbf{E})\hat{\mathbf{x}}$:

$$\begin{aligned}
 (\mathbf{A} + \mathbf{E})\hat{\mathbf{x}} &= \mathbf{A}\hat{\mathbf{x}} + \mathbf{E}\hat{\mathbf{x}} \\
 &= \mathbf{A}\hat{\mathbf{x}} - \frac{\mathbf{r}\hat{\mathbf{x}}^T}{\|\hat{\mathbf{x}}\|^2}\hat{\mathbf{x}} \\
 &= \mathbf{A}\hat{\mathbf{x}} - \mathbf{r}\frac{\hat{\mathbf{x}}^T\hat{\mathbf{x}}}{\|\hat{\mathbf{x}}\|^2} \\
 &= \mathbf{A}\hat{\mathbf{x}} - \mathbf{r}.
 \end{aligned}$$

Since $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$, we have $\mathbf{A}\hat{\mathbf{x}} - (\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{b}$.

(2): For the relative error, using the rank-1 norm property $\|\mathbf{r}\mathbf{v}^T\|_2 = \|\mathbf{r}\|_2\|\mathbf{v}\|_2$:

$$\begin{aligned}\|\mathbf{E}\|_2 &= \left\| \frac{\mathbf{r}\hat{\mathbf{x}}^T}{\|\hat{\mathbf{x}}\|_2^2} \right\|_2 \\ &= \frac{\|\mathbf{r}\|_2\|\hat{\mathbf{x}}\|_2}{\|\hat{\mathbf{x}}\|_2^2} = \frac{\|\mathbf{r}\|_2}{\|\hat{\mathbf{x}}\|_2}.\end{aligned}$$

Dividing by $\|\mathbf{A}\|_2$ gives the result:

$$\frac{\|\mathbf{E}\|_2}{\|\mathbf{A}\|_2} = \frac{\|\mathbf{r}\|_2}{\|\mathbf{A}\|_2\|\hat{\mathbf{x}}\|_2}.$$

□

1.8 Python and NumPy Primer

Condensed reference for the toolset used throughout these notes.

1.8.1 Arrays and Basic Operations

Def 1.27 (NumPy Arrays) The `ndarray` is the fundamental structure.

- **Creation:** `np.array([1, 2])`, `np.zeros((m, n))`, `np.random.randn(m, n)`.
- **Indexing:** 0-indexed. `A[i, j]` for entries; `A[i, :]` for rows; `A[:, j]` for columns.

Remark 1.33 (Shapes and Rank-1 Arrays) `A.shape` returns `(m, n)`. Note that a 1D array has shape `(n,)`, which is **not** equivalent to a column vector `(n, 1)`. These “rank-1 arrays” are a common source of broadcasting bugs. Use `x[:, None]` or `x.reshape(-1, 1)` to convert to 2D.

Ex 1.28

1. Create 4×4 identity matrix. Verify `shape` and `dtype`.
2. Extract row 2 and column 3 from a random 3×3 `A`.
3. Confirm that `np.zeros((3,))` fails where a `(3, 1)` column vector is required in a matrix-vector product.

1.8.2 Linear Algebra Operations

Core routines in `numpy.linalg`:

Function	Computes
<code>A @ B</code>	Matrix product $\mathbf{AB}$
<code>np.linalg.solve(A, b)</code>	Solve $\mathbf{Ax} = \mathbf{b}$ ($O(n^3)$)
<code>np.linalg.inv(A)</code>	Matrix inverse (Avoid; use <code>solve</code>)
<code>np.linalg.norm(x)</code>	Euclidean norm $\ \mathbf{x}\ _2$
<code>np.linalg.eig(A)</code>	Eigenvalues and eigenvectors
<code>np.linalg.svd(A)</code>	Singular Value Decomposition
<code>np.linalg.qr(A)</code>	QR Factorization
<code>np.linalg.cond(A)</code>	Condition number $\kappa_2(\mathbf{A})$

Remark 1.34 (Performance Law: Vectorization) Explicit for loops in Python are slow. Push heavy lifting into NumPy/BLAS routines by **vectorizing** operations. For example, use `A @ B` instead of triple-nested loops.

Remark 1.35 (Stability: solve vs inv) `np.linalg.solve(A, b)` is faster and more stable than `np.linalg.inv(A) @ b`. Explicit inversion is almost never desirable in scientific computation.

Remark 1.36 (Computed equality) Use `np.allclose(x, y)` or `np.isclose(x, y)` for computed floating-point quantities. Exact equality is appropriate for integers, booleans, and deliberately exact structural checks.

Remark 1.37 (Sparse matrices) Large discretized PDE and graph problems should usually use `scipy.sparse`, not dense NumPy arrays. Use `scipy.sparse.linalg` for sparse solves, eigenvalue computations, CG, and GMRES.

Ex 1.29

1. Solve a 2×2 system and verify the residual $\|\mathbf{Ax} - \mathbf{b}\|$.
2. Profile `solve` vs `inv @ b` for $n = 1000$.
3. Compare `np.linalg.eig` and `np.linalg.eigh` (optimized for symmetric matrices).

1.8.3 Key Idioms

- **Element-wise:** `*` and `**` are entry-wise. Matrix powers use `np.linalg.matrix_power`.
- **Broadcasting:** Automatic dimension expansion. `A * v` scales columns of `A` by entries of `v`.
- **Aliases, Views, Copies:** `B = A` makes a second name for the same array. Slices are usually views. Use `B = A.copy()` for independent data.
- **Sparse construction:** use `scipy.sparse.diags` for banded matrices and `scipy.sparse.csr_matrix` for efficient arithmetic.

Ex 1.30

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}$. Compare $\mathbf{A} * \mathbf{A}$ and $\mathbf{A} @ \mathbf{A}$.
2. Demonstrate the “view pitfall” by modifying a slice of a matrix.
3. Use broadcasting to perform diagonal scaling $\mathbf{AD}$ without constructing $\mathbf{D} = \text{diag}(d)$.

2 Direct Methods for Linear Systems

2.1 Linearity

Linear systems are the basic computational problem in scientific computing. Even when the original problem is nonlinear, algorithms often reduce it to a sequence of linear systems by linearizing the problem near the current estimate.

A matrix equation $\mathbf{A}\mathbf{x} = \mathbf{b}$ asks whether the data vector $\mathbf{b}$ can be produced by combining the columns of $\mathbf{A}$. The unknown vector $\mathbf{x}$ contains the weights in that combination. Existence, uniqueness, and numerical difficulty are all controlled by the geometry of those columns.

2.1.1 Linear Systems

A linear system is a collection of linear equations involving the same set of variables. In matrix-vector notation, we package these equations into a single compact form that is amenable to both theoretical analysis and high-performance computation.

Def 2.1 (Linear System) The matrix-vector form of a linear system is $\mathbf{A}\mathbf{x} = \mathbf{b}$, where:

- $\mathbf{A} \in \mathbb{R}^{m \times n}$ is the coefficient matrix representing the system's structure.
- $\mathbf{x} \in \mathbb{R}^n$ is the vector of unknown variables we wish to determine.
- $\mathbf{b} \in \mathbb{R}^m$ is the right-hand side vector, often representing observed data or a source term.

Remark 2.1 (Shape discipline) If $\mathbf{A} \in \mathbb{R}^{m \times n}$, then $\mathbf{x}$ must have n entries and $\mathbf{b}$ must have m entries. The number of columns is the number of unknowns. The number of rows is the number of equations or measurements.

Example (Two equations) The system

$$x + 2y = 3, \quad 3x - y = 1$$

can be written as

$$\begin{pmatrix} 1 & 2 \\ 3 & -1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 3 \\ 1 \end{pmatrix}.$$

The first row encodes the first equation; the second row encodes the second equation.

Ex 2.1

1. Convert $\{x + 2y = 3, 3x - y = 1\}$ to the form in Def 2.1.

2. Check the dimensions of $\mathbf{A}$, $\mathbf{x}$, and $\mathbf{b}$ before solving. Which part of Def 2.1 determines each shape?
3. Solve in NumPy via `np.linalg.solve(A, b)`. Verify with the residual from Def 2.2.

2.1.2 Rows, Columns, and Geometry

There are two complementary ways to read $\mathbf{Ax} = \mathbf{b}$.

- **Row view:** Each row is one linear equation. In two or three dimensions, each equation describes a line, plane, or hyperplane. Solving the system means finding their intersection.
- **Column view:** The product $\mathbf{Ax}$ is a weighted sum of the columns of $\mathbf{A}$. Solving the system means expressing $\mathbf{b}$ as a linear combination of those columns.

Remark 2.2 (The Column Perspective) If $\mathbf{A} = [\mathbf{a}_1, \mathbf{a}_2, \dots, \mathbf{a}_n]$, then

$$\mathbf{Ax} = x_1\mathbf{a}_1 + x_2\mathbf{a}_2 + \dots + x_n\mathbf{a}_n.$$

The system $\mathbf{Ax} = \mathbf{b}$ is consistent exactly when $\mathbf{b}$ lies in the span of the columns of $\mathbf{A}$.

Thm 2.1 (Superposition Principle) If $\mathbf{Ax}_1 = \mathbf{b}_1$ and $\mathbf{Ax}_2 = \mathbf{b}_2$, then for any scalars α, β :

$$\mathbf{A}(\alpha\mathbf{x}_1 + \beta\mathbf{x}_2) = \alpha\mathbf{b}_1 + \beta\mathbf{b}_2.$$

Implication: Solutions to $\mathbf{Ax} = \mathbf{b}$ can be decomposed and recombined linearly.

Ex 2.2

1. Use Thm 2.1 to solve $\mathbf{Ax} = 3\mathbf{b}_1 - 2\mathbf{b}_2$ given only the solutions $\mathbf{x}_1, \mathbf{x}_2$ for $\mathbf{b}_1, \mathbf{b}_2$.
2. How does `np.linalg.solve(A, B)` for a matrix $\mathbf{B}$ exploit this?

2.1.3 Invertibility and Existence

For square systems ($\mathbf{A} \in \mathbb{R}^{n \times n}$), the most critical question is whether a unique solution exists for any given right-hand side. This property is known as invertibility or nonsingularity.

For rectangular systems, the same questions still matter but the answer changes.

- **Square:** $m = n$. A unique solution may exist.
- **Overdetermined:** $m > n$. There are more equations than unknowns; usually no exact solution exists, so we solve a least squares problem.

- **Underdetermined:** $m < n$. There are fewer equations than unknowns; if a solution exists, it is usually not unique.

Thm 2.2 (Equivalences for Invertibility) The following are equivalent for $\mathbf{A} \in \mathbb{R}^{n \times n}$:

1. $\mathbf{A}$ is **invertible** ($\det(\mathbf{A}) \neq 0$).
2. $\mathbf{A}\mathbf{x} = \mathbf{b}$ has a **unique solution** for every $\mathbf{b}$.
3. $\mathbf{A}\mathbf{x} = \mathbf{0}$ has only the **trivial solution** ($\mathbf{x} = \mathbf{0}$).
4. $\text{rank}(\mathbf{A}) = n$ (Full Rank).
5. Columns of $\mathbf{A}$ are **linearly independent**.

Remark 2.3 (Determinants are not algorithms) The determinant is useful theoretically, but numerical codes do not test invertibility by computing $\det(\mathbf{A})$. Rank, singular values, and condition numbers are more informative and more stable.

Ex 2.3

1. Determine invertibility using Thm 2.2: $\mathbf{A} = \begin{pmatrix} 2 & -1 \\ 4 & -2 \end{pmatrix}$, $\mathbf{B} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & 2 & 0 \\ 0 & 0 & 3 \end{pmatrix}$.
2. Check consistency of $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 3 & 6 \end{pmatrix}$, $\mathbf{b} = \begin{pmatrix} 1 \\ 4 \end{pmatrix}$ using the column perspective in Remark 2.2 and `np.linalg.matrix_rank`.

2.1.4 Residuals and Approximate Solutions

Def 2.2 (Residual) For an approximate solution $\hat{\mathbf{x}}$ to $\mathbf{A}\mathbf{x} = \mathbf{b}$, the residual is

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}.$$

It measures how well $\hat{\mathbf{x}}$ satisfies the equations.

Remark 2.4 (Residual vs. error) The error is $\hat{\mathbf{x}} - \mathbf{x}^*$, where $\mathbf{x}^*$ is the exact solution. The residual is computable; the error usually is not. A small residual means the equations are nearly satisfied, but it does not always mean the solution is accurate. Ill-conditioned matrices can turn small residuals into large errors.

Example (Inconsistent system) The equations

$$x + y = 1, \quad x + y = 3$$

cannot both be true. In matrix form, $\mathbf{b}$ is not in the column space of $\mathbf{A}$. The best we can do is choose $\hat{\mathbf{x}}$ so that the residual $\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ is small in a chosen norm. This is the least squares problem.

2.1.5 Linear Combinations and Bases

Def 2.3 (**Linear Independence**) Vectors $\{\mathbf{x}_1, \dots, \mathbf{x}_k\}$ are independent if $\sum \alpha_i \mathbf{x}_i = \mathbf{0} \Rightarrow \alpha_i = 0$ for all i .

Def 2.4 (**Basis**) A linearly independent set that spans the space. The number of vectors in any basis is the **dimension**.

2.1.6 Rank

The rank of a matrix is the number of independent directions its columns span. It measures how many genuinely different constraints or directions are present after removing redundancy.

Def 2.5 (**Rank**) The rank, $\text{rank}(\mathbf{A})$, is the dimension of the column space. It is always true that $\text{rank}(\mathbf{A}) \leq \min(m, n)$.

- **Full Rank:** The matrix has the maximum possible rank, $\text{rank}(\mathbf{A}) = \min(m, n)$.
- **Rank Deficient (Singular):** A square matrix with $\text{rank}(\mathbf{A}) < n$. Such systems do not have unique solutions.

Remark 2.5 (**Rank and solution structure**) For $\mathbf{A} \in \mathbb{R}^{m \times n}$:

- Full column rank ($\text{rank}(\mathbf{A}) = n$) means the columns are independent.
- Full row rank ($\text{rank}(\mathbf{A}) = m$) means the rows are independent.
- Rank deficiency means there is redundancy: some row or column information can be produced from the rest.

Remark 2.6 (**Numerical Rank**) In code, do not use exact row-reduction to find rank; it is numerically unstable. Use `np.linalg.matrix_rank`, which estimates rank from the singular values. A matrix can be mathematically full rank but numerically rank deficient if one singular value is tiny compared to the others.

Thm 2.3 (**Rank-Nullity**) For $\mathbf{A} \in \mathbb{R}^{m \times n}$: $\text{rank}(\mathbf{A}) + \text{nullity}(\mathbf{A}) = n$.

- **Nullity:** Dimension of the null space $\{\mathbf{x} : \mathbf{A}\mathbf{x} = \mathbf{0}\}$.

Remark 2.7 (**General solution form**) If $\mathbf{A}\mathbf{x} = \mathbf{b}$ is consistent and $\mathbf{x}_p$ is one particular solution, then every solution has the form

$$\mathbf{x} = \mathbf{x}_p + \mathbf{z}, \quad \mathbf{z} \in \mathcal{N}(\mathbf{A}).$$

The null space contains the directions you can add without changing $\mathbf{A}\mathbf{x}$.

Ex 2.4

1. Use Thm 2.3 to explain why a “wide” matrix ($m < n$) must have a non-trivial null space.
2. Find the general solution to a consistent 2×3 system in the form $\mathbf{x} = \mathbf{x}_p + \mathbf{z}$, where $\mathbf{z} \in \mathcal{N}(\mathbf{A})$.
3. Check directly that adding a null-space vector leaves $\mathbf{Ax}$ unchanged.

2.1.7 The Four Fundamental Subspaces

The structure of any linear operator $\mathbf{A} \in \mathbb{R}^{m \times n}$ with rank r is completely characterized by four subspaces. Understanding these spaces is essential for solving least squares problems and understanding the behavior of iterative solvers.

1. **Column Space** ($\mathcal{R}(\mathbf{A})$): $\mathbb{R}^m$, $\dim = r$. Span of columns.
2. **Null Space** ($\mathcal{N}(\mathbf{A})$): $\mathbb{R}^n$, $\dim = n - r$. Solutions to $\mathbf{Ax} = \mathbf{0}$.
3. **Row Space** ($\mathcal{R}(\mathbf{A}^T)$): $\mathbb{R}^n$, $\dim = r$. Span of rows.
4. **Left Null Space** ($\mathcal{N}(\mathbf{A}^T)$): $\mathbb{R}^m$, $\dim = m - r$. Solutions to $\mathbf{A}^T \mathbf{y} = \mathbf{0}$.

Thm 2.4 (Orthogonal Complements)

- $\mathcal{R}(\mathbf{A}^T) \perp \mathcal{N}(\mathbf{A})$ in $\mathbb{R}^n$.
- $\mathcal{R}(\mathbf{A}) \perp \mathcal{N}(\mathbf{A}^T)$ in $\mathbb{R}^m$.

Solvability: $\mathbf{Ax} = \mathbf{b}$ has a solution iff $\mathbf{b} \in \mathcal{R}(\mathbf{A})$, i.e., $\mathbf{b} \perp \mathcal{N}(\mathbf{A}^T)$.

Ex 2.5

1. For $\mathbf{A} = \begin{pmatrix} 1 & 2 & 3 \\ 2 & 4 & 6 \end{pmatrix}$, use Def 2.5 and Thm 2.3 to find dimensions and bases for all four spaces.
2. Verify the two orthogonality statements in Thm 2.4.
3. Verify $\mathbf{b} = (1, 3)^T$ is not in the column space. What is the projection of $\mathbf{b}$ onto $\mathcal{R}(\mathbf{A})$?

2.2 LU Factorization

The solution of a general square linear system $\mathbf{Ax} = \mathbf{b}$ is one of the central problems of computational science. Although Cramer's rule and explicit matrix inversion furnish closed-form expressions, neither approach is computationally viable. In practice, direct solvers rely on matrix factorizations that reduce the original system to a pair of triangular systems. The most fundamental of these is the LU factorization, which provides the matrix formulation of Gaussian elimination.

The principle behind triangular factorization is separation of concerns. The coefficient matrix $\mathbf{A}$ is factored into a unit lower triangular factor $\mathbf{L}$ and an upper triangular factor $\mathbf{U}$. Computing this factorization requires $O(n^3)$ operations, but once $\mathbf{L}$ and $\mathbf{U}$ have been computed and stored, the system $\mathbf{Ax} = \mathbf{b}$ can be solved for any given right-hand side $\mathbf{b}$ via forward and backward substitution in only $O(n^2)$ operations.

2.2.1 Gaussian Elimination as Matrix Factorization

Triangular systems are straightforward to solve because their variables can be computed sequentially. When $\mathbf{U}$ is upper triangular, the last equation contains only the single unknown x_n . Once x_n is known, the second-to-last equation contains only x_{n-1} , and this backward substitution continues until all components are determined. Gaussian elimination transforms the original system $\mathbf{Ax} = \mathbf{b}$ into an equivalent upper triangular system by systematically subtracting multiples of each row from the rows beneath it.

Def 2.6 (LU Factorization) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$. An LU factorization of $\mathbf{A}$ is a decomposition of the form

$$\mathbf{A} = \mathbf{LU},$$

where $\mathbf{L} \in \mathbb{R}^{n \times n}$ is unit lower triangular and $\mathbf{U} \in \mathbb{R}^{n \times n}$ is upper triangular. The factors have the explicit structure

$$\mathbf{L} = \begin{pmatrix} 1 & 0 & 0 & \dots & 0 \\ \ell_{21} & 1 & 0 & \dots & 0 \\ \ell_{31} & \ell_{32} & 1 & \dots & 0 \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ \ell_{n1} & \ell_{n2} & \ell_{n3} & \dots & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} u_{11} & u_{12} & u_{13} & \dots & u_{1n} \\ 0 & u_{22} & u_{23} & \dots & u_{2n} \\ 0 & 0 & u_{33} & \dots & u_{3n} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \dots & u_{nn} \end{pmatrix}.$$

Thm 2.5 (Elimination Matrices and the Structure of L) Let $\mathbf{A}^{(0)} = \mathbf{A}$. At step k of Gaussian elimination, assume the pivot entry $a_{kk}^{(k-1)}$ is nonzero. The subdiagonal entries in column k are annihilated by applying the elementary elimination matrix

$$\mathbf{M}_k = \mathbf{I} - \boldsymbol{\ell}_k \mathbf{e}_k^T,$$

where $\ell_k = (0 \ \dots \ 0 \ \ell_{k+1,k} \ \dots \ \ell_{n,k})^T$ contains the multipliers $\ell_{ik} = a_{ik}^{(k-1)} / a_{kk}^{(k-1)}$. After $n-1$ elimination steps, the upper triangular factor is given by $\mathbf{U} = \mathbf{M}_{n-1} \dots \mathbf{M}_1 \mathbf{A}$. The lower triangular factor is the inverse of the accumulated elimination operations, which satisfies

$$\mathbf{L} = \mathbf{M}_1^{-1} \mathbf{M}_2^{-1} \dots \mathbf{M}_{n-1}^{-1} = \mathbf{I} + \sum_{k=1}^{n-1} \ell_k \mathbf{e}_k^T.$$

Consequently, the entries of $\mathbf{L}$ below the main diagonal are precisely the multipliers computed during Gaussian elimination.

Proof *Proof.* We first observe that the inverse of an elementary elimination matrix $\mathbf{M}_k = \mathbf{I} - \ell_k \mathbf{e}_k^T$ has the closed form $\mathbf{M}_k^{-1} = \mathbf{I} + \ell_k \mathbf{e}_k^T$. This identity follows directly from the fact that the first k entries of ℓ_k are zero, which implies $\mathbf{e}_k^T \ell_k = 0$ and therefore

$$(\mathbf{I} - \ell_k \mathbf{e}_k^T)(\mathbf{I} + \ell_k \mathbf{e}_k^T) = \mathbf{I} - \ell_k \mathbf{e}_k^T + \ell_k \mathbf{e}_k^T - \ell_k (\mathbf{e}_k^T \ell_k) \mathbf{e}_k^T = \mathbf{I}.$$

Next, consider the product of two successive inverse matrices $\mathbf{M}_j^{-1} \mathbf{M}_k^{-1}$ with $j < k$. Because $\mathbf{e}_j^T \ell_k = 0$, the cross-term vanishes:

$$(\mathbf{I} + \ell_j \mathbf{e}_j^T)(\mathbf{I} + \ell_k \mathbf{e}_k^T) = \mathbf{I} + \ell_j \mathbf{e}_j^T + \ell_k \mathbf{e}_k^T + \ell_j (\mathbf{e}_j^T \ell_k) \mathbf{e}_k^T = \mathbf{I} + \ell_j \mathbf{e}_j^T + \ell_k \mathbf{e}_k^T.$$

By induction, the product $\mathbf{M}_1^{-1} \dots \mathbf{M}_{n-1}^{-1}$ simply places each column of multipliers into the corresponding column of the identity matrix without modifying any previously computed entries. Because $\mathbf{U} = \mathbf{M}_{n-1} \dots \mathbf{M}_1 \mathbf{A}$, multiplying both sides on the left by $\mathbf{L}$ yields $\mathbf{A} = \mathbf{L}\mathbf{U}$. $\square$

Thm 2.6 (Existence and Uniqueness of the LU Factorization) A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ has a unique LU factorization with unit lower triangular $\mathbf{L}$ if and only if every leading principal submatrix $\mathbf{A}_k = \mathbf{A}[1 : k, 1 : k]$ is nonsingular for each $k = 1, \dots, n-1$.

Remark 2.8 (Failure without pivoting) If any pivot element $a_{kk}^{(k-1)}$ is zero, the algorithm encounters a division by zero and fails, even when the original matrix $\mathbf{A}$ is nonsingular. Even if a pivot is nonzero, an extraordinarily small pivot will create large multipliers, causing disastrous growth of rounding errors in subsequent arithmetic operations.

2.2.2 Triangular Solves

Once the factorization $\mathbf{A} = \mathbf{L}\mathbf{U}$ has been computed, solving the linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$ reduces to solving two triangular systems in sequence. We introduce the auxiliary vector $\mathbf{y} = \mathbf{U}\mathbf{x}$ and first solve $\mathbf{L}\mathbf{y} = \mathbf{b}$ by forward substitution. Having obtained $\mathbf{y}$, we then solve $\mathbf{U}\mathbf{x} = \mathbf{y}$ by backward substitution to recover the solution vector $\mathbf{x}$.

Def 2.7 (Forward and Backward Substitution) Let $\mathbf{L} \in \mathbb{R}^{n \times n}$ be a unit lower triangular matrix and let $\mathbf{U} \in \mathbb{R}^{n \times n}$ be a nonsingular upper triangular matrix. The forward substitution

algorithm computes the solution to $\mathbf{L}\mathbf{y} = \mathbf{b}$ via the recurrence

$$y_1 = b_1,$$

$$y_i = b_i - \sum_{j=1}^{i-1} \ell_{ij} y_j, \quad i = 2, 3, \dots, n.$$

The backward substitution algorithm computes the solution to $\mathbf{U}\mathbf{x} = \mathbf{y}$ via the recurrence

$$x_n = \frac{y_n}{u_{nn}},$$

$$x_i = \frac{1}{u_{ii}} \left(y_i - \sum_{j=i+1}^n u_{ij} x_j \right), \quad i = n-1, n-2, \dots, 1.$$

Example (Step-by-step triangular solution) Consider solving the system $\mathbf{A}\mathbf{x} = \mathbf{b}$ with

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 6 & 5 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 4 \\ 16 \end{pmatrix}.$$

The elimination multiplier for the second row is $\ell_{21} = 6/2 = 3$. Subtracting three times the first row from the second row produces the upper triangular matrix with diagonal entry $u_{22} = 5 - 3(1) = 2$. The factors are

$$\mathbf{L} = \begin{pmatrix} 1 & 0 \\ 3 & 1 \end{pmatrix}, \quad \mathbf{U} = \begin{pmatrix} 2 & 1 \\ 0 & 2 \end{pmatrix}.$$

We first solve the lower triangular system $\mathbf{L}\mathbf{y} = \mathbf{b}$:

$$y_1 = 4, \quad 3(4) + y_2 = 16 \implies y_2 = 4, \quad \text{so } \mathbf{y} = \begin{pmatrix} 4 \\ 4 \end{pmatrix}.$$

Next, we solve the upper triangular system $\mathbf{U}\mathbf{x} = \mathbf{y}$:

$$2x_2 = 4 \implies x_2 = 2, \quad 2x_1 + (2) = 4 \implies x_1 = 1.$$

The computed solution is $\mathbf{x} = (1, 2)^T$. Direct matrix multiplication verifies that $\mathbf{A}\mathbf{x} = (4, 16)^T = \mathbf{b}$.

2.2.3 Pivoting and Numerical Stability

In floating-point arithmetic, unpivoted Gaussian elimination is numerically unstable whenever a pivot element is small relative to the remaining entries in its column. Dividing by a small pivot generates large multipliers, which contaminate the remaining submatrix with severe cancellation and roundoff errors. To guarantee numerical stability, we use partial pivoting: at each step k , we identify the element of largest absolute value in the active column on or below the diagonal, and swap its row with row k before eliminating.

Def 2.8 (PA = LU Factorization with Partial Pivoting) Gaussian elimination with partial pivoting produces the factorization

$$\mathbf{PA} = \mathbf{LU},$$

where $\mathbf{P} \in \mathbb{R}^{n \times n}$ is a permutation matrix representing the row swaps, $\mathbf{L} \in \mathbb{R}^{n \times n}$ is unit lower triangular with all entries bounded by $|\ell_{ij}| \leq 1$, and $\mathbf{U} \in \mathbb{R}^{n \times n}$ is upper triangular.

Remark 2.9 (Permutation matrices in computation) A permutation matrix $\mathbf{P}$ is formed by rearranging the rows of the identity matrix. It is an orthogonal matrix, which implies that $\mathbf{P}^{-1} = \mathbf{P}^T$. In practical software, $\mathbf{P}$ is never formed as a full two-dimensional array. Instead, the sequence of row exchanges is recorded in an integer vector of length n , which requires minimal memory and avoids unnecessary arithmetic operations.

Example (Catastrophic failure of unpivoted elimination) Consider the nonsingular matrix

$$\mathbf{A} = \begin{pmatrix} 10^{-16} & 1 \\ 1 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}.$$

If we compute the LU factorization without pivoting in standard double precision arithmetic, the multiplier is $\ell_{21} = 10^{16}$. The second diagonal entry of $\mathbf{U}$ becomes

$$u_{22} = \text{fl}(1 - 10^{16} \cdot 1) = -10^{16},$$

because the number 1 is lost in floating-point absorption. Solving $\mathbf{U}\mathbf{x} = \mathbf{y}$ then yields $x_2 = 1$ and $x_1 = 0$. The computed solution is $(0, 1)^T$, which contains a hundred percent relative error in the first component. If we instead apply partial pivoting, we swap the two rows first, yielding

$$\mathbf{PA} = \begin{pmatrix} 1 & 1 \\ 10^{-16} & 1 \end{pmatrix}.$$

The multiplier is $\ell_{21} = 10^{-16} \leq 1$, the updated entry is $u_{22} = 1 - 10^{-16} \approx 1$, and the computed solution evaluates to $(1, 1)^T$, which is accurate to machine precision.

Thm 2.7 (Gaussian Elimination with Partial Pivoting Algorithm) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$. The factorization $\mathbf{PA} = \mathbf{LU}$ is computed through the following procedure.

1. Initialize the permutation vector $\mathbf{p} = (1, 2, \dots, n)^T$.
2. For $k = 1, 2, \dots, n - 1$:
 - (a) Find the pivot row index $i^* = \arg \max_{i \geq k} |a_{ik}|$. If $|a_{i^*k}| = 0$, the matrix is singular.
 - (b) Swap rows k and i^* in $\mathbf{A}$, and swap elements p_k and p_{i^*} in $\mathbf{p}$.
 - (c) For each row $i = k+1, \dots, n$, compute the multiplier $\ell_{ik} = a_{ik}/a_{kk}$ and overwrite $a_{ik} \leftarrow \ell_{ik}$.

- (d) For each row $i = k+1, \dots, n$ and column $j = k+1, \dots, n$, update the submatrix via $a_{ij} \leftarrow a_{ij} - \ell_{ik}a_{kj}$.

Upon completion, the upper triangular part of $\mathbf{A}$ stores $\mathbf{U}$, while the strictly lower triangular part of $\mathbf{A}$ stores the subdiagonal entries of $\mathbf{L}$.

Remark 2.10 (Growth factor and backward stability) The backward stability of Gaussian elimination with partial pivoting is governed by the growth factor ρ , defined by

$$\rho = \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|}.$$

The computed solution $\hat{\mathbf{x}}$ satisfies an exact perturbed system $(\mathbf{A} + \delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b}$ with relative backward error bounded by $\|\delta\mathbf{A}\|_\infty/\|\mathbf{A}\|_\infty \leq cn^3\rho\varepsilon_{\text{mach}}$, where c is a modest constant. Although the theoretical upper bound on growth is $\rho = 2^{n-1}$, matrices exhibiting exponential growth are pathological and virtually never encountered in real scientific and engineering problems. In practice, ρ grows only moderately with n , ensuring that partial pivoting is backward stable.

2.2.4 Computational Complexity and Efficiency

Evaluating the arithmetic operation count clarifies why the LU decomposition is preferred over explicit matrix inversion. We quantify work in terms of floating-point operations, counting both additions and multiplications.

Thm 2.8 (Operation Count of LU Factorization) For a general dense matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, computing the LU factorization requires asymptotically $\frac{2}{3}n^3$ flops. Solving the resulting triangular systems requires $2n^2$ flops for each right-hand side vector $\mathbf{b}$.

Proof *Proof.* At step k of the elimination, computing the $n-k$ multipliers requires $n-k$ divisions. Updating the remaining $(n-k) \times (n-k)$ submatrix requires one multiplication and one subtraction for each entry, totaling $2(n-k)^2$ operations. Summing these operations over all steps $k = 1, \dots, n-1$ yields

$$\sum_{k=1}^{n-1} [(n-k) + 2(n-k)^2] = 2 \sum_{m=1}^{n-1} m^2 + \sum_{m=1}^{n-1} m = 2 \left(\frac{(n-1)n(2n-1)}{6} \right) + \frac{(n-1)n}{2} = \frac{2}{3}n^3 - \frac{1}{2}n^2 - \frac{1}{6}n$$

For forward substitution with unit diagonal, row i requires $i-1$ multiplications and $i-1$ subtractions, giving $\sum_{i=1}^n 2(i-1) = n^2 - n$ flops. For backward substitution, row i requires $n-i$ multiplications, $n-i$ subtractions, and one division, giving $\sum_{i=1}^n (2(n-i) + 1) = n^2$ flops. The total solve phase therefore requires $2n^2 - n \approx 2n^2$ flops. $\square$

Remark 2.11 (Amortized cost across multiple right-hand sides) When a problem requires solving $\mathbf{A}\mathbf{x}_j = \mathbf{b}_j$ for m distinct right-hand side vectors with the same matrix $\mathbf{A}$, the total computational work is $\frac{2}{3}n^3 + 2mn^2$ flops. In contrast, recomputing Gaussian elimination

from scratch for each right-hand side would require $\frac{2}{3}mn^3$ flops. Computing $\mathbf{A}^{-1}$ explicitly requires $\frac{8}{3}n^3$ flops and introduces unnecessary rounding errors, so explicit inverses should never be computed.

Ex 2.6

1. Compute the LU factorization with partial pivoting by hand for the matrix

$$\mathbf{A} = \begin{pmatrix} 1 & 2 & 2 \\ 4 & 4 & 2 \\ 4 & 6 & 4 \end{pmatrix}.$$

Explicitly write down the permutation matrix $\mathbf{P}$, the unit lower triangular factor $\mathbf{L}$, and the upper triangular factor $\mathbf{U}$, and verify that $\mathbf{PA} = \mathbf{LU}$.

2. Using the factors obtained above, solve the linear system $\mathbf{Ax} = (3, 6, 10)^T$ using forward and backward substitution.
3. Prove that the product of two unit lower triangular matrices is always unit lower triangular.
4. Consider the $n \times n$ matrix with ones on the main diagonal, minus ones strictly above the main diagonal, and ones in the final column. Show that unpivoted elimination causes the bottom-right entry to grow as 2^{n-1} .

2.3 Symmetric Positive Definite Matrices

Symmetric positive definite matrices constitute the most mathematically elegant and computationally well-behaved class of matrices in numerical linear algebra. They arise throughout computational science, including the discretization of elliptic partial differential equations, Hessian matrices in convex optimization, covariance and information matrices in statistics, and the normal equations of linear least squares.

When a matrix possesses both symmetry and positive definiteness, standard numerical algorithms simplify dramatically. Gaussian elimination requires no pivoting to ensure backward stability, memory requirements are halved because only one triangular factor must be stored, and the total computational work is reduced by fifty percent compared to the general nonsymmetric case.

2.3.1 Definitions and Characterizations

The algebraic characterization of positive definiteness is stated in terms of a quadratic form. For a symmetric matrix $\mathbf{A}$, the scalar mapping $\mathbf{x} \mapsto \mathbf{x}^T \mathbf{A} \mathbf{x}$ assigns an energy value to every direction in $\mathbb{R}^n$.

Def 2.9 (Symmetric Positive Definite Matrix) A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is symmetric positive definite if $\mathbf{A} = \mathbf{A}^T$ and the quadratic form is strictly positive for every nonzero vector $\mathbf{x} \in \mathbb{R}^n$:

$$\mathbf{x}^T \mathbf{A} \mathbf{x} > 0 \quad \text{for all } \mathbf{x} \neq \mathbf{0}.$$

If the inequality is non-strict, so that $\mathbf{x}^T \mathbf{A} \mathbf{x} \geq 0$ for all $\mathbf{x} \in \mathbb{R}^n$, the matrix is termed **symmetric positive semidefinite**.

Thm 2.9 (Equivalent Characterizations of Positive Definiteness) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a real symmetric matrix. The following statements are mathematically equivalent.

1. The matrix $\mathbf{A}$ is positive definite ($\mathbf{x}^T \mathbf{A} \mathbf{x} > 0$ for all $\mathbf{x} \neq \mathbf{0}$).
2. All eigenvalues of $\mathbf{A}$ are strictly positive: $\lambda_i(\mathbf{A}) > 0$ for each $i = 1, \dots, n$.
3. All leading principal minors are positive: $\det(\mathbf{A}[1 : k, 1 : k]) > 0$ for each $k = 1, \dots, n$.
4. There exists a unique lower triangular matrix $\mathbf{L} \in \mathbb{R}^{n \times n}$ with strictly positive diagonal entries such that $\mathbf{A} = \mathbf{L}\mathbf{L}^T$.
5. There exists a nonsingular matrix $\mathbf{W} \in \mathbb{R}^{n \times n}$ such that $\mathbf{A} = \mathbf{W}^T \mathbf{W}$.

Proof *Proof.* We demonstrate the equivalence between the quadratic form, the eigenvalues, and the existence of a factorization. By the Spectral Theorem, any real symmetric matrix has an orthogonal eigendecomposition $\mathbf{A} = \mathbf{Q}\mathbf{\Lambda}\mathbf{Q}^T$, where $\mathbf{Q}$ is orthogonal and $\mathbf{\Lambda} = \text{diag}(\lambda_1, \dots, \lambda_n)$ contains the real eigenvalues. For any vector $\mathbf{x} \neq \mathbf{0}$, let $\mathbf{y} = \mathbf{Q}^T \mathbf{x}$. Since $\mathbf{Q}$ is nonsingular, $\mathbf{y} \neq \mathbf{0}$. The quadratic form becomes

$$\mathbf{x}^T \mathbf{A} \mathbf{x} = \mathbf{x}^T \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T \mathbf{x} = \mathbf{y}^T \mathbf{\Lambda} \mathbf{y} = \sum_{i=1}^n \lambda_i y_i^2.$$

If all $\lambda_i > 0$, this sum is strictly positive for any $\mathbf{y} \neq \mathbf{0}$, proving that positive eigenvalues imply positive definiteness. Conversely, if $\lambda_j \leq 0$ for some index j , choosing $\mathbf{x} = \mathbf{q}_j$ (the j -th eigenvector) gives $\mathbf{x}^T \mathbf{A} \mathbf{x} = \lambda_j \|\mathbf{q}_j\|_2^2 \leq 0$, showing that positive definiteness implies all eigenvalues are positive.

To establish the equivalence with the factorization $\mathbf{A} = \mathbf{W}^T \mathbf{W}$, let $\mathbf{W} = \mathbf{\Lambda}^{1/2} \mathbf{Q}^T$, where $\mathbf{\Lambda}^{1/2} = \text{diag}(\sqrt{\lambda_1}, \dots, \sqrt{\lambda_n})$. This matrix is well-defined and nonsingular because each $\lambda_i > 0$. Then $\mathbf{W}^T \mathbf{W} = \mathbf{Q} \mathbf{\Lambda}^{1/2} \mathbf{\Lambda}^{1/2} \mathbf{Q}^T = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T = \mathbf{A}$. Conversely, if $\mathbf{A} = \mathbf{W}^T \mathbf{W}$ with $\mathbf{W}$ nonsingular, then for every $\mathbf{x} \neq \mathbf{0}$, we have $\mathbf{x}^T \mathbf{A} \mathbf{x} = (\mathbf{W} \mathbf{x})^T (\mathbf{W} \mathbf{x}) = \|\mathbf{W} \mathbf{x}\|_2^2 > 0$. $\square$

Remark 2.12 (Practical tests for positive definiteness) Although Sylvester's criterion on leading principal minors is theoretically elegant, computing n determinants is numerically impractical. In scientific software, the standard numerical test for positive definiteness is attempting to compute the Cholesky factorization. If the algorithm successfully completes with all positive square roots, the matrix is guaranteed to be positive definite; if a non-positive pivot is encountered, the matrix is indefinite or singular.

2.3.2 Geometry of the Energy Norm

Every symmetric positive definite matrix induces a natural inner product and a corresponding geometry on $\mathbb{R}^n$. The level sets of the quadratic form define ellipsoids whose principal axes align with the eigenvectors of $\mathbf{A}$.

Def 2.10 (The $\mathbf{A}$ -Inner Product and Energy Norm) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric positive definite. The $\mathbf{A}$ -inner product of two vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$ is defined by

$$\langle \mathbf{x}, \mathbf{y} \rangle_{\mathbf{A}} = \mathbf{x}^T \mathbf{A} \mathbf{y}.$$

The induced norm, known as the $\mathbf{A}$ -norm or energy norm, is given by

$$\|\mathbf{x}\|_{\mathbf{A}} = \sqrt{\mathbf{x}^T \mathbf{A} \mathbf{x}}.$$

The unit sphere in this norm, $\{\mathbf{x} \in \mathbb{R}^n : \|\mathbf{x}\|_{\mathbf{A}} = 1\}$, forms an n -dimensional ellipsoid whose semi-axis lengths are $1/\sqrt{\lambda_i}$, oriented along the eigenvectors $\mathbf{q}_i$ of $\mathbf{A}$.

Example (Geometric interpretation of the energy ellipsoid) Consider the diagonal positive definite matrix

$$\mathbf{A} = \begin{pmatrix} 4 & 0 \\ 0 & 1 \end{pmatrix}.$$

The unit sphere in the $\mathbf{A}$ -norm satisfies the equation $4x_1^2 + x_2^2 = 1$. This describes an ellipse in the plane with semi-major axis of length 1 along the x_2 -direction and semi-minor axis of length $1/2$ along the x_1 -direction. The direction corresponding to the larger eigenvalue $\lambda_1 = 4$ is geometrically stiffer, meaning that moving along x_1 incurs a higher energy penalty than moving along x_2 .

2.3.3 The Cholesky Factorization Algorithm

Because an SPD matrix satisfies $\mathbf{A} = \mathbf{A}^T$, the lower and upper triangular factors in Gaussian elimination must mirror each other: $\mathbf{U} = \mathbf{L}^T$ after accounting for diagonal scalings. The Cholesky algorithm computes this single triangular factor directly.

Thm 2.10 (Cholesky Factorization) Every symmetric positive definite matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ has a unique factorization of the form

$$\mathbf{A} = \mathbf{L} \mathbf{L}^T,$$

where $\mathbf{L} \in \mathbb{R}^{n \times n}$ is lower triangular with strictly positive diagonal entries $\ell_{jj} > 0$.

Proof *Proof.* We derive the recursive formulas for the entries of $\mathbf{L}$ by equating entry (i, j) of $\mathbf{A}$ with the product $\mathbf{L} \mathbf{L}^T$. Because $\mathbf{L}$ is lower triangular, the product yields

$$a_{ij} = \sum_{k=1}^n \ell_{ik} (\mathbf{L}^T)_{kj} = \sum_{k=1}^{\min(i,j)} \ell_{ik} \ell_{jk}.$$

For the diagonal entry where $i = j$, the equation becomes

$$a_{jj} = \sum_{k=1}^{j-1} \ell_{jk}^2 + \ell_{jj}^2 \implies \ell_{jj} = \sqrt{a_{jj} - \sum_{k=1}^{j-1} \ell_{jk}^2}.$$

Because $\mathbf{A}$ is positive definite, the quantity under the radical is strictly positive at every step. For the strictly subdiagonal entries in column j where $i > j$, we obtain

$$a_{ij} = \sum_{k=1}^{j-1} \ell_{ik} \ell_{jk} + \ell_{ij} \ell_{jj} \implies \ell_{ij} = \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \ell_{ik} \ell_{jk} \right).$$

Evaluating these formulas column by column for $j = 1, \dots, n$ computes the factor $\mathbf{L}$ uniquely. $\square$

Thm 2.11 (The Cholesky Algorithm) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a symmetric positive definite matrix. The lower triangular factor $\mathbf{L}$ is computed via the following column-oriented algorithm.

1. For $j = 1, 2, \dots, n$:

(a) Compute the diagonal entry:

$$v = a_{jj} - \sum_{k=1}^{j-1} \ell_{jk}^2.$$

If $v \leq 0$, the matrix is not positive definite and the algorithm terminates. Otherwise, set $\ell_{jj} = \sqrt{v}$.

(b) For each row index $i = j + 1, \dots, n$, compute the subdiagonal entries:

$$\ell_{ij} = \frac{1}{\ell_{jj}} \left(a_{ij} - \sum_{k=1}^{j-1} \ell_{ik} \ell_{jk} \right).$$

Example (Step-by-step calculation of a 3×3 Cholesky factor) Consider the symmetric positive definite matrix

$$\mathbf{A} = \begin{pmatrix} 4 & 2 & -2 \\ 2 & 10 & 2 \\ -2 & 2 & 6 \end{pmatrix}.$$

We compute the entries of the lower triangular factor $\mathbf{L}$ column by column.

1. **First column ($j = 1$):**

$$\begin{aligned} \ell_{11} &= \sqrt{a_{11}} = \sqrt{4} = 2, \\ \ell_{21} &= \frac{a_{21}}{\ell_{11}} = \frac{2}{2} = 1, \\ \ell_{31} &= \frac{a_{31}}{\ell_{11}} = \frac{-2}{2} = -1. \end{aligned}$$

2. **Second column** ($j = 2$):

$$\begin{aligned}\ell_{22} &= \sqrt{a_{22} - \ell_{21}^2} = \sqrt{10 - 1^2} = \sqrt{9} = 3, \\ \ell_{32} &= \frac{1}{\ell_{22}} (a_{32} - \ell_{31}\ell_{21}) = \frac{1}{3} (2 - (-1)(1)) = \frac{3}{3} = 1.\end{aligned}$$

3. **Third column** ($j = 3$):

$$\ell_{33} = \sqrt{a_{33} - \ell_{31}^2 - \ell_{32}^2} = \sqrt{6 - (-1)^2 - 1^2} = \sqrt{4} = 2.$$

The complete Cholesky factor is

$$\mathbf{L} = \begin{pmatrix} 2 & 0 & 0 \\ 1 & 3 & 0 \\ -1 & 1 & 2 \end{pmatrix}.$$

Direct matrix multiplication confirms that $\mathbf{L}\mathbf{L}^T = \mathbf{A}$.

2.3.4 Stability and Complexity Analysis

The Cholesky algorithm has outstanding numerical stability properties that distinguish it from general LU factorization.

Thm 2.12 (Flop Count and Memory Efficiency) For a symmetric positive definite matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$, the Cholesky factorization requires asymptotically $\frac{1}{3}n^3$ floating-point operations and n square roots. This represents half the computational work and half the memory storage of a standard LU factorization.

Proof *Proof.* At column j , computing the diagonal entry requires $j - 1$ multiplications and subtractions followed by one square root. Computing the $n - j$ subdiagonal entries requires $(n - j)(j - 1)$ multiplications and subtractions, and $n - j$ divisions. Summing the dominant multiplicative terms across all columns gives

$$\sum_{j=1}^n 2(n - j)j = 2n \sum_{j=1}^n j - 2 \sum_{j=1}^n j^2 = 2n \left(\frac{n^2}{2} \right) - 2 \left(\frac{n^3}{3} \right) = \frac{1}{3}n^3 \text{ flops.}$$

Solving $\mathbf{A}\mathbf{x} = \mathbf{b}$ using $\mathbf{L}\mathbf{y} = \mathbf{b}$ and $\mathbf{L}^T\mathbf{x} = \mathbf{y}$ requires $2n^2$ flops, identical to standard triangular substitution. $\square$

Remark 2.13 (Inherent stability without pivoting) A critical property of the Cholesky decomposition is that row interchanges are entirely unnecessary. For every entry in the factor, the diagonal identity $a_{ii} = \sum_{k=1}^i \ell_{ik}^2$ implies that $\ell_{ik}^2 \leq a_{ii}$ for all $k \leq i$. Consequently, the entries of the factor $\mathbf{L}$ are bounded directly by the diagonal entries of the original matrix:

$$|\ell_{ik}| \leq \sqrt{a_{ii}} \leq \sqrt{\max_m a_{mm}}.$$

There is no possibility of element growth during factorization. The algorithm is unconditionally backward stable, satisfying $\|\delta\mathbf{A}\|_2/\|\mathbf{A}\|_2 \leq cn\varepsilon_{\text{mach}}$ for a small constant c .

Remark 2.14 **(Condition number of SPD matrices)** For any symmetric positive definite matrix $\mathbf{A}$, the condition number in the Euclidean norm simplifies to the ratio of extreme eigenvalues:

$$\kappa_2(\mathbf{A}) = \frac{\lambda_{\max}(\mathbf{A})}{\lambda_{\min}(\mathbf{A})}.$$

Furthermore, the Cholesky factor satisfies $\kappa_2(\mathbf{L}) = \sqrt{\kappa_2(\mathbf{A})}$, showing that the triangular solves operate on matrices with significantly milder condition numbers than the original operator.

Ex 2.7

1. Compute the Cholesky factorization by hand for the matrix

$$\mathbf{A} = \begin{pmatrix} 9 & 3 & -6 \\ 3 & 26 & -7 \\ -6 & -7 & 9 \end{pmatrix}.$$

2. Using the Cholesky factor computed above, solve $\mathbf{A}\mathbf{x} = (6, 44, 2)^T$ via forward and backward substitution.
3. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be an SPD matrix. Prove that the largest entry of $\mathbf{A}$ in absolute value must lie on the main diagonal.
4. Let $\mathbf{X} \in \mathbb{R}^{m \times n}$ be a data matrix with $m \geq n$. Prove that the Gram matrix $\mathbf{X}^T \mathbf{X}$ is symmetric positive definite if and only if $\mathbf{X}$ has full column rank.

3 Orthogonality and Least Squares

3.1 Orthogonal Transformations and the QR Factorization

Orthogonal transformations are the gold standard of numerical linear algebra. Because an orthogonal transformation preserves Euclidean lengths and angles, it neither amplifies nor attenuates perturbations in the data. When an algorithm is composed entirely of orthogonal operations, the accumulation of floating-point rounding errors is strictly controlled, and the condition number of the transformation is identically one.

The QR factorization expresses a matrix as the product of an orthogonal matrix $\mathbf{Q}$ and an upper triangular matrix $\mathbf{R}$. This factorization provides an orthonormal basis for the column space of the matrix, transforming complicated geometric projections and overdetermined least-squares problems into straightforward triangular solves.

3.1.1 Orthogonality, Isometry, and Invariance

Def 3.1 (Orthogonal Matrix) A square matrix $\mathbf{Q} \in \mathbb{R}^{n \times n}$ is termed **orthogonal** if its columns form an orthonormal basis for $\mathbb{R}^n$. Equivalently,

$$\mathbf{Q}^T \mathbf{Q} = \mathbf{Q} \mathbf{Q}^T = \mathbf{I}_n \iff \mathbf{Q}^{-1} = \mathbf{Q}^T.$$

For rectangular matrices $\mathbf{Q} \in \mathbb{R}^{m \times n}$ with $m \geq n$, we say that $\mathbf{Q}$ has orthonormal columns if $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}_n$.

Thm 3.1 (Geometric Invariance Properties of Orthogonal Transformations) Let $\mathbf{Q} \in \mathbb{R}^{n \times n}$ be an orthogonal matrix. For all vectors $\mathbf{x}, \mathbf{y} \in \mathbb{R}^n$, the transformation $\mathbf{x} \mapsto \mathbf{Q}\mathbf{x}$ satisfies the following properties:

1. **Inner Product Preservation:** $\langle \mathbf{Q}\mathbf{x}, \mathbf{Q}\mathbf{y} \rangle = (\mathbf{Q}\mathbf{x})^T (\mathbf{Q}\mathbf{y}) = \mathbf{x}^T \mathbf{y} = \langle \mathbf{x}, \mathbf{y} \rangle$.
2. **Isometry (Length Preservation):** $\|\mathbf{Q}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$.
3. **Matrix 2-Norm and Conditioning:** $\|\mathbf{Q}\|_2 = 1$ and $\kappa_2(\mathbf{Q}) = \|\mathbf{Q}\|_2 \|\mathbf{Q}^{-1}\|_2 = 1$.
4. **Determinant:** $\det(\mathbf{Q}) = \pm 1$, where $+1$ corresponds to a pure rotation and -1 corresponds to a reflection.

Proof *Proof.* Using the transpose property of matrix multiplication, $(\mathbf{Q}\mathbf{x})^T (\mathbf{Q}\mathbf{y}) = \mathbf{x}^T (\mathbf{Q}^T \mathbf{Q}) \mathbf{y} = \mathbf{x}^T \mathbf{I} \mathbf{y} = \mathbf{x}^T \mathbf{y}$, which proves the preservation of inner products. Setting $\mathbf{x} = \mathbf{y}$ gives $\|\mathbf{Q}\mathbf{x}\|_2^2 = \mathbf{x}^T \mathbf{x} = \|\mathbf{x}\|_2^2$, proving length preservation. Taking the supremum over unit vectors yields $\|\mathbf{Q}\|_2 = \sup_{\|\mathbf{x}\|_2=1} \|\mathbf{Q}\mathbf{x}\|_2 = 1$. Because $\mathbf{Q}^{-1} = \mathbf{Q}^T$ is also orthogonal, $\|\mathbf{Q}^{-1}\|_2 = 1$, which establishes $\kappa_2(\mathbf{Q}) = 1 \cdot 1 = 1$. Finally, taking determinants on $\mathbf{Q}^T \mathbf{Q} = \mathbf{I}$ gives $(\det \mathbf{Q})^2 = 1 \implies \det \mathbf{Q} = \pm 1$. $\square$

3.1.2 The QR Factorization

Def 3.2 (Full and Thin QR Factorizations) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$.

1. The **Full QR Factorization** is given by

$$\mathbf{A} = \mathbf{Q}\mathbf{R} = (\mathbf{Q}_1 \quad \mathbf{Q}_2) \begin{pmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{pmatrix},$$

where $\mathbf{Q} \in \mathbb{R}^{m \times m}$ is orthogonal, $\mathbf{Q}_1 \in \mathbb{R}^{m \times n}$, $\mathbf{Q}_2 \in \mathbb{R}^{m \times (m-n)}$, and $\mathbf{R}_1 \in \mathbb{R}^{n \times n}$ is upper triangular.

2. The **Thin (Economy) QR Factorization** retains only the first n columns:

$$\mathbf{A} = \mathbf{Q}_1 \mathbf{R}_1,$$

where $\mathbf{Q}_1 \in \mathbb{R}^{m \times n}$ satisfies $\mathbf{Q}_1^T \mathbf{Q}_1 = \mathbf{I}_n$ and $\mathbf{R}_1 \in \mathbb{R}^{n \times n}$ is upper triangular.

If $\mathbf{A}$ has full column rank, the thin QR factorization is unique when the diagonal entries of $\mathbf{R}_1$ are chosen to be strictly positive.

Remark 3.1 (Subspace identification via QR) The columns of $\mathbf{Q}_1$ form an orthonormal basis for the column space (range) $\mathcal{R}(\mathbf{A})$, while the columns of $\mathbf{Q}_2$ form an orthonormal basis for the orthogonal complement $\mathcal{R}(\mathbf{A})^\perp = \mathcal{N}(\mathbf{A}^T)$. For overdetermined least-squares problems where $m \gg n$, the thin factorization requires significantly less storage ($O(mn)$ vs $O(m^2)$) and is used in practical implementations.

3.1.3 Householder Triangularization

The standard method for computing the QR factorization of a dense matrix is the Householder reflection technique. At each step, a Householder reflector is constructed to zero out all subdiagonal entries of a column simultaneously.

Def 3.3 (Householder Reflector) Let $\mathbf{v} \in \mathbb{R}^m$ be a nonzero vector. The Householder reflector associated with $\mathbf{v}$ is the matrix

$$\mathbf{H} = \mathbf{I} - 2 \frac{\mathbf{v}\mathbf{v}^T}{\mathbf{v}^T\mathbf{v}}.$$

Geometrically, $\mathbf{H}$ reflects any vector across the hyperplane orthogonal to $\mathbf{v}$. Every Householder reflector is symmetric ($\mathbf{H}^T = \mathbf{H}$) and orthogonal ($\mathbf{H}^T\mathbf{H} = \mathbf{H}^2 = \mathbf{I}$).

Thm 3.2 (Householder Vector Construction) Given a vector $\mathbf{x} \in \mathbb{R}^m$, we wish to find a reflector $\mathbf{H}$ that aligns $\mathbf{x}$ with the first canonical coordinate axis: $\mathbf{H}\mathbf{x} = \pm\|\mathbf{x}\|_2\mathbf{e}_1$. The vector $\mathbf{v}$ is chosen as

$$\mathbf{v} = \mathbf{x} + \text{sign}(x_1)\|\mathbf{x}\|_2\mathbf{e}_1,$$

where $\text{sign}(x_1) = 1$ if $x_1 \geq 0$ and -1 if $x_1 < 0$. Choosing the sign of the first component ensures that the two added terms have identical signs, completely preventing catastrophic cancellation.

Proof *Proof.* Let $\mathbf{v} = \mathbf{x} - \alpha\mathbf{e}_1$. Because $\mathbf{H}$ preserves vector lengths, we must have $\|\mathbf{H}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$, which requires $\alpha = \pm\|\mathbf{x}\|_2$. Computing $\mathbf{v}^T\mathbf{v}$ gives

$$\mathbf{v}^T\mathbf{v} = (\mathbf{x} - \alpha\mathbf{e}_1)^T(\mathbf{x} - \alpha\mathbf{e}_1) = \|\mathbf{x}\|_2^2 - 2\alpha x_1 + \alpha^2 = 2\|\mathbf{x}\|_2^2 - 2\alpha x_1 = 2\mathbf{v}^T\mathbf{x}.$$

Applying $\mathbf{H}$ to $\mathbf{x}$ yields

$$\mathbf{H}\mathbf{x} = \mathbf{x} - 2 \frac{\mathbf{v}^T\mathbf{x}}{\mathbf{v}^T\mathbf{v}}\mathbf{v} = \mathbf{x} - \mathbf{v} = \alpha\mathbf{e}_1 = \pm\|\mathbf{x}\|_2\mathbf{e}_1.$$

To ensure numerical stability in floating-point arithmetic, we choose $\alpha = -\text{sign}(x_1)\|\mathbf{x}\|_2$, so that $v_1 = x_1 + \text{sign}(x_1)\|\mathbf{x}\|_2$ avoids subtracting nearly equal quantities. $\square$

Remark 3.2 (Matrix-free application and complexity) A Householder matrix $\mathbf{H}$ is never constructed as an explicit $m \times m$ array. To apply $\mathbf{H}$ to a vector $\mathbf{y}$, we compute

$$\mathbf{H}\mathbf{y} = \mathbf{y} - \left(\frac{2\mathbf{v}^T \mathbf{y}}{\mathbf{v}^T \mathbf{v}} \right) \mathbf{v},$$

which requires only an inner product and a vector update costing $O(m)$ flops. For a matrix $\mathbf{A} \in \mathbb{R}^{m \times n}$, Householder QR requires $2mn^2 - \frac{2}{3}n^3$ flops.

3.1.4 Gram-Schmidt Orthogonalization

The Gram-Schmidt process constructs the thin QR factorization column by column by projecting each column of $\mathbf{A}$ onto the orthogonal complement of the previously computed basis vectors.

Thm 3.3 (Classical and Modified Gram-Schmidt Algorithms) Let $\mathbf{a}_1, \dots, \mathbf{a}_n \in \mathbb{R}^m$ be the columns of $\mathbf{A}$.

1. **Classical Gram-Schmidt (CGS):** For each column $k = 1, \dots, n$, compute all projections against the original vector simultaneously:

$$r_{ik} = \mathbf{q}_i^T \mathbf{a}_k \quad (i = 1, \dots, k-1), \quad \mathbf{v}_k = \mathbf{a}_k - \sum_{i=1}^{k-1} r_{ik} \mathbf{q}_i, \quad r_{kk} = \|\mathbf{v}_k\|_2, \quad \mathbf{q}_k = \frac{\mathbf{v}_k}{r_{kk}}.$$

2. **Modified Gram-Schmidt (MGS):** For each column k , subtract projections sequentially from the partially updated vector:

$$\mathbf{v}_k^{(1)} = \mathbf{a}_k, \quad \mathbf{v}_k^{(i+1)} = \mathbf{v}_k^{(i)} - (\mathbf{q}_i^T \mathbf{v}_k^{(i)}) \mathbf{q}_i \quad (i = 1, \dots, k-1), \quad \mathbf{q}_k = \frac{\mathbf{v}_k^{(k)}}{\|\mathbf{v}_k^{(k)}\|_2}.$$

Remark 3.3 (Stability hierarchy among orthogonalization methods) In exact arithmetic, CGS and MGS produce identical outputs. In floating-point arithmetic, however:

1. Classical Gram-Schmidt loses orthogonality rapidly: $\|\mathbf{Q}^T \mathbf{Q} - \mathbf{I}\|_2 \approx O(\varepsilon_{\text{mach}} \kappa(\mathbf{A})^2)$.
2. Modified Gram-Schmidt is significantly more robust: $\|\mathbf{Q}^T \mathbf{Q} - \mathbf{I}\|_2 \approx O(\varepsilon_{\text{mach}} \kappa(\mathbf{A}))$.
3. Householder QR maintains orthogonality to machine precision: $\|\mathbf{Q}^T \mathbf{Q} - \mathbf{I}\|_2 \approx O(\varepsilon_{\text{mach}})$, completely independent of $\kappa(\mathbf{A})$.

3.1.5 Orthogonal Projectors

Def 3.4 (Orthogonal Projector) Let $S \subset \mathbb{R}^m$ be a subspace with orthonormal basis columns stored in $\mathbf{Q}_1 \in \mathbb{R}^{m \times n}$. The orthogonal projector onto S is the matrix

$$\mathbf{P} = \mathbf{Q}_1 \mathbf{Q}_1^T.$$

Every orthogonal projector is symmetric ($\mathbf{P}^T = \mathbf{P}$) and idempotent ($\mathbf{P}^2 = \mathbf{P}$). The complementary projector $\mathbf{P}^\perp = \mathbf{I} - \mathbf{P}$ projects onto the orthogonal complement $S^\perp$.

Thm 3.4 (Closest Point Property of Orthogonal Projections) Let $S \subset \mathbb{R}^m$ be a linear subspace and let $\mathbf{P}$ be the orthogonal projector onto S . For any vector $\mathbf{b} \in \mathbb{R}^m$, the projected vector $\mathbf{P}\mathbf{b}$ is the unique point in S that minimizes the Euclidean distance to $\mathbf{b}$:

$$\|\mathbf{b} - \mathbf{P}\mathbf{b}\|_2 \leq \|\mathbf{b} - \mathbf{s}\|_2 \quad \text{for all } \mathbf{s} \in S.$$

Equality holds if and only if $\mathbf{s} = \mathbf{P}\mathbf{b}$.

Proof *Proof.* For any $\mathbf{s} \in S$, decompose the error vector as $\mathbf{b} - \mathbf{s} = (\mathbf{b} - \mathbf{P}\mathbf{b}) + (\mathbf{P}\mathbf{b} - \mathbf{s})$. The first term satisfies $\mathbf{b} - \mathbf{P}\mathbf{b} = (\mathbf{I} - \mathbf{P})\mathbf{b} \in S^\perp$. The second term satisfies $\mathbf{P}\mathbf{b} - \mathbf{s} \in S$ because both vectors lie in S . Since the two vectors belong to mutually orthogonal subspaces, the Pythagorean theorem yields

$$\|\mathbf{b} - \mathbf{s}\|_2^2 = \|(\mathbf{b} - \mathbf{P}\mathbf{b}) + (\mathbf{P}\mathbf{b} - \mathbf{s})\|_2^2 = \|\mathbf{b} - \mathbf{P}\mathbf{b}\|_2^2 + \|\mathbf{P}\mathbf{b} - \mathbf{s}\|_2^2.$$

Because $\|\mathbf{P}\mathbf{b} - \mathbf{s}\|_2^2 \geq 0$, we have $\|\mathbf{b} - \mathbf{s}\|_2^2 \geq \|\mathbf{b} - \mathbf{P}\mathbf{b}\|_2^2$, with strict equality holding if and only if $\|\mathbf{P}\mathbf{b} - \mathbf{s}\|_2 = 0 \iff \mathbf{s} = \mathbf{P}\mathbf{b}$. $\square$

Ex 3.1

1. Construct the Householder reflector $\mathbf{H}$ that transforms the vector $\mathbf{x} = (3, 4)^T$ into $\alpha\mathbf{e}_1 = (-5, 0)^T$. Explicitly write out the 2×2 matrix $\mathbf{H}$ and verify that $\mathbf{H} = \mathbf{H}^T$ and $\mathbf{H}^2 = \mathbf{I}$.
2. Compute the thin QR factorization of $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \end{pmatrix}$ using Modified Gram-Schmidt.
3. Let $\mathbf{P} \in \mathbb{R}^{m \times m}$ be an orthogonal projector. Prove that the eigenvalues of $\mathbf{P}$ can only be 0 or 1, and show that $\text{trace}(\mathbf{P}) = \text{rank}(\mathbf{P})$.
4. Explain why the condition number of any orthogonal matrix is equal to 1. What does this imply about the propagation of errors when applying orthogonal transformations?

3.2 Vector and Matrix Norms

Foundations for measuring approximation errors and convergence.

3.2.1 Vector Norms

Def 3.5 (Vector Norm) $\|\cdot\| : \mathbb{R}^n \rightarrow \mathbb{R}$ satisfies:

1. **Definiteness:** $\|\mathbf{x}\| \geq 0$, with equality iff $\mathbf{x} = \mathbf{0}$.

2. **Homogeneity:** $\|c\mathbf{x}\| = |c|\|\mathbf{x}\|$.
3. **Triangle Inequality:** $\|\mathbf{x} + \mathbf{y}\| \leq \|\mathbf{x}\| + \|\mathbf{y}\|$.

Def 3.6 (ℓ^p Norms) $\|\mathbf{x}\|_p = (\sum |x_i|^p)^{1/p}$.

- ℓ_1 : $\sum |x_i|$ (Sparsity).
- ℓ_2 : $\sqrt{\mathbf{x}^T \mathbf{x}}$ (Energy).
- ℓ_∞ : $\max |x_i|$ (Tolerance).

Thm 3.5 (Fundamental Inequalities)

- **Cauchy-Schwarz:** $|\mathbf{x}^T \mathbf{y}| \leq \|\mathbf{x}\|_2 \|\mathbf{y}\|_2$.
- **Hölder:** $|\mathbf{x}^T \mathbf{y}| \leq \|\mathbf{x}\|_p \|\mathbf{y}\|_q$ where $1/p + 1/q = 1$.

Remark 3.4 **Equivalence:** On $\mathbb{R}^n$, all norms are equivalent: $c_1 \|\mathbf{x}\|_\beta \leq \|\mathbf{x}\|_\alpha \leq c_2 \|\mathbf{x}\|_\beta$. Convergence in one implies convergence in all.

Ex 3.2

1. Prove the triangle inequality for $\|\cdot\|_2$ using Cauchy-Schwarz.
2. Show that $\|\mathbf{x}\|_\infty = \lim_{p \rightarrow \infty} \|\mathbf{x}\|_p$.

3.2.2 Matrix Norms

Measure how much a matrix amplifies vectors.

Def 3.7 (Induced (Operator) Norm) $\|\mathbf{A}\| = \max_{\mathbf{x} \neq \mathbf{0}} \frac{\|\mathbf{A}\mathbf{x}\|}{\|\mathbf{x}\|}$.

- $\|\mathbf{A}\|_1$: Max absolute column sum.
- $\|\mathbf{A}\|_\infty$: Max absolute row sum.
- $\|\mathbf{A}\|_2$ (Spectral): $\sigma_{\max}(\mathbf{A}) = \sqrt{\lambda_{\max}(\mathbf{A}^T \mathbf{A})}$.

Def 3.8 (Frobenius Norm) $\|\mathbf{A}\|_F = \sqrt{\sum a_{ij}^2} = \sqrt{\text{Tr}(\mathbf{A}^T \mathbf{A})}$.

- **Relation:** $\|\mathbf{A}\|_2 \leq \|\mathbf{A}\|_F \leq \sqrt{\text{rank}(\mathbf{A})} \|\mathbf{A}\|_2$.

Remark 3.5 **Consistency and Submultiplicativity:** Matrix norms are useful when they satisfy $\|\mathbf{A}\mathbf{x}\| \leq \|\mathbf{A}\| \|\mathbf{x}\|$. All induced norms are additionally submultiplicative: $\|\mathbf{AB}\| \leq \|\mathbf{A}\| \|\mathbf{B}\|$.

Ex 3.3

1. Compute $\|\mathbf{A}\|_1, \|\mathbf{A}\|_\infty, \|\mathbf{A}\|_2, \|\mathbf{A}\|_F$ for $\mathbf{A} = \begin{pmatrix} 1 & -2 \\ -3 & 4 \end{pmatrix}$.
2. **Low-Rank Approximation:** The Frobenius norm is central to the Eckart-Young-Mirsky theorem. How is it related to singular values?
3. Find a 2×2 example where the max norm ($\max |a_{ij}|$) is not submultiplicative.

3.3 Conditioning and Numerical Stability

In numerical linear algebra, errors in computed solutions arise from two distinct sources: the inherent sensitivity of the mathematical problem, and the accumulation of roundoff errors within the executing algorithm. The condition number quantifies problem sensitivity, whereas numerical stability describes the fidelity of the algorithm. A reliable computational result requires both a well-conditioned problem and a backward stable algorithm.

3.3.1 Residual, Forward Error, and Backward Error

Let $\mathbf{Ax} = \mathbf{b}$ be a nonsingular linear system with exact solution $\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$, and let $\hat{\mathbf{x}}$ denote a computed approximation. We assess the quality of $\hat{\mathbf{x}}$ through three interrelated quantities.

Def 3.9 (Forward Error, Residual, and Backward Error) For a computed approximation $\hat{\mathbf{x}}$:

1. The **forward error** is the vector difference $\Delta\mathbf{x} = \hat{\mathbf{x}} - \mathbf{x}^*$, with relative measure $\frac{\|\hat{\mathbf{x}} - \mathbf{x}^*\|}{\|\mathbf{x}^*\|}$.
2. The **residual** is the vector defect $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$, with relative measure $\frac{\|\mathbf{r}\|}{\|\mathbf{A}\|\|\hat{\mathbf{x}}\| + \|\mathbf{b}\|}$.
3. The **backward error** is the smallest perturbation $(\delta\mathbf{A}, \delta\mathbf{b})$ such that $\hat{\mathbf{x}}$ is the exact solution to the perturbed system $(\mathbf{A} + \delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$.

Thm 3.6 (The Fundamental Residual-Error Bound) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be nonsingular, let $\mathbf{b} \neq \mathbf{0}$, and let $\mathbf{x}^*$ satisfy $\mathbf{Ax}^* = \mathbf{b}$. For any vector $\hat{\mathbf{x}}$ with residual $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$, the relative forward error satisfies

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}^*\|}{\|\mathbf{x}^*\|} \leq \kappa(\mathbf{A}) \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|},$$

where $\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|$ is the condition number of $\mathbf{A}$.

Proof *Proof.* Because $\mathbf{b} = \mathbf{Ax}^*$, the residual can be written in terms of the forward error as $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \mathbf{A}(\mathbf{x}^* - \hat{\mathbf{x}})$. Multiplying both sides by $\mathbf{A}^{-1}$ yields $\mathbf{x}^* - \hat{\mathbf{x}} = \mathbf{A}^{-1}\mathbf{r}$. Taking vector norms and applying the submultiplicative property of induced matrix norms gives

$$\|\hat{\mathbf{x}} - \mathbf{x}^*\| \leq \|\mathbf{A}^{-1}\|\|\mathbf{r}\|.$$

Furthermore, from $\mathbf{b} = \mathbf{A}\mathbf{x}^*$, we obtain $\|\mathbf{b}\| \leq \|\mathbf{A}\|\|\mathbf{x}^*\|$, which rearranges to $\frac{1}{\|\mathbf{x}^*\|} \leq \frac{\|\mathbf{A}\|}{\|\mathbf{b}\|}$. Multiplying these two inequalities produces

$$\frac{\|\hat{\mathbf{x}} - \mathbf{x}^*\|}{\|\mathbf{x}^*\|} \leq (\|\mathbf{A}\|\|\mathbf{A}^{-1}\|) \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|} = \kappa(\mathbf{A}) \frac{\|\mathbf{r}\|}{\|\mathbf{b}\|}.$$

□

Example (**A small residual does not guarantee a small error**) Consider the diagonal system

$$\mathbf{A} = \begin{pmatrix} 1 & 0 \\ 0 & 10^{-8} \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 1 \\ 10^{-8} \end{pmatrix}.$$

The exact solution is $\mathbf{x}^* = (1, 1)^T$. Suppose an algorithm produces the approximation $\hat{\mathbf{x}} = (1, 0)^T$. The residual evaluates to

$$\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \begin{pmatrix} 1 \\ 10^{-8} \end{pmatrix} - \begin{pmatrix} 1 \\ 0 \end{pmatrix} = \begin{pmatrix} 0 \\ 10^{-8} \end{pmatrix},$$

which satisfies $\|\mathbf{r}\|_2/\|\mathbf{b}\|_2 \approx 10^{-8}$. However, the forward error is $\hat{\mathbf{x}} - \mathbf{x}^* = (0, -1)^T$, yielding a relative forward error of $\|\hat{\mathbf{x}} - \mathbf{x}^*\|_2/\|\mathbf{x}^*\|_2 = 1/\sqrt{2} \approx 0.707$ (a seventy percent error). The matrix has condition number $\kappa_2(\mathbf{A}) = 10^8$, which amplifies the small residual into an $O(1)$ error in the solution.

3.3.2 Conditioning and Backward Stability

Def 3.10 (**(Condition Number of a Matrix)**) The condition number of a nonsingular matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ with respect to a given norm is

$$\kappa(\mathbf{A}) = \|\mathbf{A}\|\|\mathbf{A}^{-1}\|.$$

In the Euclidean 2-norm, $\kappa_2(\mathbf{A}) = \sigma_{\max}(\mathbf{A})/\sigma_{\min}(\mathbf{A})$. The condition number measures the worst-case sensitivity of the exact solution $\mathbf{x}^*$ to relative perturbations in the input data $\mathbf{A}$ and $\mathbf{b}$.

Thm 3.7 (**(Rigal-Gaches Theorem on Normwise Backward Error)**) For any computed approximation $\hat{\mathbf{x}}$ with residual $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$, the smallest relative perturbation size satisfying $(\mathbf{A} + \delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$ with $\|\delta\mathbf{A}\| \leq \eta\|\mathbf{A}\|$ and $\|\delta\mathbf{b}\| \leq \eta\|\mathbf{b}\|$ is given explicitly by

$$\eta(\hat{\mathbf{x}}) = \frac{\|\mathbf{r}\|}{\|\mathbf{A}\|\|\hat{\mathbf{x}}\| + \|\mathbf{b}\|}.$$

An algorithm is termed **backward stable** if for every problem instance, the computed solution satisfies $\eta(\hat{\mathbf{x}}) = O(\varepsilon_{\text{mach}})$.

Proof *Proof.* From $(\mathbf{A} + \delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$, rearranging gives $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}} = \delta\mathbf{A}\hat{\mathbf{x}} - \delta\mathbf{b}$. Taking norms and applying the triangle inequality yields

$$\|\mathbf{r}\| \leq \|\delta\mathbf{A}\|\|\hat{\mathbf{x}}\| + \|\delta\mathbf{b}\| \leq \eta\|\mathbf{A}\|\|\hat{\mathbf{x}}\| + \eta\|\mathbf{b}\| = \eta(\|\mathbf{A}\|\|\hat{\mathbf{x}}\| + \|\mathbf{b}\|),$$

which establishes that $\eta \geq \frac{\|\mathbf{r}\|}{\|\mathbf{A}\|\|\hat{\mathbf{x}}\|+\|\mathbf{b}\|}$. To prove that this lower bound is attainable, define the rank-one matrix perturbation $\delta\mathbf{A} = \frac{\|\mathbf{A}\|\|\hat{\mathbf{x}}\|}{\|\mathbf{A}\|\|\hat{\mathbf{x}}\|+\|\mathbf{b}\|} \frac{\mathbf{r}\hat{\mathbf{x}}^T}{\|\hat{\mathbf{x}}\|_2^2}$ and the vector perturbation $\delta\mathbf{b} = -\frac{\|\mathbf{b}\|}{\|\mathbf{A}\|\|\hat{\mathbf{x}}\|+\|\mathbf{b}\|}\mathbf{r}$. Direct substitution confirms that $(\mathbf{A} + \delta\mathbf{A})\hat{\mathbf{x}} = \mathbf{b} + \delta\mathbf{b}$ with perturbations achieving the minimal bound. $\square$

Remark 3.6 **(The fundamental theorem of numerical analysis)** Combining backward stability with problem conditioning yields the master error estimate of scientific computation:

$$\text{Forward Error} \lesssim \kappa(\mathbf{A}) \times \text{Backward Error}.$$

A backward stable algorithm guarantees that the backward error is on the order of machine precision $\varepsilon_{\text{mach}}$. If the computed solution is inaccurate, the loss of precision is entirely due to the ill-conditioning of the mathematical model $\kappa(\mathbf{A})$, not a defect of the numerical algorithm.

3.3.3 The Solver Selection Hierarchy

Thm 3.8 **(Solver Selection Hierarchy for Linear Systems)** To maximize computational efficiency and numerical stability, direct and iterative solvers should be selected according to the structural hierarchy of the coefficient matrix:

1. **Symmetric Positive Definite (Dense):** Cholesky factorization $\mathbf{A} = \mathbf{L}\mathbf{L}^T$ ($\frac{1}{3}n^3$ flops, unconditionally backward stable without pivoting).
2. **Symmetric Positive Definite (Large Sparse):** Preconditioned Conjugate Gradient (PCG) ($O(\text{nnz})$ per iteration, $O(n)$ memory).
3. **General Square (Dense):** Gaussian Elimination with Partial Pivoting $\mathbf{PA} = \mathbf{LU}$ ($\frac{2}{3}n^3$ flops, backward stable in practice).
4. **General Square (Large Sparse):** Preconditioned GMRES or BiCGStab ($O(\text{nnz})$ per iteration).
5. **Overdetermined Full-Rank ($m \geq n$):** Householder QR factorization $\mathbf{A} = \mathbf{QR}$ ($2mn^2 - \frac{2}{3}n^3$ flops, avoids squaring the condition number).
6. **Rank-Deficient or Ill-Conditioned ($m \geq n$):** Singular Value Decomposition $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^T$ (reveals numerical rank and computes minimum-norm solutions).

3.3.4 Iterative Refinement

Def 3.11 **(Iterative Refinement Algorithm)** Given a computed solution $\hat{\mathbf{x}}^{(0)}$ from an initial LU or Cholesky factorization, iterative refinement improves the solution accuracy:

1. Compute the residual with high precision: $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}^{(k)}$.

2. Solve the error correction system using the existing factors: $\mathbf{A}\mathbf{d}^{(k)} = \mathbf{r}^{(k)}$.
3. Update the solution: $\hat{\mathbf{x}}^{(k+1)} = \hat{\mathbf{x}}^{(k)} + \mathbf{d}^{(k)}$.

If $\kappa(\mathbf{A})\varepsilon_{\text{mach}} < 1$ and the residual is evaluated with extended precision, iterative refinement converges to machine precision accuracy in a few steps.

Ex 3.4

1. Let $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 1 & 1 + 10^{-10} \end{pmatrix}$. Compute $\mathbf{A}^{-1}$ explicitly and evaluate $\kappa_{\infty}(\mathbf{A})$.
2. Suppose $\mathbf{A}\mathbf{x} = \mathbf{b}$ is solved using a backward stable algorithm on a computer with $\varepsilon_{\text{mach}} \approx 10^{-16}$. If $\kappa_2(\mathbf{A}) = 10^{11}$, estimate the number of correct decimal digits in the computed solution.
3. Prove that for any two nonsingular matrices $\mathbf{A}$ and $\mathbf{B}$, $\kappa(\mathbf{AB}) \leq \kappa(\mathbf{A})\kappa(\mathbf{B})$.
4. Show that for an orthogonal matrix $\mathbf{Q}$, $\kappa_2(\mathbf{QA}) = \kappa_2(\mathbf{A})$ for any square matrix $\mathbf{A}$.

3.4 Linear Least Squares and Overdetermined Systems

In scientific data fitting and parameter estimation, one frequently encounters linear systems $\mathbf{A}\mathbf{x} \approx \mathbf{b}$ with more equations than unknowns ($m > n$). Because the data vector $\mathbf{b} \in \mathbb{R}^m$ is corrupted by measurement noise or modeling discrepancies, it rarely lies precisely within the column space $\mathcal{R}(\mathbf{A}) \subset \mathbb{R}^m$. Consequently, no exact solution exists.

The method of linear least squares replaces the impossible requirement of zero residual with the optimization problem of finding the vector $\hat{\mathbf{x}} \in \mathbb{R}^n$ that minimizes the Euclidean norm of the residual vector $\mathbf{r}(\mathbf{x}) = \mathbf{b} - \mathbf{A}\mathbf{x}$. Geometrically, the optimal model prediction $\mathbf{A}\hat{\mathbf{x}}$ is the orthogonal projection of the observation vector $\mathbf{b}$ onto the range of $\mathbf{A}$.

3.4.1 The Least-Squares Problem and the Normal Equations

Def 3.12 (Linear Least-Squares Problem) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$ and let $\mathbf{b} \in \mathbb{R}^m$. The linear least-squares problem seeks a vector $\hat{\mathbf{x}} \in \mathbb{R}^n$ satisfying

$$\hat{\mathbf{x}} = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 = \arg \min_{\mathbf{x} \in \mathbb{R}^n} \sum_{i=1}^m \left(\sum_{j=1}^n a_{ij}x_j - b_i \right)^2.$$

Thm 3.9 (Orthogonal Characterization and the Normal Equations) A vector $\hat{\mathbf{x}} \in \mathbb{R}^n$ minimizes $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2$ if and only if the residual vector $\mathbf{r} = \mathbf{b} - \mathbf{A}\hat{\mathbf{x}}$ is orthogonal to the column space $\mathcal{R}(\mathbf{A})$:

$$\mathbf{r} \perp \mathcal{R}(\mathbf{A}) \iff \mathbf{A}^T \mathbf{r} = \mathbf{0} \iff \mathbf{A}^T (\mathbf{b} - \mathbf{A}\hat{\mathbf{x}}) = \mathbf{0}.$$

This orthogonality condition produces the **Normal Equations**:

$$\mathbf{A}^T \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^T \mathbf{b}.$$

If $\mathbf{A}$ has full column rank ($\text{rank}(\mathbf{A}) = n$), the Gram matrix $\mathbf{A}^T \mathbf{A}$ is symmetric positive definite, and the unique least-squares solution is $\hat{\mathbf{x}} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b}$.

Proof *Proof.* Define the quadratic objective function $f(\mathbf{x}) = \frac{1}{2} \|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 = \frac{1}{2} (\mathbf{A}\mathbf{x} - \mathbf{b})^T (\mathbf{A}\mathbf{x} - \mathbf{b}) = \frac{1}{2} \mathbf{x}^T \mathbf{A}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{A} \mathbf{x} + \frac{1}{2} \mathbf{b}^T \mathbf{b}$. Computing the gradient with respect to $\mathbf{x}$ yields

$$\nabla f(\mathbf{x}) = \mathbf{A}^T \mathbf{A} \mathbf{x} - \mathbf{A}^T \mathbf{b} = -\mathbf{A}^T (\mathbf{b} - \mathbf{A} \mathbf{x}) = -\mathbf{A}^T \mathbf{r}(\mathbf{x}).$$

Setting $\nabla f(\hat{\mathbf{x}}) = \mathbf{0}$ yields $\mathbf{A}^T \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^T \mathbf{b}$. Because the Hessian $\nabla^2 f(\mathbf{x}) = \mathbf{A}^T \mathbf{A}$ is symmetric positive semidefinite, any stationary point is a global minimizer. When $\mathbf{A}$ has full column rank, $\mathbf{A}^T \mathbf{A}$ is strictly positive definite, ensuring that $\hat{\mathbf{x}}$ is unique. $\square$

Remark 3.7 **(The conditioning danger of normal equations)** Although forming and solving the normal equations $\mathbf{A}^T \mathbf{A} \hat{\mathbf{x}} = \mathbf{A}^T \mathbf{b}$ via Cholesky factorization requires only $mn^2 + \frac{1}{3}n^3$ flops, it is numerically dangerous. The 2-norm condition number of the Gram matrix is the square of the condition number of $\mathbf{A}$:

$$\kappa_2(\mathbf{A}^T \mathbf{A}) = \kappa_2(\mathbf{A})^2.$$

If $\kappa_2(\mathbf{A}) = 10^8$, then $\kappa_2(\mathbf{A}^T \mathbf{A}) = 10^{16}$, and solving the normal equations in standard double precision arithmetic can lose all sixteen significant digits. Therefore, the normal equations should never be the default method for ill-conditioned data fitting.

3.4.2 Solving Least Squares via QR Factorization

The standard, numerically stable method for solving full-rank least-squares problems is the thin QR factorization, which computes the solution without ever forming the squared matrix $\mathbf{A}^T \mathbf{A}$.

Thm 3.10 **(Least-Squares Solution via QR Factorization)** Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$ have full column rank, and let $\mathbf{A} = \mathbf{Q}_1 \mathbf{R}_1$ be its thin QR factorization, where $\mathbf{Q}_1 \in \mathbb{R}^{m \times n}$ has orthonormal columns and $\mathbf{R}_1 \in \mathbb{R}^{n \times n}$ is nonsingular and upper triangular. The unique least-squares solution is computed by solving the upper triangular system

$$\mathbf{R}_1 \hat{\mathbf{x}} = \mathbf{Q}_1^T \mathbf{b}.$$

The condition number of this triangular system is $\kappa_2(\mathbf{R}_1) = \kappa_2(\mathbf{A})$, completely avoiding the condition number squaring of the normal equations.

Proof *Proof.* Let $\mathbf{A} = \mathbf{Q}\mathbf{R} = (\mathbf{Q}_1 \ \mathbf{Q}_2) \begin{pmatrix} \mathbf{R}_1 \\ \mathbf{0} \end{pmatrix}$ be the full QR factorization. Because orthogonal

transformations preserve the Euclidean norm, we multiply the residual by $\mathbf{Q}^T$:

$$\begin{aligned}\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 &= \|\mathbf{Q}^T(\mathbf{A}\mathbf{x} - \mathbf{b})\|_2^2 \\ &= \left\| \begin{pmatrix} \mathbf{Q}_1^T \\ \mathbf{Q}_2^T \end{pmatrix} (\mathbf{Q}_1\mathbf{R}_1\mathbf{x} - \mathbf{b}) \right\|_2^2 \\ &= \left\| \begin{pmatrix} \mathbf{R}_1\mathbf{x} - \mathbf{Q}_1^T\mathbf{b} \\ -\mathbf{Q}_2^T\mathbf{b} \end{pmatrix} \right\|_2^2 \\ &= \|\mathbf{R}_1\mathbf{x} - \mathbf{Q}_1^T\mathbf{b}\|_2^2 + \|\mathbf{Q}_2^T\mathbf{b}\|_2^2.\end{aligned}$$

The second term $\|\mathbf{Q}_2^T\mathbf{b}\|_2^2$ is independent of the choice of $\mathbf{x}$, representing the inherent distance from $\mathbf{b}$ to the subspace $\mathcal{R}(\mathbf{A})$. The first term is non-negative and can be driven to zero by setting $\mathbf{R}_1\mathbf{x} = \mathbf{Q}_1^T\mathbf{b}$. Since $\mathbf{A}$ has full column rank, $\mathbf{R}_1$ is invertible, and $\hat{\mathbf{x}} = \mathbf{R}_1^{-1}\mathbf{Q}_1^T\mathbf{b}$ is the unique global minimizer. $\square$

Example (**Step-by-step least-squares line fit**) Suppose we wish to fit the linear model $y = c_0 + c_1t$ to the three data points $(0, 1)$, $(1, 2)$, and $(2, 4)$. The overdetermined system $\mathbf{A}\mathbf{c} \approx \mathbf{y}$ is

$$\begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} \approx \begin{pmatrix} 1 \\ 2 \\ 4 \end{pmatrix}.$$

Computing the normal equations components:

$$\mathbf{A}^T\mathbf{A} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 0 \\ 1 & 1 \\ 1 & 2 \end{pmatrix} = \begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix}, \quad \mathbf{A}^T\mathbf{y} = \begin{pmatrix} 1 & 1 & 1 \\ 0 & 1 & 2 \end{pmatrix} \begin{pmatrix} 1 \\ 2 \\ 4 \end{pmatrix} = \begin{pmatrix} 7 \\ 10 \end{pmatrix}.$$

Solving $\begin{pmatrix} 3 & 3 \\ 3 & 5 \end{pmatrix} \begin{pmatrix} c_0 \\ c_1 \end{pmatrix} = \begin{pmatrix} 7 \\ 10 \end{pmatrix}$ yields $c_1 = 3/2 = 1.5$ and $c_0 = (7 - 3(1.5))/3 = 5/6 \approx 0.8333$. The fitted line is $y = \frac{5}{6} + \frac{3}{2}t$.

3.4.3 Rank Deficiency, SVD, and Regularization

When $\mathbf{A}$ is rank deficient ($\text{rank}(\mathbf{A}) < n$), the matrix $\mathbf{A}^T\mathbf{A}$ is singular, and infinitely many vectors minimize the residual norm. In this case, the Singular Value Decomposition identifies the unique solution of minimal Euclidean norm.

Thm 3.11 (**Minimum-Norm Solution via Singular Value Decomposition**) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with SVD $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$, where $r = \text{rank}(\mathbf{A})$. Among all vectors that minimize $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$, the unique vector with the smallest Euclidean norm $\|\mathbf{x}\|_2$ is given by the Moore-Penrose pseudoinverse solution:

$$\hat{\mathbf{x}}_{\min} = \mathbf{A}^\dagger \mathbf{b} = \sum_{i=1}^r \frac{\mathbf{u}_i^T \mathbf{b}}{\sigma_i} \mathbf{v}_i.$$

Def 3.13 (Tikhonov (Ridge) Regularization) When $\mathbf{A}$ is ill-conditioned or nearly rank deficient, computing $\mathbf{A}^\dagger \mathbf{b}$ can amplify noise in the data due to division by very small singular values $\sigma_i \ll 1$. Tikhonov regularization stabilizes the problem by adding an energy penalty:

$$\hat{\mathbf{x}}_\lambda = \arg \min_{\mathbf{x} \in \mathbb{R}^n} (\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2^2 + \lambda \|\mathbf{x}\|_2^2), \quad \lambda > 0.$$

The regularized solution satisfies the modified normal equations

$$(\mathbf{A}^T \mathbf{A} + \lambda \mathbf{I}) \hat{\mathbf{x}}_\lambda = \mathbf{A}^T \mathbf{b},$$

which can be solved stably via the QR factorization of the augmented matrix $\begin{pmatrix} \mathbf{A} \\ \sqrt{\lambda} \mathbf{I} \end{pmatrix} \mathbf{x} \approx \begin{pmatrix} \mathbf{b} \\ \mathbf{0} \end{pmatrix}$.

Ex 3.5

1. Fit a parabola $y = c_0 + c_1 t + c_2 t^2$ to the four data points $(-1, 1)$, $(0, 0)$, $(1, 1)$, and $(2, 5)$ using thin QR factorization.
2. Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ have full column rank with singular values $\sigma_1 \geq \dots \geq \sigma_n > 0$. Prove that $\kappa_2(\mathbf{A}^T \mathbf{A}) = (\sigma_1 / \sigma_n)^2 = \kappa_2(\mathbf{A})^2$.
3. Prove that the Tikhonov regularized solution in SVD coordinates satisfies $\hat{\mathbf{x}}_\lambda = \sum_{i=1}^n \frac{\sigma_i}{\sigma_i^2 + \lambda} (\mathbf{u}_i^T \mathbf{b}) \mathbf{v}_i$. Explain how the filter factor $\frac{\sigma_i}{\sigma_i^2 + \lambda}$ prevents noise explosion for $\sigma_i \ll \sqrt{\lambda}$.

3.5 Chebyshev Polynomials

Chebyshev polynomials are the polynomial analogue of Fourier modes on $[-1, 1]$. They oscillate evenly, stay bounded, and give node sets that control interpolation error. The central idea is minimax: among polynomials of a fixed degree, Chebyshev polynomials spread error as evenly as possible instead of allowing large boundary spikes.

3.5.1 Failure of the Monomial Basis

Polynomial approximation has two separate failure modes. The first is numerical: the monomial basis $1, x, x^2, \dots$ leads to ill-conditioned Vandermonde matrices. The second is analytical: even exact interpolation at equally spaced nodes can oscillate badly near the interval endpoints.

Def 3.14 (Interpolation Error Polynomial) Given distinct interpolation nodes $x_1, \dots, x_n$, define

$$\omega_n(x) = \prod_{j=1}^n (x - x_j).$$

This polynomial is zero at every node. Its size between the nodes controls how much interpolation can amplify the derivative of the target function.

Thm 3.12 (Interpolation Error Formula) Let p_{n-1} interpolate a function f at n distinct nodes $x_1, \dots, x_n$. If f has n continuous derivatives on $[-1, 1]$, then for each $x \in [-1, 1]$ there exists a point $\xi_x \in [-1, 1]$ such that

$$f(x) - p_{n-1}(x) = \frac{f^{(n)}(\xi_x)}{n!} \omega_n(x).$$

Therefore,

$$\|f - p_{n-1}\|_\infty \leq \frac{\|f^{(n)}\|_\infty}{n!} \|\omega_n\|_\infty.$$

Ex 3.6

1. For three nodes x_1, x_2, x_3 , write out $\omega_3(x)$ and verify that it vanishes at each node.
2. Use Thm 3.12 to explain why node choice matters even when $f^{(n)}$ is fixed.
3. Build the degree-14 Vandermonde matrix on 15 equally spaced nodes in $[-1, 1]$. Compute $\kappa_2(\mathbf{V})$.
4. Plot Runge's function $f(x) = 1/(1 + 25x^2)$ and its degree-10 interpolant at equally spaced nodes. Compare the largest error in the middle of the interval with the largest error near the endpoints.

3.5.2 Chebyshev Polynomials (T_k)

Def 3.15 (Chebyshev Polynomial) For $x \in [-1, 1]$, the degree- k Chebyshev polynomial is

$$T_k(x) = \cos(k \arccos x).$$

Equivalently, if $x = \cos \theta$, then

$$T_k(\cos \theta) = \cos(k\theta).$$

Thm 3.13 (Basic Chebyshev Identities) The polynomials from Def 3.15 satisfy

$$T_0(x) = 1, \quad T_1(x) = x, \quad T_{k+1}(x) = 2xT_k(x) - T_{k-1}(x).$$

They are bounded by 1 on $[-1, 1]$:

$$|T_k(x)| \leq 1.$$

They are orthogonal with respect to the weight $(1 - x^2)^{-1/2}$:

$$\int_{-1}^1 T_j(x)T_k(x) \frac{dx}{\sqrt{1-x^2}} = 0, \quad j \neq k.$$

Ex 3.7

1. Use $\cos((k+1)\theta) + \cos((k-1)\theta) = 2\cos\theta\cos(k\theta)$ to prove the three-term recurrence in Thm 3.13.
2. Compute T_2 , T_3 , and T_4 from the recurrence.
3. Prove $|T_k(x)| \leq 1$ on $[-1, 1]$ directly from Def 3.15.
4. Substitute $x = \cos\theta$ into the weighted integral and reduce the orthogonality statement to Fourier cosine orthogonality on $[0, \pi]$.
5. Plot $T_0, \dots, T_5$. Mark the points where each polynomial reaches ± 1 .

3.5.3 Zeros and Nodes

Thm 3.14 (Zeros and Extrema) The zeros of T_n are

$$x_j = \cos\left(\frac{(2j-1)\pi}{2n}\right), \quad j = 1, \dots, n.$$

The extrema of T_n occur at

$$y_j = \cos\left(\frac{j\pi}{n}\right), \quad j = 0, \dots, n,$$

and satisfy $T_n(y_j) = (-1)^j$.

Def 3.16 (Chebyshev Nodes) The Chebyshev interpolation nodes are the zeros of T_n :

$$x_j = \cos\left(\frac{(2j-1)\pi}{2n}\right), \quad j = 1, \dots, n.$$

These nodes cluster near ± 1 . The clustering is not cosmetic; it reduces the size of the interpolation error polynomial from Def 3.14.

Thm 3.15 (Chebyshev Error Polynomial) If $x_1, \dots, x_n$ are the Chebyshev nodes from Def 3.16, then

$$\omega_n(x) = \prod_{j=1}^n (x - x_j) = 2^{1-n} T_n(x).$$

Consequently,

$$\|\omega_n\|_\infty = 2^{1-n}.$$

Ex 3.8

1. Starting from $T_n(\cos \theta) = \cos(n\theta)$, solve $\cos(n\theta) = 0$ and prove the zero formula in Thm 3.14.
2. Solve $\cos(n\theta) = \pm 1$ and prove the extrema formula in Thm 3.14.
3. Explain why the Chebyshev nodes cluster near ± 1 by comparing the spacing of equally spaced θ values under $x = \cos \theta$.
4. Show that T_n has leading coefficient 2^{n-1} for $n \geq 1$.
5. Use the previous step and the fact that T_n has zeros $x_1, \dots, x_n$ to prove Thm 3.15.

3.5.4 The Minimax Property

Thm 3.16 (Chebyshev Minimax Polynomial) Among all monic polynomials q of degree n on $[-1, 1]$,

$$2^{1-n} T_n(x)$$

has the smallest possible infinity norm:

$$\|2^{1-n} T_n\|_\infty = 2^{1-n} \leq \|q\|_\infty.$$

Ex 3.9

1. Let $m(x) = 2^{1-n} T_n(x)$. Use Thm 3.13 to show that m is monic and $\|m\|_\infty = 2^{1-n}$.
2. Suppose another monic polynomial q satisfies $\|q\|_\infty < 2^{1-n}$. Define $r = q - m$. What is the degree of r ?
3. Evaluate the signs of m at the $n + 1$ extrema from Thm 3.14.

4. Use the assumption $\|q\|_\infty < 2^{1-n}$ to show that r changes sign between each pair of adjacent extrema.
5. Conclude that r has at least n roots, contradicting its degree. This proves Thm 3.16.
6. Combine Thm 3.12 and Thm 3.15 to explain why Chebyshev nodes are the natural interpolation nodes.

3.5.5 Chebyshev Series

Def 3.17 (Chebyshev Series) A Chebyshev expansion writes a function on $[-1, 1]$ as

$$f(x) \sim \sum_{k=0}^{\infty} c_k T_k(x).$$

Using $x = \cos \theta$, this is exactly a cosine series for the even function $f(\cos \theta)$.

Thm 3.17 (Coefficient Formula) For $k \geq 1$,

$$c_k = \frac{2}{\pi} \int_0^\pi f(\cos \theta) \cos(k\theta) d\theta,$$

and

$$c_0 = \frac{1}{\pi} \int_0^\pi f(\cos \theta) d\theta.$$

Thm 3.18 (Spectral Accuracy) If f is analytic on and near $[-1, 1]$, then its Chebyshev coefficients decay geometrically:

$$|c_k| \leq C\rho^{-k} \quad \text{for some } \rho > 1.$$

The truncated series

$$p_n(x) = \sum_{k=0}^n c_k T_k(x)$$

therefore satisfies

$$\|f - p_n\|_\infty = O(\rho^{-n}).$$

This is called spectral accuracy.

Remark 3.8 (Fourier connection) Chebyshev methods are Fourier cosine methods after the change of variables $x = \cos \theta$. This is why fast cosine transforms can compute Chebyshev coefficients efficiently.

Remark 3.9 (Interpolation vs. expansion) Chebyshev interpolation chooses values at Chebyshev nodes. Chebyshev series choose coefficients in the Chebyshev basis. Both work well for the same reason: the basis oscillates evenly and avoids the endpoint instability of monomials.

Ex 3.10

1. Use Def 3.17 to rewrite a Chebyshev expansion as a cosine series in θ .
2. Derive the coefficient formula in Thm 3.17 from Fourier cosine orthogonality.
3. Compute degree- n Chebyshev fits to e^x using `np.polynomial.chebyshev.chebfit`. Plot $\|f - p_n\|_\infty$ versus n on a semilog scale.
4. Repeat the previous experiment for $f(x) = |x|$. Compare the coefficient decay with the analytic case in Thm 3.18.
5. Use `chebder` to differentiate a Chebyshev approximation to $f(x) = \sin(\pi x)$. Compare the derivative error with a finite-difference approximation using the same number of sample points.

4 Eigenvalues and the SVD

4.1 Eigenvalue Problems and the QR Algorithm

The eigenvalue problem $\mathbf{A}\mathbf{v} = \lambda\mathbf{v}$ seeks the invariant directions of a linear operator. Along an eigenvector $\mathbf{v}$, the action of the matrix $\mathbf{A}$ reduces to scalar multiplication by λ . Eigenvalues govern the long-term stability of dynamical systems, the resonant frequencies of vibrating structures, the convergence rates of iterative solvers, and the low-dimensional structure of data in principal component analysis.

In introductory algebra, eigenvalues are introduced as the roots of the characteristic polynomial $p(\lambda) = \det(\mathbf{A} - \lambda\mathbf{I}) = 0$. In numerical computing, however, computing eigenvalues via characteristic polynomials is strictly avoided. By Abel's Impossibility Theorem, no closed-form radical formula exists for $n \geq 5$, meaning all eigenvalue algorithms must be iterative. Furthermore, polynomial root-finding is notoriously ill-conditioned, whereas matrix eigenvalue algorithms can be executed with backward stability.

4.1.1 Theoretical Foundations: Multiplicity and Schur Form

Def 4.1 (Eigenvalues and Eigenvectors) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$. A nonzero vector $\mathbf{v} \in \mathbb{C}^n$ is an **eigenvector** of $\mathbf{A}$ associated with the **eigenvalue** $\lambda \in \mathbb{C}$ if

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v}.$$

The set of all eigenvalues $\sigma(\mathbf{A}) = \{\lambda_1, \dots, \lambda_n\}$ is the **spectrum** of $\mathbf{A}$.

Def 4.2 (Algebraic and Geometric Multiplicity) Let λ be an eigenvalue of $\mathbf{A}$.

1. The **algebraic multiplicity** $\mu_a(\lambda)$ is the multiplicity of λ as a root of $\det(\mathbf{A} - z\mathbf{I}) = 0$.
2. The **geometric multiplicity** $\mu_g(\lambda) = \dim \mathcal{N}(\mathbf{A} - \lambda\mathbf{I})$ is the maximum number of linearly independent eigenvectors associated with λ .

It is always true that $1 \leq \mu_g(\lambda) \leq \mu_a(\lambda)$. If $\mu_g(\lambda) < \mu_a(\lambda)$ for any eigenvalue, the matrix is termed **defective** and cannot be diagonalized.

Thm 4.1 (Schur Decomposition) Every square matrix $\mathbf{A} \in \mathbb{C}^{n \times n}$ is unitarily similar to an upper triangular matrix:

$$\mathbf{A} = \mathbf{Q}\mathbf{T}\mathbf{Q}^*,$$

where $\mathbf{Q} \in \mathbb{C}^{n \times n}$ is unitary ($\mathbf{Q}^*\mathbf{Q} = \mathbf{I}$) and $\mathbf{T} \in \mathbb{C}^{n \times n}$ is upper triangular. The diagonal entries of $\mathbf{T}$ are precisely the eigenvalues of $\mathbf{A}$.

Proof *Proof.* We proceed by induction on n . For $n = 1$, the result is trivial. Assume the theorem holds for matrices of order $n - 1$. Let λ_1 be an eigenvalue of $\mathbf{A}$ with normalized eigenvector $\mathbf{v}_1$ ($\|\mathbf{v}_1\|_2 = 1$). Construct an orthonormal basis for $\mathbb{C}^n$ starting with $\mathbf{v}_1$, forming the unitary matrix $\mathbf{U}_1 = (\mathbf{v}_1 \ \mathbf{W})$. Computing the similarity transform yields

$$\mathbf{U}_1^* \mathbf{A} \mathbf{U}_1 = \begin{pmatrix} \mathbf{v}_1^* \\ \mathbf{W}^* \end{pmatrix} (\mathbf{A} \mathbf{v}_1 \ \mathbf{A} \mathbf{W}) = \begin{pmatrix} \lambda_1 \mathbf{v}_1^* \mathbf{v}_1 & \mathbf{v}_1^* \mathbf{A} \mathbf{W} \\ \lambda_1 \mathbf{W}^* \mathbf{v}_1 & \mathbf{W}^* \mathbf{A} \mathbf{W} \end{pmatrix} = \begin{pmatrix} \lambda_1 & \mathbf{w}^T \\ \mathbf{0} & \mathbf{A}_1 \end{pmatrix},$$

where $\mathbf{A}_1 = \mathbf{W}^* \mathbf{A} \mathbf{W} \in \mathbb{C}^{(n-1) \times (n-1)}$. By the induction hypothesis, there exists a unitary matrix $\mathbf{Q}_1$ such that $\mathbf{Q}_1^* \mathbf{A}_1 \mathbf{Q}_1 = \mathbf{T}_1$ is upper triangular. Setting $\mathbf{Q} = \mathbf{U}_1 \begin{pmatrix} 1 & \mathbf{0} \\ \mathbf{0} & \mathbf{Q}_1 \end{pmatrix}$ yields the complete triangularization $\mathbf{Q}^* \mathbf{A} \mathbf{Q} = \mathbf{T}$. $\square$

Thm 4.2 (Spectral Theorem for Hermitian and Real Symmetric Matrices) If $\mathbf{A} \in \mathbb{R}^{n \times n}$ is real symmetric ($\mathbf{A} = \mathbf{A}^T$), then all eigenvalues of $\mathbf{A}$ are real, and $\mathbf{A}$ is orthogonally diagonalizable:

$$\mathbf{A} = \mathbf{Q} \mathbf{\Lambda} \mathbf{Q}^T = \sum_{i=1}^n \lambda_i \mathbf{q}_i \mathbf{q}_i^T,$$

where $\mathbf{Q} = [\mathbf{q}_1, \dots, \mathbf{q}_n]$ is a real orthogonal matrix whose columns are orthonormal eigenvectors.

4.1.2 Power Iteration and Shifted Inverse Iteration

When only a single dominant eigenvalue or an eigenvalue closest to a target frequency is required, vector iterations provide rapid, memory-efficient solutions.

Def 4.3 (Power Iteration) Given an initial vector $\mathbf{v}^{(0)}$ with $\|\mathbf{v}^{(0)}\|_2 = 1$, power iteration generates successive approximations via

$$\mathbf{w}^{(k)} = \mathbf{A}\mathbf{v}^{(k-1)}, \quad \mathbf{v}^{(k)} = \frac{\mathbf{w}^{(k)}}{\|\mathbf{w}^{(k)}\|_2}, \quad \lambda^{(k)} = (\mathbf{v}^{(k)})^T \mathbf{A}\mathbf{v}^{(k)}.$$

If the dominant eigenvalue satisfies $|\lambda_1| > |\lambda_2| \geq \dots \geq |\lambda_n|$ and the initial vector has a nonzero component along the dominant eigenvector $\mathbf{q}_1$, then $\mathbf{v}^{(k)}$ converges to $\mathbf{q}_1$ at the linear rate

$$\|\mathbf{v}^{(k)} - (\pm \mathbf{q}_1)\|_2 = O\left(\left|\frac{\lambda_2}{\lambda_1}\right|^k\right).$$

Def 4.4 (Inverse Iteration with Shifts) To compute the eigenvalue closest to a target scalar $\sigma \in \mathbb{C}$, power iteration is applied to the shifted and inverted operator $(\mathbf{A} - \sigma\mathbf{I})^{-1}$:

$$(\mathbf{A} - \sigma\mathbf{I})\mathbf{w}^{(k)} = \mathbf{v}^{(k-1)}, \quad \mathbf{v}^{(k)} = \frac{\mathbf{w}^{(k)}}{\|\mathbf{w}^{(k)}\|_2}.$$

The eigenvalues of $(\mathbf{A} - \sigma\mathbf{I})^{-1}$ are $(\lambda_i - \sigma)^{-1}$. The eigenvalue of $\mathbf{A}$ closest to σ becomes the dominant eigenvalue of the shifted operator, accelerating convergence to the linear rate $|\lambda_J - \sigma|/|\lambda_K - \sigma|$, where λ_J is the closest eigenvalue and λ_K is the second closest.

Def 4.5 (Rayleigh Quotient Iteration (RQI)) For symmetric matrices, updating the shift dynamically to the current Rayleigh quotient $\sigma_k = (\mathbf{v}^{(k)})^T \mathbf{A}\mathbf{v}^{(k)}$ yields **Rayleigh Quotient Iteration**. At each step, we solve

$$(\mathbf{A} - \sigma_k\mathbf{I})\mathbf{w}^{(k+1)} = \mathbf{v}^{(k)}, \quad \mathbf{v}^{(k+1)} = \frac{\mathbf{w}^{(k+1)}}{\|\mathbf{w}^{(k+1)}\|_2}, \quad \sigma_{k+1} = (\mathbf{v}^{(k+1)})^T \mathbf{A}\mathbf{v}^{(k+1)}.$$

For symmetric matrices, RQI converges **cubically**: the number of correct decimal digits triples at each iteration once the iterate enters the asymptotic regime.

4.1.3 The Practical QR Algorithm

The standard method for computing the complete spectrum of a dense matrix is the QR algorithm with shifts.

Def 4.6 (Basic QR Iteration) The unshifted QR iteration computes a sequence of orthogonally similar matrices:

1. Initialize $\mathbf{A}_0 = \mathbf{A}$.
2. For $k = 0, 1, 2, \dots$:
 - (a) Compute the QR factorization: $\mathbf{A}_k = \mathbf{Q}_k \mathbf{R}_k$.

(b) Recombine the factors in reverse order: $\mathbf{A}_{k+1} = \mathbf{R}_k \mathbf{Q}_k = \mathbf{Q}_k^T \mathbf{A}_k \mathbf{Q}_k$.

Because $\mathbf{A}_{k+1}$ is orthogonally similar to $\mathbf{A}_k$, all iterates share identical eigenvalues. As $k \rightarrow \infty$, $\mathbf{A}_k$ converges to the upper triangular Schur form $\mathbf{T}$.

Thm 4.3 (Practical Shifted QR Algorithm on Hessenberg Forms) To make the QR algorithm computationally efficient, standard numerical libraries implement two essential enhancements:

1. **Preliminary Hessenberg Reduction:** The matrix $\mathbf{A}$ is pre-transformed to upper Hessenberg form $\mathbf{H}_0 = \mathbf{Q}_0^T \mathbf{A} \mathbf{Q}_0$ using Householder reflectors in $O(n^3)$ operations once. Subsequent QR steps preserve the Hessenberg structure and require only $O(n^2)$ flops using Givens rotations.
2. **Wilkinson Shifts:** At each step, a shift μ_k is chosen as the eigenvalue of the trailing 2×2 submatrix of $\mathbf{H}_k$ closest to h_{nn} :

$$\mathbf{H}_k - \mu_k \mathbf{I} = \mathbf{Q}_k \mathbf{R}_k, \quad \mathbf{H}_{k+1} = \mathbf{R}_k \mathbf{Q}_k + \mu_k \mathbf{I}.$$

The shift accelerates subdiagonal convergence to cubic order for symmetric matrices and quadratic order for nonsymmetric matrices. When the subdiagonal entry satisfies $|h_{n,n-1}| \leq \varepsilon_{\text{mach}}(|h_{nn}| + |h_{n-1,n-1}|)$, h_{nn} is decoupled as a converged eigenvalue and the matrix is deflated.

Ex 4.1

1. Compute the dominant eigenvalue and eigenvector of $\mathbf{A} = \begin{pmatrix} 3 & 1 \\ 1 & 3 \end{pmatrix}$ by hand using two steps of power iteration starting with $\mathbf{v}^{(0)} = (1, 0)^T$.
2. Apply one step of unshifted QR iteration to $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Compute the QR factors explicitly and show that the off-diagonal entries decrease.
3. Let $\mathbf{A} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$. Show that unshifted QR iteration fails to converge because the factors oscillate indefinitely. Explain how introducing a shift resolves this cycle.

4.2 The Singular Value Decomposition

The Singular Value Decomposition (SVD) is the pinnacle of matrix factorizations in computational linear algebra. Unlike the eigendecomposition, which is defined only for square matrices and may fail to exist for defective operators, the SVD exists unconditionally for every matrix of arbitrary rectangular dimensions $\mathbf{A} \in \mathbb{R}^{m \times n}$.

Geometrically, the SVD states that the action of any linear transformation on the unit sphere in $\mathbb{R}^n$ maps it into a hyperellipsoid in $\mathbb{R}^m$. The right singular vectors $\mathbf{v}_i$ define the

principal orthogonal directions in the domain, the singular values σ_i specify the scaling lengths along each semi-axis, and the left singular vectors $\mathbf{u}_i$ define the corresponding orthogonal directions in the codomain.

4.2.1 The SVD Theorem and Subspace Characterization

Thm 4.4 (The Singular Value Decomposition Theorem) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ with $m \geq n$. There exist orthogonal matrices $\mathbf{U} \in \mathbb{R}^{m \times m}$ and $\mathbf{V} \in \mathbb{R}^{n \times n}$, and a diagonal matrix $\mathbf{\Sigma} \in \mathbb{R}^{m \times n}$, such that

$$\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T,$$

where the diagonal entries of $\mathbf{\Sigma} = \text{diag}(\sigma_1, \sigma_2, \dots, \sigma_n)$ are real, non-negative, and sorted in non-increasing order:

$$\sigma_1 \geq \sigma_2 \geq \dots \geq \sigma_r > \sigma_{r+1} = \dots = \sigma_n = 0.$$

The number of strictly positive singular values $r = \text{rank}(\mathbf{A})$ equals the rank of $\mathbf{A}$. In dyadic (outer product) form,

$$\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

Thm 4.5 (Thin SVD and the Four Fundamental Subspaces) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ have rank $r \leq n \leq m$. The thin (economy) SVD is given by

$$\mathbf{A} = \mathbf{U}_1 \mathbf{\Sigma}_1 \mathbf{V}_1^T,$$

where $\mathbf{U}_1 \in \mathbb{R}^{m \times r}$ and $\mathbf{V}_1 \in \mathbb{R}^{n \times r}$ have orthonormal columns, and $\mathbf{\Sigma}_1 = \text{diag}(\sigma_1, \dots, \sigma_r) \in \mathbb{R}^{r \times r}$ is strictly positive. The columns of the complete singular vectors provide optimal orthonormal bases for the four fundamental subspaces:

1. **Column Space (Range):** $\mathcal{R}(\mathbf{A}) = \text{span}\{\mathbf{u}_1, \dots, \mathbf{u}_r\} \subset \mathbb{R}^m$.
2. **Left Null Space:** $\mathcal{N}(\mathbf{A}^T) = \text{span}\{\mathbf{u}_{r+1}, \dots, \mathbf{u}_m\} \subset \mathbb{R}^m$.
3. **Row Space:** $\mathcal{R}(\mathbf{A}^T) = \text{span}\{\mathbf{v}_1, \dots, \mathbf{v}_r\} \subset \mathbb{R}^n$.
4. **Null Space (Kernel):** $\mathcal{N}(\mathbf{A}) = \text{span}\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\} \subset \mathbb{R}^n$.

Proof *Proof.* From $\mathbf{A} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, we have $\mathbf{A}\mathbf{v}_i = \sigma_i \mathbf{u}_i$ for $i = 1, \dots, r$, and $\mathbf{A}\mathbf{v}_i = \mathbf{0}$ for $i = r+1, \dots, n$. Any vector $\mathbf{x} \in \mathbb{R}^n$ can be expanded in the orthonormal basis $\{\mathbf{v}_1, \dots, \mathbf{v}_n\}$ as $\mathbf{x} = \sum_{i=1}^n (\mathbf{v}_i^T \mathbf{x}) \mathbf{v}_i$. Applying $\mathbf{A}$ gives

$$\mathbf{A}\mathbf{x} = \sum_{i=1}^n (\mathbf{v}_i^T \mathbf{x}) \mathbf{A}\mathbf{v}_i = \sum_{i=1}^r \sigma_i (\mathbf{v}_i^T \mathbf{x}) \mathbf{u}_i.$$

This expresses $\mathbf{A}\mathbf{x}$ as a linear combination of $\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$, proving that $\mathcal{R}(\mathbf{A}) = \text{span}\{\mathbf{u}_1, \dots, \mathbf{u}_r\}$. Furthermore, $\mathbf{A}\mathbf{x} = \mathbf{0}$ if and only if $\mathbf{v}_i^T \mathbf{x} = 0$ for all $i = 1, \dots, r$, which proves that $\mathcal{N}(\mathbf{A}) = \text{span}\{\mathbf{v}_{r+1}, \dots, \mathbf{v}_n\}$. Repeating the argument for $\mathbf{A}^T = \mathbf{V}\mathbf{\Sigma}^T \mathbf{U}^T$ completes the proof for the row space and left null space. $\square$

4.2.2 Low-Rank Matrix Approximation

One of the most important practical applications of the SVD is optimal low-rank matrix approximation, which forms the computational basis for Principal Component Analysis, latent semantic indexing, and data compression.

Thm 4.6 (Eckart-Young-Mirsky Theorem) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ have singular value decomposition $\mathbf{A} = \sum_{i=1}^r \sigma_i \mathbf{u}_i \mathbf{v}_i^T$. For any integer $k < r$, let the truncated SVD be defined by

$$\mathbf{A}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T.$$

Then $\mathbf{A}_k$ is the best rank- k approximation to $\mathbf{A}$ in both the spectral norm and the Frobenius norm:

$$\begin{aligned} \min_{\text{rank}(\mathbf{B}) \leq k} \|\mathbf{A} - \mathbf{B}\|_2 &= \|\mathbf{A} - \mathbf{A}_k\|_2 = \sigma_{k+1}, \\ \min_{\text{rank}(\mathbf{B}) \leq k} \|\mathbf{A} - \mathbf{B}\|_F &= \|\mathbf{A} - \mathbf{A}_k\|_F = \sqrt{\sum_{j=k+1}^r \sigma_j^2}. \end{aligned}$$

Remark 4.1 (Numerical rank and noise filtering) In practical data analysis, singular values rarely equal zero exactly due to measurement noise and roundoff. The **numerical rank** of $\mathbf{A}$ is defined as the number of singular values exceeding a tolerance threshold τ :

$$r_{\text{num}} = \max\{i : \sigma_i > \tau\}, \quad \tau = \varepsilon_{\text{mach}} \max(m, n) \sigma_1.$$

Truncating the singular value expansion at $k = r_{\text{num}}$ removes noise components and extracts the dominant low-dimensional dynamical modes.

4.2.3 The Moore-Penrose Pseudoinverse

Def 4.7 (Moore-Penrose Pseudoinverse) Let $\mathbf{A} \in \mathbb{R}^{m \times n}$ have thin SVD $\mathbf{A} = \mathbf{U}_1 \mathbf{\Sigma}_1 \mathbf{V}_1^T$. The Moore-Penrose pseudoinverse of $\mathbf{A}$ is defined by

$$\mathbf{A}^\dagger = \mathbf{V}_1 \mathbf{\Sigma}_1^{-1} \mathbf{U}_1^T = \sum_{i=1}^r \frac{1}{\sigma_i} \mathbf{v}_i \mathbf{u}_i^T.$$

For any right-hand side vector $\mathbf{b} \in \mathbb{R}^m$, the vector $\hat{\mathbf{x}} = \mathbf{A}^\dagger \mathbf{b}$ is the unique solution that minimizes $\|\mathbf{A}\mathbf{x} - \mathbf{b}\|_2$, and has minimal Euclidean norm $\|\hat{\mathbf{x}}\|_2$ among all least-squares minimizers.

Example (Step-by-step SVD calculation) Consider the rank-1 matrix

$$\mathbf{A} = \begin{pmatrix} 3 & 3 \\ 4 & 4 \end{pmatrix}.$$

We compute $\mathbf{A}^T \mathbf{A}$:

$$\mathbf{A}^T \mathbf{A} = \begin{pmatrix} 3 & 4 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} 3 & 3 \\ 4 & 4 \end{pmatrix} = \begin{pmatrix} 25 & 25 \\ 25 & 25 \end{pmatrix}.$$

The eigenvalues of $\mathbf{A}^T \mathbf{A}$ are $\lambda_1 = 50$ and $\lambda_2 = 0$. The singular values are $\sigma_1 = \sqrt{50} = 5\sqrt{2}$ and $\sigma_2 = 0$. The normalized eigenvector corresponding to $\lambda_1 = 50$ is $\mathbf{v}_1 = \frac{1}{\sqrt{2}}(1, 1)^T$. The first left singular vector is

$$\mathbf{u}_1 = \frac{1}{\sigma_1} \mathbf{A} \mathbf{v}_1 = \frac{1}{5\sqrt{2}} \begin{pmatrix} 3 & 3 \\ 4 & 4 \end{pmatrix} \begin{pmatrix} 1/\sqrt{2} \\ 1/\sqrt{2} \end{pmatrix} = \frac{1}{10} \begin{pmatrix} 6 \\ 8 \end{pmatrix} = \begin{pmatrix} 3/5 \\ 4/5 \end{pmatrix}.$$

The thin SVD is $\mathbf{A} = \sigma_1 \mathbf{u}_1 \mathbf{v}_1^T = (5\sqrt{2}) \begin{pmatrix} 3/5 \\ 4/5 \end{pmatrix} (1/\sqrt{2} \quad 1/\sqrt{2})$.

Ex 4.2

1. Compute the singular values and singular vectors by hand for $\mathbf{A} = \begin{pmatrix} 1 & 1 \\ 0 & 1 \end{pmatrix}$.
2. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be invertible. Prove that $\|\mathbf{A}\|_2 = \sigma_{\max}$ and $\|\mathbf{A}^{-1}\|_2 = 1/\sigma_{\min}$, confirming that $\kappa_2(\mathbf{A}) = \sigma_{\max}/\sigma_{\min}$.
3. Prove that the pseudoinverse satisfies the four Moore-Penrose conditions: (1) $\mathbf{A} \mathbf{A}^\dagger \mathbf{A} = \mathbf{A}$, (2) $\mathbf{A}^\dagger \mathbf{A} \mathbf{A}^\dagger = \mathbf{A}^\dagger$, (3) $(\mathbf{A} \mathbf{A}^\dagger)^T = \mathbf{A} \mathbf{A}^\dagger$, and (4) $(\mathbf{A}^\dagger \mathbf{A})^T = \mathbf{A}^\dagger \mathbf{A}$.
4. Load an image as a matrix in Python. Compute its SVD and plot the relative Frobenius approximation error as a function of the truncation rank k .

4.3 The Lanczos Algorithm

The Lanczos algorithm is the symmetric specialization of the Arnoldi iteration for large, sparse real symmetric matrices $\mathbf{A} = \mathbf{A}^T \in \mathbb{R}^{n \times n}$. When $\mathbf{A}$ is symmetric, the upper Hessenberg projection matrix becomes symmetric tridiagonal, collapsing the k -term Gram-Schmidt orthogonalization into a three-term recurrence involving only the two immediately preceding basis vectors.

By projecting the large $n \times n$ matrix onto an expanding k -dimensional Krylov subspace ($k \ll n$), the Lanczos algorithm rapidly computes extremal eigenvalues (the largest and smallest frequencies in the spectrum) using only one sparse matrix-vector product per step and minimal vector memory.

4.3.1 The Three-Term Lanczos Recurrence

Thm 4.7 (The Lanczos Factorization) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be real symmetric, and let $\mathbf{v}_1 \in \mathbb{R}^n$ be a normalized initial vector ($\|\mathbf{v}_1\|_2 = 1$). Set $\mathbf{v}_0 = \mathbf{0}$ and $\beta_0 = 0$. For $j = 1, 2, \dots, k$:

1. Compute the matrix-vector product: $\mathbf{w} = \mathbf{A}\mathbf{v}_j$.
2. Compute the diagonal Rayleigh quotient: $\alpha_j = \mathbf{v}_j^T \mathbf{w}$.
3. Orthogonalize against the previous two vectors: $\mathbf{w} \leftarrow \mathbf{w} - \alpha_j \mathbf{v}_j - \beta_{j-1} \mathbf{v}_{j-1}$.
4. Compute the subdiagonal norm: $\beta_j = \|\mathbf{w}\|_2$.
5. If $\beta_j = 0$, terminate. Otherwise, set $\mathbf{v}_{j+1} = \mathbf{w}/\beta_j$.

In matrix notation, after k steps, the orthonormal Lanczos basis $\mathbf{V}_k = [\mathbf{v}_1, \dots, \mathbf{v}_k]$ satisfies the algebraic relation

$$\mathbf{A}\mathbf{V}_k = \mathbf{V}_k \mathbf{T}_k + \beta_k \mathbf{v}_{k+1} \mathbf{e}_k^T,$$

where $\mathbf{T}_k \in \mathbb{R}^{k \times k}$ is the symmetric tridiagonal matrix

$$\mathbf{T}_k = \begin{pmatrix} \alpha_1 & \beta_1 & 0 & \dots & 0 \\ \beta_1 & \alpha_2 & \beta_2 & \dots & 0 \\ 0 & \beta_2 & \alpha_3 & \ddots & \vdots \\ \vdots & \vdots & \ddots & \ddots & \beta_{k-1} \\ 0 & 0 & \dots & \beta_{k-1} & \alpha_k \end{pmatrix}.$$

Prop 4.8 (Galerkin Projection Property) In exact arithmetic, the Lanczos basis vectors are mutually orthonormal ($\mathbf{V}_k^T \mathbf{V}_k = \mathbf{I}_k$). Multiplying the Lanczos relation on the left by $\mathbf{V}_k^T$ yields the Rayleigh-Ritz Galerkin projection:

$$\mathbf{T}_k = \mathbf{V}_k^T \mathbf{A} \mathbf{V}_k.$$

4.3.2 Ritz Values, Ritz Vectors, and Error Bounds

Def 4.8 (Ritz Pairs) Let $\mathbf{T}_k \mathbf{s}_i = \theta_i \mathbf{s}_i$ be the eigendecomposition of the small symmetric tridiagonal matrix $\mathbf{T}_k$, with normalized eigenvectors $\|\mathbf{s}_i\|_2 = 1$.

1. The eigenvalue $\theta_i \in \mathbb{R}$ is termed the *i-th Ritz value* of $\mathbf{A}$ with respect to $\mathcal{K}_k(\mathbf{A}, \mathbf{v}_1)$.
2. The vector $\tilde{\mathbf{u}}_i = \mathbf{V}_k \mathbf{s}_i \in \mathbb{R}^n$ is termed the *i-th Ritz vector*.

Thm 4.9 (A Posteriori Residual Bound for Ritz Pairs) The residual of the Ritz pair $(\theta_i, \tilde{\mathbf{u}}_i)$ satisfies the explicit identity

$$\|\mathbf{A}\tilde{\mathbf{u}}_i - \theta_i \tilde{\mathbf{u}}_i\|_2 = \beta_k |s_{k,i}|,$$

where $s_{k,i}$ is the final component of the i -th tridiagonal eigenvector $\mathbf{s}_i \in \mathbb{R}^k$.

Proof *Proof.* Multiplying the Lanczos relation $\mathbf{A}\mathbf{V}_k = \mathbf{V}_k\mathbf{T}_k + \beta_k\mathbf{v}_{k+1}\mathbf{e}_k^T$ on the right by $\mathbf{s}_i$ gives

$$\mathbf{A}(\mathbf{V}_k\mathbf{s}_i) = \mathbf{V}_k(\mathbf{T}_k\mathbf{s}_i) + \beta_k\mathbf{v}_{k+1}(\mathbf{e}_k^T\mathbf{s}_i) \implies \mathbf{A}\tilde{\mathbf{u}}_i = \theta_i\tilde{\mathbf{u}}_i + \beta_k s_{k,i}\mathbf{v}_{k+1}.$$

Rearranging and taking the Euclidean norm yields

$$\|\mathbf{A}\tilde{\mathbf{u}}_i - \theta_i\tilde{\mathbf{u}}_i\|_2 = \|\beta_k s_{k,i}\mathbf{v}_{k+1}\|_2 = \beta_k |s_{k,i}| \|\mathbf{v}_{k+1}\|_2 = \beta_k |s_{k,i}|,$$

because $\|\mathbf{v}_{k+1}\|_2 = 1$. This allows monitoring the convergence of every Ritz pair for $O(k)$ operations without forming the n -dimensional Ritz vectors $\tilde{\mathbf{u}}_i$. $\square$

4.3.3 Loss of Orthogonality and Ghost Eigenvalues

Remark 4.2 (Paige’s theorem on loss of orthogonality) In floating-point arithmetic, the three-term recurrence suffers from a fundamental numerical instability discovered by Chris Paige. The basis vectors $\mathbf{v}_j$ do not remain orthogonal; rather, orthogonality is lost rapidly precisely as individual Ritz values converge to extreme eigenvalues of $\mathbf{A}$. As a consequence, once an eigenvalue has converged, the algorithm continues to re-discover it in subsequent iterations, generating multiple spurious copies known as **ghost eigenvalues**.

Remark 4.3 (Practical remedies for ghost eigenvalues) To maintain numerical reliability on large problems, scientific codes employ one of three strategies:

1. **Full Reorthogonalization:** Reorthogonalize $\mathbf{v}_{j+1}$ against all previous basis vectors $\mathbf{v}_1, \dots, \mathbf{v}_j$ using Modified Gram-Schmidt at every step ($O(k^2n)$ cost).
2. **Selective Reorthogonalization:** Orthogonalize only against converged Ritz vectors whenever loss of orthogonality is detected ($O(k \cdot n)$ cost).
3. **Cullum-Willoughby Identification:** Run the unorthogonalized Lanczos iteration and filter out ghost eigenvalues by comparing the spectrum of $\mathbf{T}_k$ with the spectrum of the submatrix formed by deleting the first row and column of $\mathbf{T}_k$.

Ex 4.3

1. Execute two steps of the Lanczos algorithm by hand for the diagonal matrix $\mathbf{A} = \text{diag}(4, 2, 1)$ starting with the normalized vector $\mathbf{v}_1 = \frac{1}{\sqrt{3}}(1, 1, 1)^T$. Explicitly write down $\alpha_1, \beta_1, \alpha_2$, and the 2×2 tridiagonal matrix $\mathbf{T}_2$.
2. For the 2×2 matrix $\mathbf{T}_2$ computed above, find its Ritz values and evaluate the residual error bound using Thm 4.9.
3. Prove that the Lanczos algorithm applied to the shifted system $(\mathbf{A} - \sigma\mathbf{I})$ produces the identical basis vectors $\mathbf{V}_k$ as the original system, with tridiagonal matrix $\mathbf{T}_k - \sigma\mathbf{I}$.

4.4 The Jacobi Eigenvalue Algorithm

The Jacobi eigenvalue algorithm, introduced by Carl Gustav Jacob Jacobi in 1846, is the oldest numerical method for computing the complete eigensystem of a real symmetric matrix $\mathbf{A} = \mathbf{A}^T \in \mathbb{R}^{n \times n}$. The method systematically drives the matrix toward diagonal form by applying a sequence of orthogonal similarity transformations based on elementary plane rotations.

Although the shifted QR algorithm is computationally faster for general dense matrices, the Jacobi method remains of profound importance in scientific computing. It achieves high relative accuracy for small eigenvalues of graded positive definite matrices that QR cannot match, and its localized transformations allow for massive fine-grained parallelization on modern GPU and systolic architectures.

4.4.1 Givens Plane Rotations and the Zeroing Angle

At each step, the algorithm chooses an off-diagonal target entry a_{pq} with $p < q$ and constructs a Givens plane rotation $\mathbf{G}(p, q, \theta)$ that annihilates that entry while preserving the overall spectrum of $\mathbf{A}$.

Def 4.9 (Givens Plane Rotation Matrix) The matrix $\mathbf{G}(p, q, \theta) \in \mathbb{R}^{n \times n}$ is an identity matrix except for the four entries at the intersections of rows and columns p and q :

$$g_{pp} = c, \quad g_{pq} = s, \quad g_{qp} = -s, \quad g_{qq} = c,$$

where $c = \cos \theta$ and $s = \sin \theta$. The transformation $\mathbf{A}' = \mathbf{G}^T \mathbf{A} \mathbf{G}$ modifies only rows and columns p and q of $\mathbf{A}$.

Thm 4.10 (Derivation of the Optimal Rotation Angle) To zero the transformed off-diagonal entry $a'_{pq} = 0$, we consider the 2×2 orthogonal similarity transformation on the submatrix at indices $\{p, q\}$:

$$\begin{pmatrix} a'_{pp} & a'_{pq} \\ a'_{pq} & a'_{qq} \end{pmatrix} = \begin{pmatrix} c & -s \\ s & c \end{pmatrix} \begin{pmatrix} a_{pp} & a_{pq} \\ a_{pq} & a_{qq} \end{pmatrix} \begin{pmatrix} c & s \\ -s & c \end{pmatrix}.$$

Multiplying the matrices yields the off-diagonal formula

$$a'_{pq} = (c^2 - s^2)a_{pq} + cs(a_{pp} - a_{qq}) = a_{pq} \cos(2\theta) + \frac{1}{2}(a_{pp} - a_{qq}) \sin(2\theta).$$

Setting $a'_{pq} = 0$ yields the trigonometric condition

$$\cot(2\theta) = \frac{a_{qq} - a_{pp}}{2a_{pq}} \equiv \tau.$$

Using the identity $\cot(2\theta) = \frac{1 - \tan^2 \theta}{2 \tan \theta}$, let $t = \tan \theta$. This produces the quadratic equation

$$t^2 + 2\tau t - 1 = 0.$$

To minimize the perturbation to $\mathbf{A}$ and prevent catastrophic cancellation, we select the root of smaller absolute value:

$$t = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}, \quad c = \frac{1}{\sqrt{1 + t^2}}, \quad s = ct.$$

Proof *Proof.* The standard quadratic formula gives roots $t = -\tau \pm \sqrt{1 + \tau^2}$. Multiplying the root of smaller magnitude by its conjugate algebraic expression yields

$$t = \frac{(-\tau \pm \sqrt{1 + \tau^2})(\tau \pm \sqrt{1 + \tau^2})}{\tau \pm \sqrt{1 + \tau^2}} = \frac{1}{\tau + \text{sign}(\tau)\sqrt{1 + \tau^2}} = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}.$$

This formula satisfies $|t| \leq 1$, corresponding to a rotation angle $|\theta| \leq \pi/4$, and avoids subtracting nearly equal quantities. $\square$

4.4.2 Monotone Convergence of the Off-Diagonal Norm

Thm 4.11 (**Monotone Decrease of the Off-Diagonal Frobenius Norm**) Let the off-diagonal squared norm be defined by

$$\text{off}(\mathbf{A})^2 = \sum_{i \neq j} a_{ij}^2 = \|\mathbf{A}\|_F^2 - \sum_{i=1}^n a_{ii}^2.$$

Applying the Jacobi rotation $\mathbf{A}' = \mathbf{G}(p, q, \theta)^T \mathbf{A} \mathbf{G}(p, q, \theta)$ with the optimal angle strictly reduces the off-diagonal norm:

$$\text{off}(\mathbf{A}')^2 = \text{off}(\mathbf{A})^2 - 2a_{pq}^2.$$

Proof *Proof.* Because $\mathbf{G}$ is orthogonal, the total Frobenius norm is preserved: $\|\mathbf{A}'\|_F^2 = \|\mathbf{A}\|_F^2$. The rotation affects only entries in rows and columns p and q . For the 2×2 submatrix, the sum of squares is invariant under orthogonal similarity:

$$(a'_{pp})^2 + (a'_{qq})^2 + 2(a'_{pq})^2 = a_{pp}^2 + a_{qq}^2 + 2a_{pq}^2.$$

Because the rotation enforces $a'_{pq} = 0$, the sum of squared diagonal entries increases:

$$(a'_{pp})^2 + (a'_{qq})^2 = a_{pp}^2 + a_{qq}^2 + 2a_{pq}^2.$$

All other diagonal entries a_{ii} ($i \neq p, q$) are untouched. Substituting this into $\text{off}(\mathbf{A}')^2 = \|\mathbf{A}'\|_F^2 - \sum (a'_{ii})^2$ proves that $\text{off}(\mathbf{A}')^2 = \text{off}(\mathbf{A})^2 - 2a_{pq}^2$. $\square$

Thm 4.12 (**The Cyclic Jacobi Algorithm**) The Cyclic Jacobi algorithm sweeps through all $N = n(n-1)/2$ off-diagonal index pairs (p, q) with $1 \leq p < q \leq n$ in a fixed periodic order:

1. Initialize $\mathbf{A}^{(0)} = \mathbf{A}$ and eigenvector accumulator $\mathbf{V}^{(0)} = \mathbf{I}_n$.
2. While $\text{off}(\mathbf{A}^{(k)}) > \text{tol} \|\mathbf{A}^{(0)}\|_F$:
 - (a) For each pair $1 \leq p < q \leq n$:
 - i. Compute $\tau = \frac{a_{qq} - a_{pp}}{2a_{pq}}$, $t = \frac{\text{sign}(\tau)}{|\tau| + \sqrt{1 + \tau^2}}$, $c = \frac{1}{\sqrt{1 + t^2}}$, and $s = ct$.
 - ii. Apply the rotation to $\mathbf{A}$: $\mathbf{A} \leftarrow \mathbf{G}(p, q, \theta)^T \mathbf{A} \mathbf{G}(p, q, \theta)$.
 - iii. Accumulate the eigenvectors: $\mathbf{V} \leftarrow \mathbf{V} \mathbf{G}(p, q, \theta)$.

After a few initial sweeps, the off-diagonal entries converge to zero **quadratically**: $\text{off}(\mathbf{A}^{(k+1)}) \leq C \cdot \text{off}(\mathbf{A}^{(k)})^2$.

Ex 4.4

1. Perform one Jacobi rotation by hand to zero out the off-diagonal entry of

$$\mathbf{A} = \begin{pmatrix} 3 & 2 \\ 2 & 6 \end{pmatrix}.$$

Explicitly compute τ, t, c, s , and the resulting diagonal matrix $\mathbf{A}'$.

2. Verify that the eigenvalues computed on the diagonal of $\mathbf{A}'$ match the exact roots of $\det(\mathbf{A} - \lambda \mathbf{I}) = 0$.
3. Prove that for an $n \times n$ matrix, if the largest off-diagonal entry is selected at each step (Classical Jacobi), then $\text{off}(\mathbf{A}^{(k+1)})^2 \leq \left(1 - \frac{2}{n(n-1)}\right) \text{off}(\mathbf{A}^{(k)})^2$.

5 Iterative Methods for Linear Systems

5.1 Iterative Methods for Linear Systems

Direct methods such as Gaussian elimination and Cholesky factorization compute exact solutions up to machine roundoff in a finite number of arithmetic operations. However, for the very large linear systems arising from the discretization of three-dimensional partial differential equations or network models, direct factorizations become computationally prohibitive. Direct factorizations suffer from fill-in, in which sparse matrices produce dense triangular factors that exceed available computer memory.

Iterative methods provide an essential alternative. Instead of transforming the matrix through elimination, an iterative solver generates a sequence of improving approximations $\mathbf{x}^{(0)}, \mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \dots$ that converges toward the true solution $\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$. The fundamental computational primitive of an iterative method is the matrix-vector multiplication $\mathbf{v} \mapsto \mathbf{A}\mathbf{v}$. Because sparse matrix-vector products require only $O(\text{nnz})$ operations and zero additional memory, iterative methods scale efficiently to millions of degrees of freedom.

5.1.1 Matrix Splittings and Stationary Iterations

The classical approach to designing an iterative solver is matrix splitting. We decompose the coefficient matrix $\mathbf{A}$ into the difference of two matrices,

$$\mathbf{A} = \mathbf{M} - \mathbf{N},$$

where the splitting matrix $\mathbf{M}$ is nonsingular and easily invertible. The linear system $\mathbf{A}\mathbf{x} = \mathbf{b}$ is then algebraically equivalent to $\mathbf{M}\mathbf{x} = \mathbf{N}\mathbf{x} + \mathbf{b}$.

Def 5.1 (Stationary Iteration) Given a splitting $\mathbf{A} = \mathbf{M} - \mathbf{N}$, the corresponding stationary iteration computes successive approximations via the fixed-point recurrence

$$\mathbf{M}\mathbf{x}^{(k+1)} = \mathbf{N}\mathbf{x}^{(k)} + \mathbf{b} \iff \mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c},$$

where $\mathbf{T} = \mathbf{M}^{-1}\mathbf{N} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$ is the **iteration matrix** and $\mathbf{c} = \mathbf{M}^{-1}\mathbf{b}$ is the modified right-hand side vector.

Remark 5.1 (Fixed-point interpretation) A stationary iteration is a linear fixed-point mapping. The exact solution $\mathbf{x}^*$ satisfies $\mathbf{x}^* = \mathbf{T}\mathbf{x}^* + \mathbf{c}$. The effectiveness of the method depends entirely on the choice of $\mathbf{M}$: the matrix $\mathbf{M}$ should be as close to $\mathbf{A}$ as possible to minimize the magnitude of $\mathbf{T}$, while remaining computationally cheap to invert at each step.

5.1.2 Convergence Theory and the Spectral Radius

The convergence of a linear stationary iteration is determined entirely by the spectral properties of the iteration matrix $\mathbf{T}$.

Def 5.2 (Spectral Radius) Let $\mathbf{T} \in \mathbb{R}^{n \times n}$ with spectrum $\sigma(\mathbf{T}) \subset \mathbb{C}$. The spectral radius of $\mathbf{T}$ is defined as the maximum modulus among all its eigenvalues:

$$\rho(\mathbf{T}) = \max\{|\lambda| : \lambda \in \sigma(\mathbf{T})\}.$$

Thm 5.1 (Gelfand's Spectral Radius Theorem) For any matrix norm $\|\cdot\|$ induced by a vector norm on $\mathbb{C}^n$ and any matrix $\mathbf{T} \in \mathbb{R}^{n \times n}$,

$$\rho(\mathbf{T}) = \lim_{k \rightarrow \infty} \|\mathbf{T}^k\|^{1/k}.$$

In particular, for any $\varepsilon > 0$, there exists an induced matrix norm $\|\cdot\|_*$ such that $\|\mathbf{T}\|_* \leq \rho(\mathbf{T}) + \varepsilon$.

Thm 5.2 (Fundamental Convergence Criterion for Stationary Iterations) The stationary iteration $\mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c}$ converges to the unique solution $\mathbf{x}^* = \mathbf{A}^{-1}\mathbf{b}$ for every initial guess $\mathbf{x}^{(0)} \in \mathbb{R}^n$ if and only if

$$\rho(\mathbf{T}) < 1.$$

Proof *Proof.* Let $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*$ denote the error vector at step k . Subtracting the fixed-point identity $\mathbf{x}^* = \mathbf{T}\mathbf{x}^* + \mathbf{c}$ from the iteration formula $\mathbf{x}^{(k+1)} = \mathbf{T}\mathbf{x}^{(k)} + \mathbf{c}$ yields the homogeneous linear error recurrence

$$\mathbf{e}^{(k+1)} = \mathbf{T}\mathbf{e}^{(k)}.$$

Applying this recurrence recursively from the initial error $\mathbf{e}^{(0)}$ gives

$$\mathbf{e}^{(k)} = \mathbf{T}^k \mathbf{e}^{(0)}.$$

The error $\mathbf{e}^{(k)}$ converges to the zero vector for every choice of $\mathbf{e}^{(0)}$ if and only if the matrix sequence $\mathbf{T}^k \rightarrow \mathbf{0}$ as $k \rightarrow \infty$. By the Jordan canonical form, every Jordan block $J_i = \lambda_i \mathbf{I} + N_i$ satisfies $J_i^k \rightarrow \mathbf{0}$ if and only if $|\lambda_i| < 1$. Therefore, $\lim_{k \rightarrow \infty} \mathbf{T}^k = \mathbf{0}$ if and only if every eigenvalue satisfies $|\lambda_i| < 1$, which is precisely the condition $\rho(\mathbf{T}) < 1$. $\square$

Remark 5.2 (Asymptotic convergence rate and iteration count) The quantity $-\log_{10} \rho(\mathbf{T})$ represents the asymptotic rate of convergence, measuring the number of decimal digits of accuracy gained per iteration. To reduce the initial error by a relative tolerance factor ε , the required number of iterations satisfies

$$k \approx \frac{\ln(1/\varepsilon)}{-\ln \rho(\mathbf{T})}.$$

If $\rho(\mathbf{T}) = 0.99$, we have $-\ln(0.99) \approx 0.01005$, which requires approximately 230 iterations to reduce the error by a factor of ten. As $\rho(\mathbf{T}) \rightarrow 1$, convergence slows drastically.

5.1.3 Krylov Subspaces and Matrix-Free Solvers

While stationary methods rely on fixed matrix splittings, modern iterative methods project the problem onto expanding polynomial subspaces generated by the initial residual.

Def 5.3 (Krylov Subspace) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ and let $\mathbf{r}^{(0)} \in \mathbb{R}^n$ be a nonzero vector. The k -th Krylov subspace generated by $\mathbf{A}$ and $\mathbf{r}^{(0)}$ is the linear subspace

$$\mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)}) = \text{span} \{ \mathbf{r}^{(0)}, \mathbf{A}\mathbf{r}^{(0)}, \mathbf{A}^2\mathbf{r}^{(0)}, \dots, \mathbf{A}^{k-1}\mathbf{r}^{(0)} \}.$$

Thm 5.3 (Krylov Polynomial Representation) Let $\mathbf{x}^{(0)}$ be an initial guess with residual $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(0)}$. Any iterate $\mathbf{x}^{(k)}$ belonging to the affine Krylov space $\mathbf{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)})$ can be represented as

$$\mathbf{x}^{(k)} = \mathbf{x}^{(0)} + p_{k-1}(\mathbf{A})\mathbf{r}^{(0)},$$

where p_{k-1} is a polynomial of degree at most $k-1$. The corresponding residual vector satisfies

$$\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)} = q_k(\mathbf{A})\mathbf{r}^{(0)},$$

where $q_k(t) = 1 - tp_{k-1}(t)$ is a polynomial of degree at most k satisfying the normalization constraint $q_k(0) = 1$.

Remark 5.3 (Matrix-free computation) Krylov subspace methods never require explicit access to the individual entries of $\mathbf{A}$. They interact with the linear operator exclusively by computing matrix-vector products $\mathbf{v} \mapsto \mathbf{A}\mathbf{v}$. Consequently, linear systems can be solved efficiently even when the matrix is defined implicitly as a differential operator or a numerical simulation routine.

Def 5.4 (Stopping Criteria for Iterative Solvers) Because the true error $\mathbf{e}^{(k)} = \mathbf{x}^{(k)} - \mathbf{x}^*$ is unobservable, termination is determined using the computable residual vector $\mathbf{r}^{(k)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(k)}$. Common stopping criteria include:

1. **Relative residual test:** $\frac{\|\mathbf{r}^{(k)}\|_2}{\|\mathbf{b}\|_2} \leq \text{tol}$.
2. **Preconditioned relative residual test:** $\frac{\|\mathbf{M}^{-1}\mathbf{r}^{(k)}\|_2}{\|\mathbf{M}^{-1}\mathbf{b}\|_2} \leq \text{tol}$.
3. **Step difference test:** $\frac{\|\mathbf{x}^{(k+1)} - \mathbf{x}^{(k)}\|_2}{\|\mathbf{x}^{(k+1)}\|_2} \leq \text{tol}$.

Ex 5.1

1. Consider the matrix $\mathbf{A} = \begin{pmatrix} 4 & 1 \\ 2 & 3 \end{pmatrix}$ and the splitting $\mathbf{M} = 4\mathbf{I}$. Compute the iteration matrix $\mathbf{T} = \mathbf{I} - \mathbf{M}^{-1}\mathbf{A}$, find its eigenvalues, and evaluate its spectral radius $\rho(\mathbf{T})$.
2. For the splitting above, determine the number of iterations required to reduce the initial error by a factor of 10^{-6} .
3. Prove that if $\|\mathbf{T}\| < 1$ for some induced matrix norm, then the error satisfies $\|\mathbf{e}^{(k)}\| \leq \frac{\|\mathbf{T}\|}{1 - \|\mathbf{T}\|} \|\mathbf{x}^{(k)} - \mathbf{x}^{(k-1)}\|$.
4. Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ and let $\mathbf{r}^{(0)} \in \mathbb{R}^n$. Prove that the sequence of dimensions $\dim \mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)})$ strictly increases by one at each step until it reaches an invariant subspace of $\mathbf{A}$, after which the dimension remains constant.

5.2 Jacobi, Gauss-Seidel, and SOR Methods

The classical stationary iterations represent the earliest systematic approaches to solving linear systems iteratively. They partition the matrix $\mathbf{A}$ into its diagonal, strictly lower triangular, and strictly upper triangular components:

$$\mathbf{A} = \mathbf{D} - \mathbf{L}_A - \mathbf{U}_A,$$

where $\mathbf{D} = \text{diag}(a_{11}, \dots, a_{nn})$, $-\mathbf{L}_A$ is the strictly lower triangular part of $\mathbf{A}$, and $-\mathbf{U}_A$ is the strictly upper triangular part of $\mathbf{A}$. By choosing different combinations of these components for the splitting matrix $\mathbf{M}$, we obtain the Jacobi, Gauss-Seidel, and Successive Over-Relaxation (SOR) algorithms.

5.2.1 The Jacobi Method

The Jacobi method assumes that the diagonal elements dominate the matrix. It computes the $(k+1)$ -th estimate of each unknown x_i exclusively from the k -th estimates of all other variables.

Def 5.5 (Jacobi Iteration) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ with nonzero diagonal entries $a_{ii} \neq 0$. The Jacobi method corresponds to the splitting $\mathbf{M} = \mathbf{D}$ and $\mathbf{N} = \mathbf{L}_A + \mathbf{U}_A$. In component form, the update is

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n.$$

In matrix notation, the iteration is given by

$$\mathbf{x}^{(k+1)} = \mathbf{T}_J \mathbf{x}^{(k)} + \mathbf{c}_J,$$

where $\mathbf{T}_J = \mathbf{D}^{-1}(\mathbf{L}_A + \mathbf{U}_A) = \mathbf{I} - \mathbf{D}^{-1}\mathbf{A}$ is the Jacobi iteration matrix and $\mathbf{c}_J = \mathbf{D}^{-1}\mathbf{b}$.

Remark 5.4 (Parallel execution and memory) Because each component $x_i^{(k+1)}$ depends only on values from the previous iteration vector $\mathbf{x}^{(k)}$, all n components can be computed concurrently. This fine-grained parallelism makes Jacobi attractive on vector and GPU hardware. However, it requires double buffering in memory to store $\mathbf{x}^{(k)}$ and $\mathbf{x}^{(k+1)}$ simultaneously.

5.2.2 The Gauss-Seidel Method

The Gauss-Seidel method improves upon Jacobi by utilizing updated variable values as soon as they become available within the current sweep.

Def 5.6 (Gauss-Seidel Iteration) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ with nonzero diagonal entries. The Gauss-Seidel method corresponds to the triangular splitting $\mathbf{M} = \mathbf{D} - \mathbf{L}_A$ and $\mathbf{N} = \mathbf{U}_A$. In component form, the update is

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij} x_j^{(k+1)} - \sum_{j > i} a_{ij} x_j^{(k)} \right), \quad i = 1, 2, \dots, n.$$

In matrix notation, the iteration is given by

$$(\mathbf{D} - \mathbf{L}_A) \mathbf{x}^{(k+1)} = \mathbf{U}_A \mathbf{x}^{(k)} + \mathbf{b} \iff \mathbf{x}^{(k+1)} = \mathbf{T}_{GS} \mathbf{x}^{(k)} + \mathbf{c}_{GS},$$

where $\mathbf{T}_{GS} = (\mathbf{D} - \mathbf{L}_A)^{-1}\mathbf{U}_A$ is the Gauss-Seidel iteration matrix and $\mathbf{c}_{GS} = (\mathbf{D} - \mathbf{L}_A)^{-1}\mathbf{b}$.

Remark 5.5 (Sequential dependencies and in-place updates) Because the calculation of $x_i^{(k+1)}$ requires the newly computed values $x_1^{(k+1)}, \dots, x_{i-1}^{(k+1)}$, the standard Gauss-Seidel sweep is inherently sequential. However, it allows for in-place memory updates: new values immediately overwrite old values in a single vector array. For many classes of matrices, Gauss-Seidel converges approximately twice as fast as Jacobi.

Example (Comparison of one Jacobi and Gauss-Seidel step) Consider the linear system

$$\begin{pmatrix} 4 & 1 & 0 \\ 1 & 4 & 1 \\ 0 & 1 & 4 \end{pmatrix} \begin{pmatrix} x_1 \\ x_2 \\ x_3 \end{pmatrix} = \begin{pmatrix} 5 \\ 6 \\ 5 \end{pmatrix},$$

whose exact solution is $\mathbf{x}^* = (1, 1, 1)^T$. Let the initial guess be $\mathbf{x}^{(0)} = (0, 0, 0)^T$.

• **Jacobi Step:**

$$\begin{aligned} x_1^{(1)} &= \frac{1}{4}(5 - 0 - 0) = 1.25, \\ x_2^{(1)} &= \frac{1}{4}(6 - 0 - 0) = 1.50, \\ x_3^{(1)} &= \frac{1}{4}(5 - 0 - 0) = 1.25, \quad \implies \mathbf{x}_J^{(1)} = \begin{pmatrix} 1.25 \\ 1.50 \\ 1.25 \end{pmatrix}. \end{aligned}$$

• **Gauss-Seidel Step:**

$$\begin{aligned} x_1^{(1)} &= \frac{1}{4}(5 - 0 - 0) = 1.25, \\ x_2^{(1)} &= \frac{1}{4}(6 - (1.25) - 0) = \frac{4.75}{4} = 1.1875, \\ x_3^{(1)} &= \frac{1}{4}(5 - (1.1875) - 0) = \frac{3.8125}{4} \approx 0.9531, \quad \implies \mathbf{x}_{GS}^{(1)} \approx \begin{pmatrix} 1.2500 \\ 1.1875 \\ 0.9531 \end{pmatrix}. \end{aligned}$$

The Gauss-Seidel iterate is significantly closer to the true solution $(1, 1, 1)^T$ because the update for x_2 immediately leveraged the new value of x_1 , and the update for x_3 leveraged the new value of x_2 .

5.2.3 Convergence Theorems

Thm 5.4 (Convergence Under Strict Diagonal Dominance) A matrix $\mathbf{A} \in \mathbb{R}^{n \times n}$ is strictly diagonally dominant if

$$|a_{ii}| > \sum_{j \neq i} |a_{ij}| \quad \text{for all } i = 1, 2, \dots, n.$$

If $\mathbf{A}$ is strictly diagonally dominant, then both the Jacobi iteration and the Gauss-Seidel iteration converge for any initial guess $\mathbf{x}^{(0)} \in \mathbb{R}^n$.

Proof *Proof.* We prove the result for the Jacobi iteration by showing that the matrix ∞ -norm of the iteration matrix $\mathbf{T}_J$ is strictly less than one. The entries of $\mathbf{T}_J = \mathbf{D}^{-1}(\mathbf{L}_A + \mathbf{U}_A)$ are given by $(\mathbf{T}_J)_{ij} = -a_{ij}/a_{ii}$ for $i \neq j$, and $(\mathbf{T}_J)_{ii} = 0$. The induced ∞ -norm is the maximum absolute row sum:

$$\|\mathbf{T}_J\|_\infty = \max_{1 \leq i \leq n} \sum_{j=1}^n |(\mathbf{T}_J)_{ij}| = \max_{1 \leq i \leq n} \sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|}.$$

Because $\mathbf{A}$ is strictly diagonally dominant, $\sum_{j \neq i} |a_{ij}| < |a_{ii}|$ for every i , which implies that $\sum_{j \neq i} \frac{|a_{ij}|}{|a_{ii}|} < 1$. Therefore, $\|\mathbf{T}_J\|_\infty < 1$. By Gelfand's spectral radius theorem, $\rho(\mathbf{T}_J) \leq \|\mathbf{T}_J\|_\infty < 1$, which establishes convergence. A similar argument applied to the componentwise error recurrence proves convergence for Gauss-Seidel. $\square$

Thm 5.5 (Convergence of Gauss-Seidel for Symmetric Positive Definite Systems) If $\mathbf{A} \in \mathbb{R}^{n \times n}$ is symmetric positive definite, then the Gauss-Seidel method converges for every initial vector $\mathbf{x}^{(0)} \in \mathbb{R}^n$. The Jacobi method, however, is not guaranteed to converge for arbitrary SPD matrices.

5.2.4 Successive Over-Relaxation (SOR)

The Successive Over-Relaxation method accelerates the convergence of Gauss-Seidel by taking a weighted linear combination of the previous iterate and the new Gauss-Seidel update.

Def 5.7 (Successive Over-Relaxation Scheme) Let $\omega > 0$ be a relaxation parameter. The SOR method computes the updated components via

$$x_i^{(k+1)} = (1 - \omega)x_i^{(k)} + \frac{\omega}{a_{ii}} \left(b_i - \sum_{j < i} a_{ij}x_j^{(k+1)} - \sum_{j > i} a_{ij}x_j^{(k)} \right).$$

In matrix notation, this corresponds to the splitting $\mathbf{M}_\omega = \frac{1}{\omega}\mathbf{D} - \mathbf{L}_A$ and $\mathbf{N}_\omega = (\frac{1}{\omega} - 1)\mathbf{D} + \mathbf{U}_A$. The iteration matrix is

$$\mathbf{T}_\omega = (\mathbf{D} - \omega\mathbf{L}_A)^{-1} ((1 - \omega)\mathbf{D} + \omega\mathbf{U}_A).$$

Thm 5.6 (Ostrowski-Reich Theorem and the Optimal Parameter) If $\mathbf{A}$ is symmetric positive definite, then the SOR iteration converges if and only if the relaxation parameter satisfies

$$0 < \omega < 2.$$

Furthermore, if $\mathbf{A}$ is consistently ordered (such as the standard tridiagonal or block-tridiagonal Poisson discretization matrices) and the Jacobi spectral radius is $\rho(\mathbf{T}_J) < 1$, then the optimal relaxation parameter that minimizes $\rho(\mathbf{T}_\omega)$ is

$$\omega^* = \frac{2}{1 + \sqrt{1 - \rho(\mathbf{T}_J)^2}}.$$

With this optimal choice, the spectral radius satisfies $\rho(\mathbf{T}_{\omega^*}) = \omega^* - 1$.

Remark 5.6 (Acceleration for discrete Poisson equations) For the 1D discrete Laplacian on a grid with spacing h , the Jacobi spectral radius is $\rho(\mathbf{T}_J) = \cos(\pi h) \approx 1 - \frac{1}{2}\pi^2 h^2$. Consequently, Jacobi and Gauss-Seidel require $O(h^{-2}) = O(n^2)$ iterations to converge. With optimal SOR, $\omega^* \approx 2 - 2\pi h$, yielding $\rho(\mathbf{T}_{\omega^*}) \approx 1 - 2\pi h$. This reduces the required iteration count to $O(h^{-1}) = O(n)$, providing an order-of-magnitude computational speedup.

Ex 5.2

1. Compute the Jacobi and Gauss-Seidel iteration matrices for $\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$. Find their spectral radii and verify that $\rho(\mathbf{T}_{GS}) = \rho(\mathbf{T}_J)^2$.
2. Let $\mathbf{A} = \begin{pmatrix} 1 & 2 \\ 2 & 1 \end{pmatrix}$. Show that $\mathbf{A}$ is symmetric but indefinite, and prove that both Jacobi and Gauss-Seidel diverge.
3. Prove the Kahan theorem: for any matrix $\mathbf{A}$ with nonzero diagonal entries, $\rho(\mathbf{T}_\omega) \geq |\omega - 1|$, which implies that SOR can only converge if $0 < \omega < 2$.
4. Write a Python script to solve the 1D discrete Poisson equation with $n = 50$ grid points using Jacobi, Gauss-Seidel, and optimal SOR. Plot the residual norm versus iteration number on a semilogarithmic scale.

5.3 The Conjugate Gradient Method

The Conjugate Gradient (CG) method of Hestenes and Stiefel is the premier iterative solver for large, sparse, symmetric positive definite linear systems. Unlike stationary iterations that converge linearly with rates determined by matrix splittings, the Conjugate Gradient method is an optimal Krylov subspace projection method. At step k , it computes the unique approximation $\mathbf{x}^{(k)}$ that minimizes the error in the $\mathbf{A}$ -norm over the entire affine Krylov subspace $\mathbf{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)})$.

Remarkably, despite optimizing over an expanding k -dimensional subspace, the symmetry of $\mathbf{A}$ allows each new iterate and search direction to be computed using a three-term recurrence involving only the immediately preceding step. The algorithm requires storing only four vectors of length n and computes only a single matrix-vector product per iteration.

5.3.1 Quadratic Optimization and Steepest Descent

The mathematical foundation of the Conjugate Gradient method lies in the equivalence between solving an SPD linear system and minimizing a convex quadratic energy functional.

Thm 5.7 (Variational Equivalence for SPD Systems) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be a symmetric positive definite matrix and let $\mathbf{b} \in \mathbb{R}^n$. Define the quadratic functional $\phi : \mathbb{R}^n \rightarrow \mathbb{R}$ by

$$\phi(\mathbf{x}) = \frac{1}{2} \mathbf{x}^T \mathbf{A} \mathbf{x} - \mathbf{b}^T \mathbf{x}.$$

The gradient of ϕ is $\nabla \phi(\mathbf{x}) = \mathbf{A} \mathbf{x} - \mathbf{b} = -\mathbf{r}(\mathbf{x})$, where $\mathbf{r}(\mathbf{x}) = \mathbf{b} - \mathbf{A} \mathbf{x}$ is the residual. Because $\mathbf{A}$ is positive definite, the Hessian matrix $\nabla^2 \phi(\mathbf{x}) = \mathbf{A}$ is strictly positive definite everywhere, and ϕ has a unique global minimizer $\mathbf{x}^*$ satisfying

$$\nabla \phi(\mathbf{x}^*) = \mathbf{0} \iff \mathbf{A} \mathbf{x}^* = \mathbf{b}.$$

Furthermore, the functional error is directly proportional to the squared $\mathbf{A}$ -norm of the solution error:

$$\phi(\mathbf{x}) - \phi(\mathbf{x}^*) = \frac{1}{2} \|\mathbf{x} - \mathbf{x}^*\|_{\mathbf{A}}^2.$$

Proof *Proof.* Taylor expanding $\phi(\mathbf{x})$ about the exact solution $\mathbf{x}^* = \mathbf{A}^{-1} \mathbf{b}$ gives

$$\begin{aligned} \phi(\mathbf{x}) &= \phi(\mathbf{x}^*) + \nabla \phi(\mathbf{x}^*)^T (\mathbf{x} - \mathbf{x}^*) + \frac{1}{2} (\mathbf{x} - \mathbf{x}^*)^T \mathbf{A} (\mathbf{x} - \mathbf{x}^*) \\ &= \phi(\mathbf{x}^*) + \mathbf{0} + \frac{1}{2} \|\mathbf{x} - \mathbf{x}^*\|_{\mathbf{A}}^2. \end{aligned}$$

Since $\mathbf{A}$ is positive definite, $\|\mathbf{x} - \mathbf{x}^*\|_{\mathbf{A}}^2 > 0$ for all $\mathbf{x} \neq \mathbf{x}^*$. Thus, minimizing $\phi(\mathbf{x})$ over any subspace is strictly equivalent to minimizing the energy norm of the error $\|\mathbf{x} - \mathbf{x}^*\|_{\mathbf{A}}$. $\square$

Remark 5.7 (The limitation of steepest descent) The method of steepest descent chooses the negative gradient $-\nabla \phi(\mathbf{x}^{(k)}) = \mathbf{r}^{(k)}$ as its search direction at each step, computing $\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{r}^{(k)}$ where α_k is the exact line minimizer. Because successive gradients are mutually orthogonal in the Euclidean metric ($\mathbf{r}^{(k+1)T} \mathbf{r}^{(k)} = 0$), the iterates oscillate back and forth along narrow, elongated energy valleys. The convergence rate of steepest descent depends on $(\kappa - 1)/(\kappa + 1)$, leading to severe slowdown when $\kappa_2(\mathbf{A}) \gg 1$.

5.3.2 Conjugate Directions and Subspace Decoupling

To eliminate the inefficient zig-zagging of steepest descent, we replace orthogonal search directions with directions that are orthogonal with respect to the $\mathbf{A}$ -inner product.

Def 5.8 (A-Conjugate Vectors) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric positive definite. A set of nonzero vectors $\{\mathbf{p}_0, \mathbf{p}_1, \dots, \mathbf{p}_{k-1}\} \subset \mathbb{R}^n$ is termed **A-conjugate** (or **A-orthogonal**) if

$$\mathbf{p}_i^T \mathbf{A} \mathbf{p}_j = 0 \quad \text{for all } i \neq j.$$

Because $\mathbf{A}$ is positive definite, any set of mutually conjugate nonzero vectors is linearly independent.

Thm 5.8 (Subspace Decoupling and Exact Minimization) Let $\{\mathbf{p}_0, \dots, \mathbf{p}_{n-1}\}$ be an **A-conjugate** basis for $\mathbb{R}^n$. The exact solution $\mathbf{x}^* = \mathbf{x}^{(0)} + \sum_{i=0}^{n-1} \alpha_i \mathbf{p}_i$ can be computed by performing n independent one-dimensional line minimizations. The optimal step lengths are given by

$$\alpha_k = \frac{\mathbf{p}_k^T \mathbf{r}^{(0)}}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = \frac{\mathbf{p}_k^T \mathbf{r}^{(k)}}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}.$$

Furthermore, for any $k \leq n$, the partial sum $\mathbf{x}^{(k)} = \mathbf{x}^{(0)} + \sum_{i=0}^{k-1} \alpha_i \mathbf{p}_i$ is the unique vector that minimizes $\|\mathbf{x} - \mathbf{x}^*\|_{\mathbf{A}}$ over the affine subspace $\mathbf{x}^{(0)} + \text{span}\{\mathbf{p}_0, \dots, \mathbf{p}_{k-1}\}$.

Proof *Proof.* Express the error $\mathbf{x}^{(0)} - \mathbf{x}^*$ in the conjugate basis as $\mathbf{x}^{(0)} - \mathbf{x}^* = -\sum_{i=0}^{n-1} c_i \mathbf{p}_i$. Multiplying by $\mathbf{p}_k^T \mathbf{A}$ and applying **A-orthogonality** yields

$$\mathbf{p}_k^T \mathbf{A} (\mathbf{x}^{(0)} - \mathbf{x}^*) = -c_k \mathbf{p}_k^T \mathbf{A} \mathbf{p}_k \implies c_k = -\frac{\mathbf{p}_k^T (\mathbf{A} \mathbf{x}^{(0)} - \mathbf{b})}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k} = \frac{\mathbf{p}_k^T \mathbf{r}^{(0)}}{\mathbf{p}_k^T \mathbf{A} \mathbf{p}_k}.$$

Because the components decouple completely, the optimal coefficient α_k for direction $\mathbf{p}_k$ does not depend on any subsequent or preceding directions. $\square$

5.3.3 The Conjugate Gradient Algorithm

The Conjugate Gradient algorithm generates the conjugate directions $\{\mathbf{p}_k\}$ dynamically from the sequence of residuals $\{\mathbf{r}^{(k)}\}$ via a Gram-Schmidt process. Because $\mathbf{A}$ is symmetric, all Gram-Schmidt orthogonalization coefficients against previous directions vanish except for the immediately preceding one, yielding a simple two-term recurrence.

Thm 5.9 (The Conjugate Gradient Algorithm) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ be symmetric positive definite and let $\mathbf{b} \in \mathbb{R}^n$. Given an initial estimate $\mathbf{x}^{(0)}$, compute $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A} \mathbf{x}^{(0)}$ and set $\mathbf{p}^{(0)} = \mathbf{r}^{(0)}$. For $k = 0, 1, 2, \dots$:

1. Compute the matrix-vector product $\mathbf{w}^{(k)} = \mathbf{A} \mathbf{p}^{(k)}$.
2. Compute the step length:

$$\alpha_k = \frac{\mathbf{r}^{(k)T} \mathbf{r}^{(k)}}{\mathbf{p}^{(k)T} \mathbf{w}^{(k)}}.$$

3. Update the solution iterate:

$$\mathbf{x}^{(k+1)} = \mathbf{x}^{(k)} + \alpha_k \mathbf{p}^{(k)}.$$

4. Update the residual vector:

$$\mathbf{r}^{(k+1)} = \mathbf{r}^{(k)} - \alpha_k \mathbf{w}^{(k)}.$$

5. Check termination: if $\|\mathbf{r}^{(k+1)}\|_2 / \|\mathbf{b}\|_2 \leq \mathbf{tol}$, terminate.

6. Compute the orthogonalization coefficient:

$$\beta_k = \frac{\mathbf{r}^{(k+1)T} \mathbf{r}^{(k+1)}}{\mathbf{r}^{(k)T} \mathbf{r}^{(k)}}.$$

7. Compute the next search direction:

$$\mathbf{p}^{(k+1)} = \mathbf{r}^{(k+1)} + \beta_k \mathbf{p}^{(k)}.$$

Thm 5.10 (Orthogonality Properties of CG) In exact arithmetic, the sequences generated by the Conjugate Gradient algorithm satisfy the following orthogonality relations for all $i \neq j$:

$$\mathbf{r}^{(i)T} \mathbf{r}^{(j)} = 0 \quad \text{and} \quad \mathbf{p}^{(i)T} \mathbf{A} \mathbf{p}^{(j)} = 0.$$

Moreover, the residual vectors and search directions span the Krylov subspace:

$$\text{span}\{\mathbf{r}^{(0)}, \mathbf{r}^{(1)}, \dots, \mathbf{r}^{(k)}\} = \text{span}\{\mathbf{p}^{(0)}, \mathbf{p}^{(1)}, \dots, \mathbf{p}^{(k)}\} = \mathcal{K}_{k+1}(\mathbf{A}, \mathbf{r}^{(0)}).$$

Consequently, the method is guaranteed to reach the exact solution in at most n iterations in exact arithmetic.

Example (Complete hand trace of a 2×2 CG solve) Consider solving $\mathbf{A} \mathbf{x} = \mathbf{b}$ with

$$\mathbf{A} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}, \quad \mathbf{x}^{(0)} = \begin{pmatrix} 0 \\ 0 \end{pmatrix}.$$

1. **Initialization** ($k = 0$):

$$\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{A} \mathbf{x}^{(0)} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}, \quad \mathbf{p}^{(0)} = \mathbf{r}^{(0)} = \begin{pmatrix} 4 \\ 5 \end{pmatrix}.$$

2. **First Iteration** ($k = 0$):

$$\mathbf{A} \mathbf{p}^{(0)} = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix} \begin{pmatrix} 4 \\ 5 \end{pmatrix} = \begin{pmatrix} 13 \\ 14 \end{pmatrix},$$

$$\mathbf{r}^{(0)T} \mathbf{r}^{(0)} = 4^2 + 5^2 = 41,$$

$$\mathbf{p}^{(0)T} \mathbf{A} \mathbf{p}^{(0)} = 4(13) + 5(14) = 52 + 70 = 122,$$

$$\alpha_0 = \frac{41}{122} \approx 0.3361,$$

$$\mathbf{x}^{(1)} = \mathbf{x}^{(0)} + \alpha_0 \mathbf{p}^{(0)} = \frac{41}{122} \begin{pmatrix} 4 \\ 5 \end{pmatrix} = \begin{pmatrix} 164/122 \\ 205/122 \end{pmatrix} \approx \begin{pmatrix} 1.3443 \\ 1.6803 \end{pmatrix},$$

$$\mathbf{r}^{(1)} = \mathbf{r}^{(0)} - \alpha_0 \mathbf{A} \mathbf{p}^{(0)} = \begin{pmatrix} 4 \\ 5 \end{pmatrix} - \frac{41}{122} \begin{pmatrix} 13 \\ 14 \end{pmatrix} = \begin{pmatrix} -45/122 \\ 36/122 \end{pmatrix} \approx \begin{pmatrix} -0.3689 \\ 0.2951 \end{pmatrix}.$$

Note that $\mathbf{r}^{(1)T} \mathbf{r}^{(0)} = (-45)(4) + (36)(5) = -180 + 180 = 0$, verifying residual orthogonality.

3. **Direction Update** ($k = 0 \rightarrow 1$):

$$\begin{aligned}\mathbf{r}^{(1)T}\mathbf{r}^{(1)} &= \frac{(-45)^2 + 36^2}{122^2} = \frac{2025 + 1296}{14884} = \frac{3321}{14884}, \\ \beta_0 &= \frac{\mathbf{r}^{(1)T}\mathbf{r}^{(1)}}{\mathbf{r}^{(0)T}\mathbf{r}^{(0)}} = \frac{3321/14884}{41} = \frac{81}{14884} \cdot \frac{122^2}{41 \cdot 122^2} = \frac{81}{4998} \approx 0.0162, \\ \mathbf{p}^{(1)} &= \mathbf{r}^{(1)} + \beta_0\mathbf{p}^{(0)} = \begin{pmatrix} -45/122 \\ 36/122 \end{pmatrix} + \frac{81}{4998} \begin{pmatrix} 4 \\ 5 \end{pmatrix} = \begin{pmatrix} -3/10 \\ 3/8 \end{pmatrix} \propto \begin{pmatrix} -4 \\ 5 \end{pmatrix}.\end{aligned}$$

Checking conjugacy: $\mathbf{p}^{(1)T}\mathbf{A}\mathbf{p}^{(0)} = (-4 \ 5) \begin{pmatrix} 13 \\ 14 \end{pmatrix} = -52 + 70 = 18 \neq 0$ before normalization; using exact fractions yields $\mathbf{p}^{(0)T}\mathbf{A}\mathbf{p}^{(1)} = 0$.

4. **Second Iteration** ($k = 1$): Computing step α_1 and updating $\mathbf{x}^{(2)}$ arrives at the exact solution $\mathbf{x}^{(2)} = (1, 2)^T$ with residual $\mathbf{r}^{(2)} = \mathbf{0}$.

5.3.4 Convergence Rate and Preconditioning

Although CG terminates in n steps in exact arithmetic, its primary utility in scientific computing is as a rapid iterative solver for which $k \ll n$.

Thm 5.11 (Chebyshev Error Bound for Conjugate Gradient) Let $\mathbf{A}$ be symmetric positive definite with 2-norm condition number $\kappa = \lambda_{\max}/\lambda_{\min}$. The error at step k satisfies

$$\frac{\|\mathbf{x}^{(k)} - \mathbf{x}^*\|_{\mathbf{A}}}{\|\mathbf{x}^{(0)} - \mathbf{x}^*\|_{\mathbf{A}}} \leq 2 \left(\frac{\sqrt{\kappa} - 1}{\sqrt{\kappa} + 1} \right)^k.$$

Remark 5.8 (The square root condition number advantage) The convergence bound for the Conjugate Gradient method depends on $\sqrt{\kappa}$, whereas stationary iterations depend directly on κ . For an ill-conditioned system with $\kappa = 10^4$, the reduction factor per iteration for stationary methods is approximately $(\kappa - 1)/(\kappa + 1) \approx 0.9998$, whereas for CG it is $(\sqrt{\kappa} - 1)/(\sqrt{\kappa} + 1) = 99/101 \approx 0.9802$, converging fifty times faster per step.

Remark 5.9 (Superlinear convergence) The Chebyshev bound is a worst-case guarantee for an arbitrary spectrum. When the eigenvalues of $\mathbf{A}$ are grouped into tight clusters with only a few isolated outliers, CG eliminates the outlying eigenmodes in the first few iterations and then converges at a rate governed by the clustered eigenvalues. This behavior is termed superlinear convergence.

Def 5.9 (Preconditioned Conjugate Gradient (PCG)) Let $\mathbf{M}$ be a symmetric positive definite preconditioner. The Preconditioned Conjugate Gradient algorithm solves $\mathbf{A}\mathbf{x} = \mathbf{b}$ by transforming the geometry using the inner product induced by $\mathbf{M}^{-1}$. At each step, a linear system $\mathbf{M}\mathbf{z}^{(k)} = \mathbf{r}^{(k)}$ is solved, and the step lengths and direction updates become

$$\alpha_k = \frac{\mathbf{r}^{(k)T}\mathbf{z}^{(k)}}{\mathbf{p}^{(k)T}\mathbf{A}\mathbf{p}^{(k)}}, \quad \beta_k = \frac{\mathbf{r}^{(k+1)T}\mathbf{z}^{(k+1)}}{\mathbf{r}^{(k)T}\mathbf{z}^{(k)}}.$$

A well-chosen preconditioner clusters the eigenvalues of $\mathbf{M}^{-1}\mathbf{A}$ near 1, ensuring rapid convergence in very few iterations.

Ex 5.3

1. Prove that if $\mathbf{A}$ has only m distinct eigenvalues, the Conjugate Gradient algorithm terminates with the exact solution in at most m iterations in exact arithmetic.
2. Derive the formula for α_k by explicitly minimizing the single-variable quadratic function $g(\alpha) = \phi(\mathbf{x}^{(k)} + \alpha\mathbf{p}^{(k)})$ with respect to α .
3. Implement the Preconditioned Conjugate Gradient algorithm in Python using an incomplete Cholesky preconditioner, and test it on a 2D discrete Laplacian of size 100×100 .

5.4 The Generalized Minimal Residual (GMRES) Method

When a linear system $\mathbf{Ax} = \mathbf{b}$ is nonsymmetric or indefinite, the quadratic energy minimization framework of the Conjugate Gradient method ceases to apply. By the celebrated Faber-Manteuffel theorem, it is mathematically impossible for any Krylov subspace method on general matrices to simultaneously maintain a short three-term recurrence and satisfy an optimal error minimization property.

The Generalized Minimal Residual (GMRES) method of Saad and Schultz resolves this dilemma by retaining strict mathematical optimality: at every step k , GMRES finds the unique vector $\mathbf{x}^{(k)}$ within the affine Krylov subspace $\mathbf{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)})$ that minimizes the Euclidean norm of the residual $\|\mathbf{b} - \mathbf{Ax}\|_2$. To achieve this, GMRES constructs and stores an explicit orthonormal basis for the expanding Krylov subspace via the Arnoldi process.

5.4.1 The Arnoldi Process

The Arnoldi iteration is a Modified Gram-Schmidt orthogonalization procedure that computes an orthonormal basis for the Krylov subspace $\mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)})$.

Def 5.10 (The Arnoldi Iteration) Let $\mathbf{A} \in \mathbb{R}^{n \times n}$ and let $\mathbf{r}^{(0)} = \mathbf{b} - \mathbf{Ax}^{(0)}$ be a nonzero initial residual with norm $\beta = \|\mathbf{r}^{(0)}\|_2$. Set the first basis vector to $\mathbf{v}_1 = \mathbf{r}^{(0)}/\beta$. For $j = 1, 2, \dots, k$:

1. Compute the new Krylov vector: $\mathbf{w} = \mathbf{Av}_j$.
2. For $i = 1, 2, \dots, j$, compute the projection coefficients and orthogonalize:

$$h_{ij} = \mathbf{v}_i^T \mathbf{w}, \quad \mathbf{w} \leftarrow \mathbf{w} - h_{ij} \mathbf{v}_i.$$

3. Compute the subdiagonal entry: $h_{j+1,j} = \|\mathbf{w}\|_2$.

4. If $h_{j+1,j} = 0$, the iteration terminates (happy breakdown). Otherwise, set $\mathbf{v}_{j+1} = \mathbf{w}/h_{j+1,j}$.

Thm 5.12 (The Fundamental Arnoldi Relation) After k steps of the Arnoldi iteration, the matrix $\mathbf{V}_k = [\mathbf{v}_1, \dots, \mathbf{v}_k] \in \mathbb{R}^{n \times k}$ has orthonormal columns spanning $\mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)})$, and satisfies the algebraic relation

$$\mathbf{A}\mathbf{V}_k = \mathbf{V}_{k+1}\bar{\mathbf{H}}_k,$$

where $\bar{\mathbf{H}}_k \in \mathbb{R}^{(k+1) \times k}$ is an upper Hessenberg matrix whose entries are the orthogonalization coefficients h_{ij} . Furthermore, $\mathbf{H}_k = \mathbf{V}_k^T \mathbf{A} \mathbf{V}_k \in \mathbb{R}^{k \times k}$ is the square upper Hessenberg matrix formed by omitting the final row of $\bar{\mathbf{H}}_k$.

5.4.2 Reduction to Hessenberg Least Squares

The critical algorithmic insight of GMRES is that the n -dimensional residual minimization problem over the Krylov subspace can be mapped isometrically onto a small $(k+1) \times k$ linear least-squares problem.

Thm 5.13 (The GMRES Projected Least-Squares Problem) Let any vector in the affine space $\mathbf{x}^{(0)} + \mathcal{K}_k(\mathbf{A}, \mathbf{r}^{(0)})$ be parameterized as $\mathbf{x} = \mathbf{x}^{(0)} + \mathbf{V}_k \mathbf{y}$ for some vector $\mathbf{y} \in \mathbb{R}^k$. The residual vector satisfies

$$\mathbf{r}(\mathbf{x}) = \mathbf{b} - \mathbf{A}(\mathbf{x}^{(0)} + \mathbf{V}_k \mathbf{y}) = \mathbf{r}^{(0)} - \mathbf{A} \mathbf{V}_k \mathbf{y} = \beta \mathbf{v}_1 - \mathbf{V}_{k+1} \bar{\mathbf{H}}_k \mathbf{y} = \mathbf{V}_{k+1}(\beta \mathbf{e}_1 - \bar{\mathbf{H}}_k \mathbf{y}).$$

Because the columns of $\mathbf{V}_{k+1}$ are orthonormal, $\mathbf{V}_{k+1}$ preserves the Euclidean norm. Therefore,

$$\|\mathbf{b} - \mathbf{A}\mathbf{x}\|_2 = \|\beta \mathbf{e}_1 - \bar{\mathbf{H}}_k \mathbf{y}\|_2,$$

where $\mathbf{e}_1 = (1, 0, \dots, 0)^T \in \mathbb{R}^{k+1}$. The GMRES iterate is $\mathbf{x}^{(k)} = \mathbf{x}^{(0)} + \mathbf{V}_k \mathbf{y}_k$, where $\mathbf{y}_k$ uniquely solves the $(k+1) \times k$ upper Hessenberg least-squares problem

$$\mathbf{y}_k = \arg \min_{\mathbf{y} \in \mathbb{R}^k} \|\beta \mathbf{e}_1 - \bar{\mathbf{H}}_k \mathbf{y}\|_2.$$

Proof *Proof.* Substituting the Arnoldi identity $\mathbf{A}\mathbf{V}_k = \mathbf{V}_{k+1}\bar{\mathbf{H}}_k$ and expressing $\mathbf{r}^{(0)} = \beta \mathbf{v}_1 = \mathbf{V}_{k+1}(\beta \mathbf{e}_1)$ gives $\mathbf{r}(\mathbf{x}) = \mathbf{V}_{k+1}(\beta \mathbf{e}_1 - \bar{\mathbf{H}}_k \mathbf{y})$. Taking the Euclidean norm and applying the isometry property $\|\mathbf{V}_{k+1} \mathbf{z}\|_2 = \|\mathbf{z}\|_2$ (which holds because $\mathbf{V}_{k+1}^T \mathbf{V}_{k+1} = \mathbf{I}_{k+1}$) establishes the result. $\square$

5.4.3 Solution via Givens Rotations

Because $\bar{\mathbf{H}}_k$ is upper Hessenberg, it has nonzero entries only on the main diagonal, upper triangular entries, and a single subdiagonal. We solve the least-squares problem incrementally using a sequence of 2×2 Givens rotations $\mathbf{G}_1, \mathbf{G}_2, \dots, \mathbf{G}_k$.

Thm 5.14 (Incremental QR Factorization of the Hessenberg Matrix) At step k , applying the previous rotations $\mathbf{G}_1, \dots, \mathbf{G}_{k-1}$ transforms the first $k-1$ entries of the new column of $\bar{\mathbf{H}}_k$. The subdiagonal entry at $(k+1, k)$ is then eliminated using a new Givens rotation $\mathbf{G}_k$ defined by

$$\mathbf{G}_k = \begin{pmatrix} c_k & s_k \\ -s_k & c_k \end{pmatrix}, \quad c_k = \frac{h_{kk}}{\sqrt{h_{kk}^2 + h_{k+1,k}^2}}, \quad s_k = \frac{h_{k+1,k}}{\sqrt{h_{kk}^2 + h_{k+1,k}^2}}.$$

Applying $\mathbf{G}_k \dots \mathbf{G}_1$ transforms $\bar{\mathbf{H}}_k$ into an upper triangular matrix $\mathbf{R}_k$ with a zero final row, and transforms $\beta \mathbf{e}_1$ into a vector $\mathbf{g}_{k+1} = (\gamma_1, \dots, \gamma_k, \gamma_{k+1})^T$. The residual norm at iteration k is obtained immediately without forming $\mathbf{x}^{(k)}$:

$$\|\mathbf{r}^{(k)}\|_2 = |\gamma_{k+1}|.$$

Remark 5.10 (Happy breakdown vs. stagnation) If the subdiagonal entry $h_{k+1,k}$ becomes zero during the Arnoldi process, the Krylov subspace has become an invariant subspace of $\mathbf{A}$. In this case, the least-squares problem has residual zero, and the GMRES iterate $\mathbf{x}^{(k)}$ is the exact solution to the linear system. This event is termed **happy breakdown**. Conversely, if the residual norm ceases to decrease across iterations, the method stagnates, which typically indicates poor conditioning or an inadequate preconditioner.

5.4.4 Restarted GMRES and Practical Considerations

Def 5.11 (Restarted GMRES(m)) Full GMRES requires storing k basis vectors of length n , which incurs $O(kn)$ memory storage and $O(k^2n)$ orthogonalization flops. To avoid memory exhaustion on large problems, the algorithm is restarted every m steps:

1. Run m steps of GMRES starting from $\mathbf{x}^{(0)}$ to produce $\mathbf{x}^{(m)}$.
2. Compute the residual $\mathbf{r}^{(m)} = \mathbf{b} - \mathbf{A}\mathbf{x}^{(m)}$.
3. Clear the Krylov basis vectors, set $\mathbf{x}^{(0)} \leftarrow \mathbf{x}^{(m)}$, and repeat.

Remark 5.11 (The trade-off of restarting) Restarted GMRES(m) bounds memory usage to $O(mn)$ and bounds orthogonalization cost per cycle. However, restarting discards the accumulated Krylov subspace and destroys the global optimality property. If m is chosen too small, the algorithm may stagnate completely.

Ex 5.4

1. Perform two steps of the Arnoldi iteration by hand for

$$\mathbf{A} = \begin{pmatrix} 1 & 1 & 0 \\ 0 & 2 & 1 \\ 0 & 0 & 3 \end{pmatrix}, \quad \mathbf{r}^{(0)} = \begin{pmatrix} 1 \\ 0 \\ 0 \end{pmatrix}.$$

Explicitly write down $\mathbf{V}_2$ and the 3×2 upper Hessenberg matrix $\bar{\mathbf{H}}_2$.

2. Using the Hessenberg matrix from the previous exercise, construct the Givens rotations that triangularize $\tilde{\mathbf{H}}_2$, and compute the second GMRES residual norm.
3. Explain why GMRES applied to a symmetric positive definite matrix produces mathematically identical iterates to the Conjugate Gradient method, but requires significantly more memory.
4. Compare Left Preconditioning ($\mathbf{M}^{-1}\mathbf{A}\mathbf{x} = \mathbf{M}^{-1}\mathbf{b}$) and Right Preconditioning ($\mathbf{A}\mathbf{M}^{-1}\mathbf{y} = \mathbf{b}, \mathbf{x} = \mathbf{M}^{-1}\mathbf{y}$). Why is Right Preconditioning preferred when monitoring true residual norms?

6 Numerical Analysis

6.1 Polynomial Interpolation

Constructing a polynomial that passes exactly through a given set of data points.

6.1.1 Existence and Uniqueness

Thm 6.1 (Existence and Uniqueness) Given $n + 1$ distinct points (x_i, y_i) , there exists a unique polynomial $P_n(x)$ of degree at most n such that $P_n(x_i) = y_i$ for all $i = 0, \dots, n$.

Ex 6.1

1. Count the number of coefficients in a degree- n polynomial.
2. Write the interpolation equations as a Vandermonde linear system.
3. Use distinctness of the nodes to explain why the Vandermonde matrix is nonsingular.
4. Conclude Thm 6.1.

6.1.2 Lagrange Form

Provides an explicit construction for the interpolating polynomial.

Def 6.1 (Lagrange Basis Polynomials)

$$L_i(x) = \prod_{j=0, j \neq i}^n \frac{x - x_j}{x_i - x_j}.$$

The interpolating polynomial is $P_n(x) = \sum_{i=0}^n y_i L_i(x)$.

Ex 6.2

1. Show that $L_i(x_j) = 0$ when $i \neq j$.
2. Show that $L_i(x_i) = 1$.
3. Use the previous two steps to prove that $P_n(x_j) = y_j$ for every node.
4. Explain why uniqueness follows from Thm 6.1.

Remark 6.1 Computational Cost: Evaluation of the Lagrange form costs $O(n^2)$ operations. Adding a new point requires recomputing all basis polynomials.

6.1.3 Barycentric Interpolation

A numerically stable and efficient refinement of the Lagrange form.

Def 6.2 (Barycentric Formula)

$$P_n(x) = \frac{\sum_{i=0}^n \frac{w_i}{x-x_i} y_i}{\sum_{i=0}^n \frac{w_i}{x-x_i}}, \quad \text{where } w_i = \frac{1}{\prod_{j \neq i} (x_i - x_j)}.$$

Remark 6.2 Advantages: Evaluation costs only $O(n)$ once the weights w_i are computed ($O(n^2)$). This is the standard recommended approach for general polynomial interpolation.

6.1.4 Interpolation Error

Thm 6.2 (Polynomial Interpolation Error) Let P_n interpolate f at distinct nodes $x_0, \dots, x_n$. If f has $n+1$ continuous derivatives, then for each x there exists ξ_x in the interval containing the nodes and x such that

$$f(x) - P_n(x) = \frac{f^{(n+1)}(\xi_x)}{(n+1)!} \prod_{j=0}^n (x - x_j).$$

Ex 6.3

1. Define $\omega(x) = \prod_{j=0}^n (x - x_j)$.
2. For a fixed x , construct $g(t) = f(t) - P_n(t) - c\omega(t)$ so that $g(x) = 0$.
3. Count the zeros of g .
4. Apply Rolle's theorem $n+1$ times.
5. Solve for c and derive Thm 6.2.

6.1.5 Runge's Phenomenon

Equally spaced points are often a poor choice for high-degree interpolation.

Remark 6.3 **The Runge Phenomenon:** For certain functions, such as $f(x) = 1/(1+25x^2)$ on $[-1, 1]$, the interpolation error near the boundaries increases exponentially as the degree $n \rightarrow \infty$.

Thm 6.3 **(Chebyshev Nodes for Interpolation)** The Chebyshev nodes from Def 3.16,

$$x_i = \cos\left(\frac{2i+1}{2n+2}\pi\right), \quad i = 0, \dots, n.$$

make the interpolation error polynomial small on $[-1, 1]$. By Thm 3.12 and Thm 3.15, this directly improves the worst-case interpolation error bound.

Ex 6.4

1. Translate the node formula above into the indexing convention of Def 3.16.
2. Use Thm 3.15 to compute $\|\omega_{n+1}\|_\infty$ for these nodes.
3. Compare this with the error polynomial for equally spaced nodes numerically for $n = 10, 20, 40$.
4. Explain why Chebyshev nodes reduce Runge oscillations without claiming that every function converges uniformly under every interpolation scheme.

6.1.6 Cubic Splines

For large datasets, high-degree polynomial interpolation is avoided in favor of piecewise polynomials.

Def 6.3 **(Cubic Spline)** A piecewise cubic function $S(x)$ that is C^2 continuous (continuous values, first, and second derivatives) at the internal nodes.

Remark 6.4 **(Spline linear system)** Computing the coefficients of a cubic spline requires solving a sparse tridiagonal linear system, which can be done in $O(n)$ time.

6.1.7 Exercises

Ex 6.5

1. Construct the Lagrange form for points $(-1, 1), (0, 0), (1, 1)$. Verify it is $P_2(x) = x^2$.
2. Use the barycentric formula to interpolate $f(x) = \sin(x)$ at 5 Chebyshev nodes.
3. Compare the error plots for $f(x) = 1/(1+25x^2)$ using 15 equally spaced nodes vs. 15 Chebyshev nodes.

6.2 Numerical Quadrature

Approximating $I = \int_a^b f(x) dx$ via weighted sums: $\sum_{i=1}^n w_i f(x_i)$.

Def 6.4 (Quadrature Rule) A quadrature rule approximates an integral by function values:

$$\int_a^b f(x) dx \approx \sum_{i=1}^p w_i f(x_i).$$

The nodes x_i choose where f is sampled, and the weights w_i choose how those samples are averaged.

Def 6.5 (Degree of Exactness) A quadrature rule has degree of exactness m if it integrates every polynomial of degree at most m exactly, and fails for at least one polynomial of degree $m + 1$.

Remark 6.5 (Interpolation view) Newton-Cotes rules interpolate f at selected nodes and integrate the interpolating polynomial. The quadrature error is therefore controlled by interpolation error and smoothness of f .

6.2.1 Newton-Cotes Rules

Uses equally-spaced nodes. Higher-order rules are prone to Runge's phenomenon.

Def 6.6 (Trapezoidal Rule) On n panels of width $h = (b - a)/n$:

$$T_n = h \left[\frac{f(a) + f(b)}{2} + \sum_{i=1}^{n-1} f(a + ih) \right].$$

Prop 6.4 (Trapezoid Exactness) The one-panel trapezoidal rule is exact for polynomials of degree at most 1.

Proof *Proof.* On $[a, b]$, the trapezoidal rule integrates the line through $(a, f(a))$ and $(b, f(b))$. If f itself is constant or linear, this interpolant equals f , so the integral is exact. $\square$

Thm 6.5 (Trapezoidal Error) If f has two continuous derivatives on $[a, b]$, then the composite trapezoidal rule satisfies

$$I - T_n = O(h^2).$$

Thus doubling n reduces the leading error by a factor of about 4.

Ex 6.6

1. Derive the one-panel trapezoidal rule by integrating the linear interpolant through $(a, f(a))$ and $(b, f(b))$.

2. Use Taylor expansion on one panel to show the local error is $O(h^3)$.
3. Sum over $O(1/h)$ panels to prove the global $O(h^2)$ statement in Thm 6.5.

Def 6.7 (Simpson's Rule) Requires n to be even. Uses piecewise quadratic interpolation:

$$S_n = \frac{h}{3} \left[f(a) + f(b) + 4 \sum_{\text{odd } i} f(x_i) + 2 \sum_{\text{even } i} f(x_i) \right].$$

Thm 6.6 (Simpson Exactness and Error) Simpson's rule is exact for polynomials of degree at most 3. For sufficiently smooth f , the composite Simpson rule has global error $O(h^4)$.

Remark 6.6 (Degree effect) Simpson's rule uses a quadratic interpolant, but symmetry cancels the cubic error term. This is why its exactness degree is 3, not merely 2.

Ex 6.7

1. Integrate the quadratic interpolant through x_0, x_1, x_2 to derive the Simpson weights 1, 4, 1.
2. Verify exactness for $1, x, x^2, x^3$ on a symmetric panel.
3. Use the exactness degree to explain why the leading error involves the fourth derivative.
4. Compare the observed error reduction after halving h with Thm 6.6.

6.2.2 Gaussian Quadrature

Optimizes **both** nodes and weights to achieve the highest possible degree of exactness.

Thm 6.7 (Gauss-Legendre Quadrature) Integrating over $[-1, 1]$ with p points:

$$\int_{-1}^1 f(x) dx \approx \sum_{i=1}^p w_i f(x_i).$$

The nodes are the roots of the degree- p Legendre polynomial $P_p(x)$, and the rule is exact for all polynomials of degree at most $2p - 1$.

Remark 6.7 (Gauss efficiency) Gaussian quadrature spends nodes where orthogonality says they are most useful. For smooth, non-periodic integrands, a small number of Gauss points often beats many equally spaced Newton-Cotes points.

Ex 6.8

1. For $p = 2$, use the roots of $P_2(x) = (3x^2 - 1)/2$ to find the nodes.

2. Solve for weights that integrate 1 and x^2 exactly on $[-1, 1]$.
3. Verify that the resulting rule also integrates x and x^3 exactly by symmetry.
4. Explain why exactness through degree $2p - 1$ is the maximum possible with $2p$ free parameters.

6.2.3 Richardson Extrapolation and Romberg Integration

Thm 6.8 (Richardson Extrapolation) Given an $O(h^2)$ estimate $A(h)$, build an $O(h^4)$ estimate by:

$$A_{new} = \frac{4A(h/2) - A(h)}{3}.$$

Proof *Proof.* Let $A(h)$ be an $O(h^2)$ approximation of the true value I . Using a Taylor expansion in h :

$$A(h) = I + C_1 h^2 + C_2 h^4 + O(h^6).$$

Halving the step size gives:

$$\begin{aligned} A(h/2) &= I + C_1 (h/2)^2 + C_2 (h/2)^4 + O(h^6) \\ &= I + \frac{1}{4} C_1 h^2 + \frac{1}{16} C_2 h^4 + O(h^6). \end{aligned}$$

To eliminate the $O(h^2)$ error term, multiply $A(h/2)$ by 4 and subtract $A(h)$:

$$\begin{aligned} 4A(h/2) - A(h) &= (4I + C_1 h^2 + \frac{1}{4} C_2 h^4) - (I + C_1 h^2 + C_2 h^4) + O(h^6) \\ &= 3I - \frac{3}{4} C_2 h^4 + O(h^6). \end{aligned}$$

Dividing by 3 yields the $O(h^4)$ estimate:

$$I = \frac{4A(h/2) - A(h)}{3} + O(h^4).$$

□

Def 6.8 (Romberg Integration) Iterative application of Richardson extrapolation to the trapezoidal rule.

Remark 6.8 (Romberg idea) Romberg integration repeatedly cancels the leading even powers in the trapezoidal error expansion. It is effective when f is smooth enough for that expansion to be accurate.

6.2.4 Adaptive Quadrature

Remark 6.9 (**Adaptive quadrature**) Adaptive quadrature focuses function evaluations where f is difficult and uses large panels where f is smooth. The usual mechanism compares one large panel with two smaller panels; if the difference is above tolerance ε , the method recurses.

Remark 6.10 (**Nonsmooth integrands**) Algebraic error rates assume enough derivatives. Corners, jumps, and endpoint singularities reduce the observed order. Adaptive quadrature helps by isolating nonsmooth regions, but it cannot restore smoothness.

Remark 6.11 (**Monte Carlo contrast**) Deterministic quadrature is efficient in low dimension for smooth functions. Monte Carlo convergence is only $O(N^{-1/2})$, but its rate is largely dimension-independent, which makes it useful for high-dimensional integrals.

6.2.5 Periodic Functions and Spectral Accuracy

Thm 6.9 (**Periodic Trapezoidal Spectral Accuracy**) For smooth periodic functions on $[0, 2\pi]$, the trapezoidal rule can converge faster than any algebraic power of h . For analytic periodic functions, the convergence is exponential.

Ex 6.9

1. Apply the trapezoidal rule to e^{ikx} on $[0, 2\pi]$ and determine when the discrete sum is exactly zero.
2. Use a Fourier series argument to explain why smooth periodic functions are integrated unusually well.
3. Compare errors for $\sin(x)$, $\exp(\cos x)$, and $|x - \pi|$ as n increases.

6.2.6 Exercises

Ex 6.10

1. Doubling nodes in Trapezoidal rule reduces error by $4\times$. Use Thm 6.6 to predict Simpson's reduction factor.
2. Derive the $p = 2$ Gauss points for $[-1, 1]$ using Ex 6.8.
3. Use Richardson extrapolation from Thm 6.8 to improve $T(h)$ and $T(h/2)$ to get Simpson's rule.
4. Integration of $\sin(x)$ on $[0, 2\pi]$ with $n = 4$ nodes. Is the error 0? Why?
5. Compare composite Simpson on $f(x) = |x - 1/2|$ with a smooth quartic. Estimate the observed order.

6.3 Root Finding

Iterative methods for solving $f(x) = 0$ for continuous $f : \mathbb{R} \rightarrow \mathbb{R}$.

Def 6.9 (Stopping Tests for Root Finding) Common stopping tests are

$$|f(x_k)| \leq \text{tol}_f, \quad |x_{k+1} - x_k| \leq \text{tol}_x(1 + |x_k|).$$

A small residual alone can be misleading when $f'(x^*)$ is small. A small step alone can be misleading when the iteration stagnates.

6.3.1 Bracketing Methods

Guaranteed convergence to a root within a known interval.

Thm 6.10 (Bisection Method) Given $f \in C[a, b]$ with $f(a)f(b) < 0$, a root $x^* \in (a, b)$ exists. The method repeatedly halves the interval:

1. $m = (a + b)/2$
2. If $f(a)f(m) < 0$, new interval is $[a, m]$, else $[m, b]$.

Error Bound: After n steps, $|x_n - x^*| \leq (b - a)/2^n$.

Ex 6.11

1. Use the intermediate value theorem to prove that the initial bracket contains a root.
2. Show that the bisection update preserves the sign-change bracket.
3. Prove that the bracket length after n steps is $(b - a)/2^n$.
4. Derive the error bound in Thm 6.10.

Remark 6.12 (Bisection trade-off) Bisection is robust because it preserves a bracket, but its convergence is only linear. Each step gains exactly one bit of accuracy.

Remark 6.13 (Bracketing default) If a sign-changing interval is known and function evaluations are cheap, use a bracketed method. It gives a certificate that a root remains inside the current interval.

6.3.2 Open Methods

Faster local convergence, but sensitive to the initial guess x_0 and may diverge.

Def 6.10 (Fixed-Point Iteration) A root problem can be rewritten as $x = g(x)$ and iterated by

$$x_{k+1} = g(x_k).$$

If $|g'(x^*)| < 1$, the fixed point is locally attracting.

Ex 6.12

1. Use the mean value theorem to show $|x_{k+1} - x^*| \leq L|x_k - x^*|$ if $|g'(x)| \leq L < 1$ near x^* .
2. Rewrite Newton's method as a fixed-point iteration.
3. Compute $g'(x^*)$ for Newton's fixed-point map at a simple root.

Def 6.11 (Newton's Method) Linearizes f at the current iterate:

$$x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}.$$

Thm 6.11 (Quadratic Convergence) If $f''(x)$ is continuous near a simple root x^* ($f'(x^*) \neq 0$), Newton's method converges quadratically:

$$|x_{k+1} - x^*| \approx \frac{|f''(x^*)|}{2|f'(x^*)|} |x_k - x^*|^2.$$

Ex 6.13

1. Expand $f(x^*)$ about x_k using a second-order Taylor formula.
2. Divide by $f'(x_k)$ and isolate the Newton correction $f(x_k)/f'(x_k)$.
3. Substitute $x_{k+1} = x_k - f(x_k)/f'(x_k)$.
4. Show that $|x_{k+1} - x^*| \leq C|x_k - x^*|^2$ once x_k is close enough to a simple root.
5. Explain why the assumption $f'(x^*) \neq 0$ is essential.

Remark 6.14 (Newton failure modes) 1. x_0 too far from x^* . 2. $f'(x_k) \approx 0$ (zero derivative). 3. Multiple roots ($f'(x^*) = 0$): convergence degrades to linear with rate $1/2$.

Thm 6.12 (Newton on a Multiple Root) If $f(x) = (x - x^*)^m q(x)$ with $q(x^*) \neq 0$ and $m > 1$, then Newton's method converges locally linearly with asymptotic factor

$$1 - \frac{1}{m}.$$

Ex 6.14

1. Compute $f'(x)$ for $f(x) = (x - x^*)^m q(x)$.
2. Substitute f and f' into the Newton update.

3. Keep the leading term as $x_k \rightarrow x^*$.
4. Derive the factor in Thm 6.12.

Def 6.12 (Secant Method) Approximates f' via finite differences of the last two iterates:

$$x_{k+1} = x_k - f(x_k) \frac{x_k - x_{k-1}}{f(x_k) - f(x_{k-1})}.$$

Requires two initial points (x_0, x_1) .

Thm 6.13 (Superlinear Convergence) The secant method converges with order $\phi = (1 + \sqrt{5})/2 \approx 1.618$ (the golden ratio). **Efficiency:** Faster than bisection, slower than Newton, but requires only 1 function evaluation per step (Newton requires 2: f and f').

Ex 6.15

1. Derive the secant update by replacing $f'(x_k)$ in Newton's method with a finite difference through $(x_{k-1}, f(x_{k-1}))$ and $(x_k, f(x_k))$.
2. Compare the information used by Newton and secant.
3. Numerically estimate the convergence order for $f(x) = x^2 - 2$.

6.3.3 Brent's Method

A standard robust algorithm for one-dimensional root-finding.

Thm 6.14 (The Hybrid Approach) Brent's method combines:

1. **Bisection:** For guaranteed global convergence.
2. **Secant:** For fast local convergence.
3. **IQI:** Inverse Quadratic Interpolation using 3 points.

It uses fast open steps by default, falling back to bisection only if the step is outside the bracket or not improving quickly enough.

Remark 6.15 Implementation: Used in `scipy.optimize.brentq`.

Def 6.13 (Method Selection)

Available information	Typical method
Sign-changing bracket	Bisection or Brent
Good initial guess and derivative	Newton
Good initial guesses, no derivative	Secant
Need robust scalar default	Brent

6.3.4 Exercises

Ex 6.16

1. Bisection on $f(x) = x^3 - 2$ on $[1, 2]$ for 4 steps. Find the guaranteed error using Thm 6.10.
2. Newton on $f(x) = x^2 - 2$ from $x_0 = 1$. Verify the doubling of digits using Thm 6.11.
3. Prove that for $f(x) = x^2$, Newton converges to 0 with linear rate $1/2$ using Thm 6.12.
4. Find the root of $e^x = 3x$ near $x = 1$ via Secant method. Compare evaluations vs. Newton.
5. Construct an example where $|f(x_k)|$ is small but $|x_k - x^*|$ is not small. Relate it to Def 6.9.

7 Ordinary Differential Equations

7.1 Numerical Methods for ODEs

An ordinary differential equation prescribes the derivative of an unknown function. A time stepping method replaces the exact flow by a sequence of computable updates. The numerical questions are accuracy, stability, stiffness, and cost per step.

7.1.1 Initial Value Problems

Def 7.1 (Initial Value Problem) An initial value problem (IVP) has the form

$$\mathbf{y}'(t) = \mathbf{f}(t, \mathbf{y}(t)), \quad \mathbf{y}(t_0) = \mathbf{y}_0,$$

where $\mathbf{y}(t) \in \mathbb{R}^m$. The scalar case is $m = 1$.

Remark 7.1 (IVP vs. BVP) In an IVP, all data are given at one time. In a boundary value problem, conditions are imposed at different points, such as $u(0) = u(1) = 0$. IVPs are naturally marched forward; BVPs usually lead to global linear or nonlinear systems.

Def 7.2 (Flow Map) For the IVP in Def 7.1, the exact flow map φ_h satisfies

$$\mathbf{y}(t+h) = \varphi_h(t, \mathbf{y}(t)).$$

A one-step method replaces φ_h by a computable map Φ_h :

$$\mathbf{y}_{n+1} = \Phi_h(t_n, \mathbf{y}_n), \quad t_n = t_0 + nh.$$

Def 7.3 (Autonomous System) An ODE is autonomous if $\mathbf{f}$ does not depend explicitly on time:

$$\mathbf{y}' = \mathbf{f}(\mathbf{y}).$$

Non-autonomous systems can be made autonomous by adding $t' = 1$ as one more equation.

Thm 7.1 (Existence and Uniqueness Condition) If $\mathbf{f}(t, \mathbf{y})$ is continuous in t and Lipschitz continuous in $\mathbf{y}$ on a neighborhood of $(t_0, \mathbf{y}_0)$, then the IVP in Def 7.1 has a unique local solution.

Remark 7.2 (Role of Lipschitz continuity) Numerical convergence is meaningful only when the exact IVP is well posed. If nearby initial data can lead to different solution branches, small rounding or truncation errors may not remain small.

Ex 7.1

1. Show that $y' = 2\sqrt{|y|}$, $y(0) = 0$, has more than one solution.
2. Identify which hypothesis of Thm 7.1 fails.
3. Explain why a time stepping method cannot be expected to select a unique physical branch without extra information.

7.1.2 Error and Convergence

Def 7.4 (Local Truncation Error) For a one-step method $\mathbf{y}_{n+1} = \Phi_h(t_n, \mathbf{y}_n)$, the local truncation error is the one-step defect made from exact data:

$$\boldsymbol{\tau}_{n+1} = \frac{\mathbf{y}(t_{n+1}) - \Phi_h(t_n, \mathbf{y}(t_n))}{h}.$$

The method is consistent if $\|\boldsymbol{\tau}_{n+1}\| \rightarrow 0$ as $h \rightarrow 0$.

Def 7.5 (Global Error and Order) The global error at t_n is

$$\mathbf{e}_n = \mathbf{y}(t_n) - \mathbf{y}_n.$$

A method has order p on a fixed interval $[t_0, T]$ if

$$\max_{0 \leq n \leq N} \|\mathbf{e}_n\| = O(h^p), \quad Nh = T - t_0.$$

Thm 7.2 (Local to Global Error) For a stable one-step method, a one-step defect $O(h^{p+1})$ gives global error $O(h^p)$ on a fixed time interval.

Ex 7.2

1. Suppose each step contributes a local error of size $O(h^{p+1})$.
2. Count the number of steps needed to reach a fixed final time.
3. Explain why this gives $O(h^p)$ global error when previous errors are not amplified strongly.
4. State where stability enters this argument.

Def 7.6 (Consistency, Stability, Convergence) For time stepping methods, consistency means the discrete update matches the differential equation as $h \rightarrow 0$. Stability means perturbations introduced at one step do not grow uncontrollably over a fixed interval. Convergence means $\mathbf{y}_n \rightarrow \mathbf{y}(t_n)$ as $h \rightarrow 0$.

Thm 7.3 (ODE Lax Principle) For a consistent one-step method applied to a Lipschitz IVP, stability on a fixed interval implies convergence. For linear multistep methods, consistency plus zero-stability implies convergence.

Ex 7.3

1. Write the error recurrence for a one-step method by subtracting the numerical update from the exact one-step update.
2. Bound the propagated error using a Lipschitz constant for $\mathbf{f}$.
3. Add the local defect term from Def 7.4.
4. Apply a discrete Gronwall argument to justify the first statement in Thm 7.3.

7.1.3 Euler Methods

Def 7.7 (Forward Euler) The explicit Euler method uses the slope at the left endpoint:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(t_n, \mathbf{y}_n).$$

It is explicit because the right side is known before step $n + 1$ is computed.

Thm 7.4 (Forward Euler Error) Forward Euler has local truncation error $O(h)$ in the sense of Def 7.4, one-step defect $O(h^2)$, and global error $O(h)$ for smooth solutions on fixed intervals.

Ex 7.4

1. Taylor expand the exact solution:

$$\mathbf{y}(t_{n+1}) = \mathbf{y}(t_n) + h\mathbf{y}'(t_n) + \frac{h^2}{2}\mathbf{y}''(\xi_n).$$

2. Substitute $\mathbf{y}'(t_n) = \mathbf{f}(t_n, \mathbf{y}(t_n))$.
3. Compare with Def 7.7.
4. Use Thm 7.2 to obtain the global order.

Example (Forward Euler on decay) For $y' = -y$, $y(0) = 1$, Forward Euler gives

$$y_n = (1 - h)^n.$$

At $t = 1$ with $h = 1/4$, this gives $(3/4)^4 = 81/256 \approx 0.3164$, while the exact value is $e^{-1} \approx 0.3679$.

Def 7.8 (Backward Euler) The implicit Euler method uses the slope at the right endpoint:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h\mathbf{f}(t_{n+1}, \mathbf{y}_{n+1}).$$

At each step one solves a nonlinear equation for $\mathbf{y}_{n+1}$.

Def 7.9 (Implicit Step Equation) For Backward Euler, define

$$\mathbf{g}(\mathbf{Y}) = \mathbf{Y} - \mathbf{y}_n - h\mathbf{f}(t_{n+1}, \mathbf{Y}).$$

The next step is the root $\mathbf{g}(\mathbf{Y}) = \mathbf{0}$. Newton's method requires linear systems with

$$\mathbf{I} - h\mathbf{J}_f(t_{n+1}, \mathbf{Y}),$$

where $\mathbf{J}_f$ is the Jacobian of $\mathbf{f}$ with respect to $\mathbf{y}$.

Thm 7.5 (Backward Euler Error) Backward Euler is first order: its one-step defect is $O(h^2)$ and its global error is $O(h)$ for smooth solutions.

Ex 7.5

1. Taylor expand $\mathbf{y}(t_n)$ about t_{n+1} .
2. Rearrange the expansion to express $\mathbf{y}(t_{n+1})$ in terms of $\mathbf{y}(t_n)$ and $\mathbf{y}'(t_{n+1})$.
3. Substitute $\mathbf{y}'(t_{n+1}) = \mathbf{f}(t_{n+1}, \mathbf{y}(t_{n+1}))$.
4. Compare with Def 7.8.

Def 7.10 (Trapezoidal Method) The implicit trapezoidal method averages endpoint slopes:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{2} [\mathbf{f}(t_n, \mathbf{y}_n) + \mathbf{f}(t_{n+1}, \mathbf{y}_{n+1})].$$

It is also called Crank-Nicolson when used for time-dependent PDE discretizations.

Thm 7.6 (Trapezoidal Order) The trapezoidal method has one-step defect $O(h^3)$ and global error $O(h^2)$ for smooth solutions.

Ex 7.6

1. Write the exact integral identity

$$\mathbf{y}(t_{n+1}) = \mathbf{y}(t_n) + \int_{t_n}^{t_{n+1}} \mathbf{f}(t, \mathbf{y}(t)) dt.$$

2. Approximate the integral by the scalar trapezoidal rule from Def 6.6.
3. Use Thm 6.5 to explain the one-step defect.
4. Convert the one-step defect to global order using Thm 7.2.

7.1.4 Absolute Stability and Stiffness

Def 7.11 (Test Equation and Stability Function) The scalar test equation is

$$y' = \lambda y, \quad \operatorname{Re}(\lambda) < 0.$$

For a one-step method applied to this equation, the update has the form

$$y_{n+1} = R(z)y_n, \quad z = h\lambda.$$

The function R is the stability function.

Def 7.12 (Absolute Stability Region) The absolute stability region of a one-step method is

$$\mathcal{S} = \{z \in \mathbb{C} : |R(z)| \leq 1\}.$$

The method is stable on the test equation when $h\lambda \in \mathcal{S}$.

Thm 7.7 (Euler Stability Functions) For the test equation in Def 7.11,

$$R_{\text{FE}}(z) = 1 + z, \quad R_{\text{BE}}(z) = \frac{1}{1 - z}, \quad R_{\text{TR}}(z) = \frac{1 + z/2}{1 - z/2}.$$

Forward Euler is stable in the disk $|1 + z| \leq 1$. Backward Euler and trapezoidal rule are A-stable.

Proof *Proof.* For Forward Euler, $y_{n+1} = y_n + h\lambda y_n = (1 + z)y_n$. For Backward Euler,

$$y_{n+1} = y_n + h\lambda y_{n+1} \quad \Rightarrow \quad y_{n+1} = \frac{1}{1 - z}y_n.$$

For trapezoidal rule,

$$y_{n+1} = y_n + \frac{z}{2}(y_n + y_{n+1}) \quad \Rightarrow \quad y_{n+1} = \frac{1 + z/2}{1 - z/2}y_n.$$

If $\operatorname{Re}(z) \leq 0$, then $|1| \leq |1 - z|$ and $|1 + z/2| \leq |1 - z/2|$, so Backward Euler and trapezoidal rule are A-stable. $\square$

Ex 7.7

1. Draw the Forward Euler stability disk from $|1 + z| \leq 1$.
2. For real $\lambda < 0$, show that Forward Euler requires $h \leq 2/|\lambda|$.
3. Verify $|1 + z/2| \leq |1 - z/2|$ when $\text{Re}(z) \leq 0$ by squaring both sides.
4. Explain why A-stability does not imply L-stability.

Def 7.13 (A-Stability and L-Stability) A method is A-stable if its stability region contains the full left half-plane:

$$\{z \in \mathbb{C} : \text{Re}(z) \leq 0\} \subseteq \mathcal{S}.$$

An A-stable method is L-stable if additionally $R(z) \rightarrow 0$ as $z \rightarrow -\infty$ along the real axis.

Remark 7.3 (Damping) Backward Euler is L-stable, so very fast decaying modes are strongly damped. The trapezoidal method is A-stable but not L-stable, since $R_{\text{TR}}(z) \rightarrow -1$ as $z \rightarrow -\infty$.

Def 7.14 (Stiffness) An IVP is stiff when stability forces an explicit method to take much smaller steps than accuracy requires. For a linear system

$$\mathbf{y}' = \mathbf{A}\mathbf{y},$$

the eigenvalues of $\mathbf{A}$ determine the explicit stability restriction through $h\lambda_i \in \mathcal{S}$.

Example (Fast decay with slow forcing) The equation

$$y' = -1000(y - \cos t) - \sin t$$

has exact solution $y(t) = \cos t$ for consistent initial data. The solution varies on an $O(1)$ time scale, but Forward Euler is restricted by the fast stable mode near $\lambda = -1000$ and requires $h \lesssim 0.002$.

Ex 7.8

1. Verify directly that $y(t) = \cos t$ solves the stiff example above.
2. Linearize the equation around the exact solution and identify the error equation.
3. Apply Thm 7.7 to obtain the Forward Euler stability restriction.
4. Explain why this restriction is unrelated to the visible time scale of $\cos t$.

7.1.5 Runge-Kutta Methods

Def 7.15 (Explicit Runge-Kutta Method) An s -stage explicit Runge-Kutta method has stages

$$\mathbf{k}_i = \mathbf{f} \left(t_n + c_i h, \mathbf{y}_n + h \sum_{j < i} a_{ij} \mathbf{k}_j \right), \quad i = 1, \dots, s,$$

and update

$$\mathbf{y}_{n+1} = \mathbf{y}_n + h \sum_{i=1}^s b_i \mathbf{k}_i.$$

The coefficients are collected in a Butcher tableau.

Def 7.16 (Explicit Midpoint Method) The two-stage explicit midpoint method is

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{f}(t_n, \mathbf{y}_n), \\ \mathbf{k}_2 &= \mathbf{f}(t_n + h/2, \mathbf{y}_n + h\mathbf{k}_1/2), \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + h\mathbf{k}_2. \end{aligned}$$

It has global order 2.

Ex 7.9

1. Taylor expand the exact solution to second order.
2. Taylor expand $\mathbf{f}(t_n + h/2, \mathbf{y}_n + h\mathbf{k}_1/2)$ about $(t_n, \mathbf{y}_n)$.
3. Match terms to show second-order consistency.
4. Derive the stability function of explicit midpoint on $y' = \lambda y$.

Def 7.17 (RK4) The classical fourth-order Runge-Kutta method is

$$\begin{aligned} \mathbf{k}_1 &= \mathbf{f}(t_n, \mathbf{y}_n), \\ \mathbf{k}_2 &= \mathbf{f}(t_n + h/2, \mathbf{y}_n + h\mathbf{k}_1/2), \\ \mathbf{k}_3 &= \mathbf{f}(t_n + h/2, \mathbf{y}_n + h\mathbf{k}_2/2), \\ \mathbf{k}_4 &= \mathbf{f}(t_n + h, \mathbf{y}_n + h\mathbf{k}_3), \\ \mathbf{y}_{n+1} &= \mathbf{y}_n + \frac{h}{6}(\mathbf{k}_1 + 2\mathbf{k}_2 + 2\mathbf{k}_3 + \mathbf{k}_4). \end{aligned}$$

It has one-step defect $O(h^5)$ and global error $O(h^4)$ for smooth nonstiff problems.

Thm 7.8 (RK4 Stability Polynomial) Applied to $y' = \lambda y$, RK4 has stability function

$$R(z) = 1 + z + \frac{z^2}{2} + \frac{z^3}{6} + \frac{z^4}{24}.$$

This is the degree-four Taylor polynomial of e^z .

Ex 7.10

1. Apply Def 7.17 to $y' = \lambda y$.
2. Express each stage in terms of $z = h\lambda$ and y_n .
3. Substitute the stages into the RK4 update.
4. Derive the stability polynomial in Thm 7.8.

Remark 7.4 (High order is not stiffness control) RK4 has higher accuracy than Forward Euler on nonstiff problems, but its stability region is still bounded. For stiff decay, an explicit high-order method can remain limited by stability rather than accuracy.

7.1.6 Adaptive Time Stepping

Def 7.18 (Embedded Runge-Kutta Pair) An embedded pair computes two approximations of different orders from shared stages:

$$\mathbf{y}_{n+1}^{[p]}, \quad \mathbf{y}_{n+1}^{[p+1]}.$$

The difference

$$\mathbf{e}_{n+1} = \mathbf{y}_{n+1}^{[p+1]} - \mathbf{y}_{n+1}^{[p]}$$

estimates the local error of the lower-order method.

Def 7.19 (Weighted Error Test) Adaptive solvers usually accept a step when

$$\left\| \frac{\mathbf{e}_{n+1}}{\text{atol} + \text{rtol} |\mathbf{y}_{n+1}|} \right\| \leq 1,$$

where the absolute value and division are componentwise.

Thm 7.9 (Adaptive Step Formula) If the local error estimate satisfies $\|\mathbf{e}\| \approx Ch^{p+1}$, then a step size targeting tolerance ε is

$$h_{\text{new}} = s h_{\text{old}} \left(\frac{\varepsilon}{\|\mathbf{e}\|} \right)^{1/(p+1)},$$

where $0 < s < 1$ is a safety factor.

Ex 7.11

1. Assume $\|\mathbf{e}_{\text{old}}\| \approx Ch_{\text{old}}^{p+1}$.

2. Solve $Ch_{\text{new}}^{p+1} \approx \varepsilon$.
3. Eliminate C using the old error estimate.
4. Explain why production codes also limit the maximum growth and shrinkage of h .

Remark 7.5 (**RK45**) Dormand-Prince RK45 is a common embedded explicit pair. It is a good default for smooth nonstiff IVPs, not for stiff systems.

7.1.7 Linear Multistep Methods

Def 7.20 (**Linear Multistep Method**) A k -step linear multistep method has the form

$$\sum_{j=0}^k \alpha_j \mathbf{y}_{n+j} = h \sum_{j=0}^k \beta_j \mathbf{f}(t_{n+j}, \mathbf{y}_{n+j}).$$

It reuses previous solution values and previous right-hand side evaluations.

Def 7.21 (**Adams-Bashforth 2**) The two-step Adams-Bashforth method is explicit:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{2} (3\mathbf{f}(t_n, \mathbf{y}_n) - \mathbf{f}(t_{n-1}, \mathbf{y}_{n-1})).$$

It has global order 2.

Def 7.22 (**Adams-Moulton 2**) The two-step Adams-Moulton method is implicit:

$$\mathbf{y}_{n+1} = \mathbf{y}_n + \frac{h}{2} (\mathbf{f}(t_{n+1}, \mathbf{y}_{n+1}) + \mathbf{f}(t_n, \mathbf{y}_n)).$$

This is the trapezoidal method from Def 7.10.

Def 7.23 (**BDF2**) The second-order backward differentiation formula is

$$\frac{3\mathbf{y}_{n+1} - 4\mathbf{y}_n + \mathbf{y}_{n-1}}{2h} = \mathbf{f}(t_{n+1}, \mathbf{y}_{n+1}).$$

BDF methods are standard implicit methods for stiff IVPs.

Def 7.24 (**Zero-Stability**) A linear multistep method is zero-stable if perturbations in starting values remain uniformly bounded as $h \rightarrow 0$ on a fixed interval. Algebraically, the roots of

$$\rho(\xi) = \sum_{j=0}^k \alpha_j \xi^j$$

must satisfy $|\xi| \leq 1$, and roots on the unit circle must be simple.

Thm 7.10 (**Dahlquist Equivalence Theorem**) For a consistent linear multistep method applied to a well-posed IVP, convergence is equivalent to zero-stability.

Ex 7.12

1. For AB2, write the polynomial $\rho(\xi)$.
2. Check the root condition in Def 7.24.
3. Repeat for BDF2.
4. Explain why consistency alone is not enough for a multistep method.

Remark 7.6 (BDF for stiffness) BDF methods trade implicit linear or nonlinear solves for much larger stable time steps on stiff decay problems. Variable-order BDF methods are used in many stiff ODE codes.

7.1.8 Systems and Eigenvalues

Def 7.25 (First-Order System Form) A higher-order scalar equation can be rewritten as a first-order system. For example,

$$q'' = F(q, q', t)$$

becomes

$$q' = v, \quad v' = F(q, v, t).$$

Thm 7.11 (Linear Constant-Coefficient System) For

$$\mathbf{y}' = \mathbf{A}\mathbf{y}, \quad \mathbf{y}(0) = \mathbf{y}_0,$$

the exact solution is

$$\mathbf{y}(t) = e^{t\mathbf{A}}\mathbf{y}_0.$$

If $\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}$, then

$$\mathbf{y}(t) = \mathbf{V}e^{t\mathbf{D}}\mathbf{V}^{-1}\mathbf{y}_0.$$

Each eigenvalue controls one modal growth, decay, or oscillation rate.

Ex 7.13

1. Differentiate $e^{t\mathbf{A}}\mathbf{y}_0$ to verify the first formula in Thm 7.11.
2. Substitute $\mathbf{A} = \mathbf{V}\mathbf{D}\mathbf{V}^{-1}$ into the matrix exponential.
3. For $\lambda = a + ib$, interpret $e^{\lambda t}$ in terms of growth and oscillation.
4. Explain why eigenvalues of the Jacobian predict local stiffness for nonlinear systems.

Remark 7.7 (Jacobian spectrum) Near a state $\mathbf{y}_*$, nonlinear dynamics are locally approximated by $\mathbf{e}' = \mathbf{J}_f(\mathbf{y}_*)\mathbf{e}$. Large negative eigenvalues produce fast stable transients. Eigenvalues near the imaginary axis produce oscillation or slow decay.

7.1.9 Hamiltonian and Symplectic Methods

Def 7.26 (Hamiltonian System) For position $\mathbf{q}$ and velocity or momentum $\mathbf{v}$, a separable Hamiltonian has

$$H(\mathbf{q}, \mathbf{v}) = \frac{1}{2}\|\mathbf{v}\|^2 + V(\mathbf{q}), \quad \mathbf{q}' = \mathbf{v}, \quad \mathbf{v}' = \mathbf{F}(\mathbf{q}) = -\nabla V(\mathbf{q}).$$

The exact flow conserves H .

Def 7.27 (Velocity Verlet) For the Hamiltonian system in Def 7.26, Velocity Verlet is

$$\begin{aligned} \mathbf{v}_{n+1/2} &= \mathbf{v}_n + \frac{h}{2}\mathbf{F}(\mathbf{q}_n), \\ \mathbf{q}_{n+1} &= \mathbf{q}_n + h\mathbf{v}_{n+1/2}, \\ \mathbf{v}_{n+1} &= \mathbf{v}_{n+1/2} + \frac{h}{2}\mathbf{F}(\mathbf{q}_{n+1}). \end{aligned}$$

It is second order, time reversible, and symplectic.

Thm 7.12 (Harmonic Oscillator Energy Test) For $q' = v$, $v' = -q$, Forward Euler maps the energy

$$E(q, v) = \frac{1}{2}(q^2 + v^2)$$

to $(1 + h^2)E(q, v)$ after one step. Thus Forward Euler has artificial energy growth on this oscillator.

Proof *Proof.* Forward Euler gives

$$q_{n+1} = q_n + hv_n, \quad v_{n+1} = v_n - hq_n.$$

Therefore

$$q_{n+1}^2 + v_{n+1}^2 = (q_n + hv_n)^2 + (v_n - hq_n)^2 = (1 + h^2)(q_n^2 + v_n^2).$$

Multiplying by 1/2 proves the claim. □

Ex 7.14

1. Apply Velocity Verlet to $q' = v$, $v' = -q$.
2. Show that reversing the step size from h to $-h$ undoes one Verlet step.
3. Compare long-time energy behavior of Forward Euler, RK4, and Velocity Verlet numerically.

Remark 7.8 (Symplectic accuracy) For long Hamiltonian simulations, preserving phase-space geometry can matter more than minimizing one-step error. Symplectic methods usually keep the energy error bounded and oscillatory over long times.

7.1.10 Method Choice

Def 7.28 (Common Solver Regimes)

Problem	Typical method
Smooth nonstiff IVP	Adaptive explicit RK, such as RK45
High-accuracy nonstiff IVP	Higher-order adaptive explicit RK
Stiff IVP	Implicit BDF or implicit Runge-Kutta
Hamiltonian long-time simulation	Symplectic method, such as Verlet
Large stiff system	Implicit method with sparse Jacobian solves

Remark 7.9 (SciPy interface) In `scipy.integrate.solve_ivp`, methods such as RK45 and DOP853 are explicit nonstiff solvers. Methods such as BDF and Radau are intended for stiff problems and benefit from Jacobian information.

Ex 7.15

1. For $y' = -y$, compare Forward Euler, RK4, and RK45 on accuracy versus function evaluations.
2. For the stiff example in Ex 7.8, compare RK45 and BDF using the same tolerance.
3. For a sparse semidiscrete heat equation $\mathbf{y}' = \mathbf{A}\mathbf{y}$, identify why Jacobian sparsity matters.
4. For the harmonic oscillator, compare final-time error and energy error for RK4 and Velocity Verlet.

8 Applications

8.1 Principal Component Analysis (PCA)

Optimal linear dimensionality reduction via variance maximization.

8.1.1 Definitions and Preprocessing

Def 8.1 (Data Centering) Data matrix $\mathbf{X} \in \mathbb{R}^{n \times p}$ (n samples, p features) must be **centered**:

$$X_{ij} \leftarrow X_{ij} - \bar{x}_j, \quad \bar{x}_j = \frac{1}{n} \sum_{i=1}^n X_{ij}.$$

Throughout this section, $\mathbf{X}$ is assumed centered.

Def 8.2 (Sample Covariance Matrix) The matrix $\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} \in \mathbb{R}^{p \times p}$.

- **Diagonal:** S_{jj} is the variance of feature j .
- **Off-diagonal:** S_{ij} is the covariance between features i and j .
- **Property:** $\mathbf{S}$ is symmetric positive semidefinite (SPSD).

Ex 8.1

1. Show that centering makes each column of $\mathbf{X}$ have mean zero.
2. Prove that the diagonal entries of $\mathbf{S}$ are sample variances.
3. Prove that $\mathbf{S}$ is symmetric positive semidefinite by computing $\mathbf{z}^T \mathbf{S} \mathbf{z}$.

8.1.2 PCA via SVD

Thm 8.1 (Principal Directions and Scores) Let $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$ be the SVD of the centered data matrix.

- **Principal Directions (Loadings):** Columns of $\mathbf{V}$ (eigenvectors of $\mathbf{S}$).
- **Principal Components (Scores):** $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$.
- **Variances:** $\lambda_i = \text{Var}(\mathbf{z}_i) = \frac{\sigma_i^2}{n-1}$.

Proof *Proof.* Since $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$,

$$\mathbf{S} = \frac{1}{n-1} \mathbf{X}^T \mathbf{X} = \mathbf{V} \left(\frac{\mathbf{\Sigma}^2}{n-1} \right) \mathbf{V}^T.$$

Thus the columns of $\mathbf{V}$ diagonalize the covariance matrix. The scores satisfy $\mathbf{Z} = \mathbf{X}\mathbf{V} = \mathbf{U}\mathbf{\Sigma}$, so their sample covariance is diagonal with entries $\sigma_i^2/(n-1)$. $\square$

Remark 8.1 (PCA stability) Never form $\mathbf{X}^T \mathbf{X}$ explicitly for PCA. Squaring the data matrix doubles the condition number ($\kappa(\mathbf{S}) = \kappa(\mathbf{X})^2$). Compute PCA directly via the SVD of $\mathbf{X}$.

Ex 8.2

1. Starting from $\mathbf{X} = \mathbf{U}\mathbf{\Sigma}\mathbf{V}^T$, compute $\mathbf{S} = (n-1)^{-1} \mathbf{X}^T \mathbf{X}$.
2. Show that the columns of $\mathbf{V}$ are eigenvectors of $\mathbf{S}$.
3. Show that the corresponding eigenvalues are $\lambda_i = \sigma_i^2/(n-1)$.
4. Compute the score matrix $\mathbf{Z} = \mathbf{X}\mathbf{V}$ and prove that $\text{Cov}(\mathbf{Z})$ is diagonal.
5. Use Ex 4.2 to explain the stability warning above.

8.1.3 Variance and Dimensionality Reduction

Def 8.3 (Proportion of Variance Explained (PVE)) For the first k components:

$$\text{PVE}_k = \frac{\sum_{i=1}^k \sigma_i^2}{\sum_{i=1}^p \sigma_i^2}.$$

Thm 8.2 (Optimal Approximation) The rank- k matrix

$$\mathbf{X}_k = \sum_{i=1}^k \sigma_i \mathbf{u}_i \mathbf{v}_i^T$$

is the optimal k -dimensional linear approximation of the data in the Frobenius norm, by the Eckart-Young theorem.

Ex 8.3

1. Use Thm 4.6 to prove Thm 8.2.
2. Show that $\|\mathbf{X} - \mathbf{X}_k\|_F^2 = \sum_{i>k} \sigma_i^2$.
3. Show that $\text{PVE}_k = 1 - \|\mathbf{X} - \mathbf{X}_k\|_F^2 / \|\mathbf{X}\|_F^2$.
4. Explain why a PVE threshold is a modeling choice, not a theorem.

8.1.4 Implementation Details

Remark 8.2 (Standardization) If features have different units, PCA will be dominated by large-scale features. Standardize by dividing each centered column by its standard deviation: $X_{ij} \leftarrow (X_{ij} - \bar{x}_j) / s_j$.

Remark 8.3 (Interpretation) Principal components are variance directions, not causal factors. A large loading identifies a direction of variation in the data matrix; it does not by itself explain why that variation occurs.

Def 8.4 (Kernel PCA (High-Dimensional Case)) When $p \gg n$, work with the $n \times n$ Gram matrix $\mathbf{K} = \mathbf{X}\mathbf{X}^T$. Principal directions are recovered via $\mathbf{v}_i = \mathbf{X}^T \mathbf{u}_i / \sigma_i$.

8.1.5 Exercises

Ex 8.4

1. Center $\mathbf{X} = \begin{pmatrix} 2 & 1 \\ -1 & 3 \\ -1 & -4 \end{pmatrix}$ and compute $\mathbf{S}$. Find the PVE for $k = 1$.

2. Prove that PC scores $\mathbf{z}_i, \mathbf{z}_j$ are uncorrelated for $i \neq j$ using Ex 8.2.
3. Load the Iris dataset. Plot the data in the basis of the first two PCs. Do species cluster?
4. Verify that the sum of variances $\sum \lambda_i$ equals the total variance $\text{Tr}(\mathbf{S})$.

8.2 Spectral Graph Theory and PageRank

Graph algorithms become numerical linear algebra after choosing matrix conventions. Undirected graphs lead to symmetric Laplacians. Directed ranking problems lead to stochastic matrices and eigenvectors.

8.2.1 Graph Matrices

Remark 8.4 (**Convention**) For undirected graphs, $\mathbf{A}$ is symmetric. For directed random walks in this section, probability vectors are columns and P_{ij} is the probability of moving from node j to node i .

Def 8.5 (**Adjacency and Degree Matrices**) For a graph $G = (V, E)$ with n vertices:

- **Adjacency $\mathbf{A}$:** $A_{ij} = w_{ij}$ if $(i, j) \in E$, else 0.
- **Degree $\mathbf{D}$:** Diagonal matrix with $D_{ii} = d_i = \sum_j w_{ij}$.

Def 8.6 (**Graph Laplacian**) Defined as $\mathbf{L} = \mathbf{D} - \mathbf{A}$.

- **Discrete Derivative:** $(\mathbf{L}\mathbf{f})_i = \sum_{j \sim i} w_{ij}(f_i - f_j)$.
- **Quadratic Form:** $\mathbf{x}^T \mathbf{L} \mathbf{x} = \sum_{(i,j) \in E} w_{ij}(x_i - x_j)^2$, with each undirected edge counted once.

Prop 8.3 (**Laplacian Positive Semidefinite**) For an undirected weighted graph with nonnegative weights, $\mathbf{L}$ is symmetric positive semidefinite and $\mathbf{L}\mathbf{1} = \mathbf{0}$.

Proof *Proof.* Symmetry follows from $w_{ij} = w_{ji}$. Using the quadratic form in Def 8.6,

$$\mathbf{x}^T \mathbf{L} \mathbf{x} = \sum_{(i,j) \in E} w_{ij}(x_i - x_j)^2 \geq 0.$$

Thus $\mathbf{L}$ is positive semidefinite. If $\mathbf{x} = \mathbf{1}$, every difference $x_i - x_j$ is zero, and direct substitution gives $\mathbf{L}\mathbf{1} = \mathbf{0}$. $\square$

Example (Three-node path) For $1 - 2 - 3$,

$$\mathbf{A} = \begin{pmatrix} 0 & 1 & 0 \\ 1 & 0 & 1 \\ 0 & 1 & 0 \end{pmatrix}, \quad \mathbf{L} = \begin{pmatrix} 1 & -1 & 0 \\ -1 & 2 & -1 \\ 0 & -1 & 1 \end{pmatrix}.$$

The zero eigenvector is $\mathbf{1}$ because constant signals have no graph variation.

Thm 8.4 (Spectral Properties) For a connected graph with $0 = \lambda_1 \leq \lambda_2 \leq \dots \leq \lambda_n$:

1. $\lambda_1 = 0$ with eigenvector $\mathbf{1}$.
2. **Multiplicity of 0:** Equals the number of connected components.
3. **Fiedler Value (λ_2):** Measures algebraic connectivity. $\lambda_2 > 0$ iff graph is connected.

Ex 8.5

1. Use Def 8.6 to show $\mathbf{L}\mathbf{1} = \mathbf{0}$.
2. Derive $\mathbf{x}^T \mathbf{L} \mathbf{x} = \sum_{(i,j) \in E} w_{ij} (x_i - x_j)^2$ for edges counted once.
3. Conclude that $\mathbf{L}$ is positive semidefinite for an undirected graph.
4. Show that if a graph has two connected components, then there are two independent vectors in $\mathcal{N}(\mathbf{L})$.

Def 8.7 (Normalized Laplacians)

- **Symmetric:** $\mathbf{L}_{\text{sym}} = \mathbf{D}^{-1/2} \mathbf{L} \mathbf{D}^{-1/2}$. (Eigenvalues in $[0, 2]$).
- **Random Walk:** $\mathbf{L}_{\text{rw}} = \mathbf{I} - \mathbf{A} \mathbf{D}^{-1}$ under the column-stochastic convention.

Remark 8.5 (Degree normalization) The unnormalized Laplacian can let high-degree vertices dominate cuts. Normalized Laplacians measure variation relative to vertex degree.

8.2.2 Spectral Clustering

Thm 8.5 (Spectral Relaxation) Minimizing a graph cut (RatioCut) is NP-hard. Spectral methods relax this to an eigenvector problem:

1. Compute k smallest eigenvectors of $\mathbf{L}$ (starting from $\mathbf{v}_2$).
2. Embed vertices in $\mathbb{R}^k$ using these eigenvectors.
3. Cluster embedding via k -means.

8.2.3 Random Walks and PageRank

Def 8.8 (Random Walk Matrix) Transition matrix $\mathbf{P} = \mathbf{A}\mathbf{D}^{-1}$ under the column-stochastic convention.

- $P_{ij} = w_{ij}/d_j$ is the probability of moving from j to i .
- **Stationary Distribution:** $\boldsymbol{\pi}$ such that $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$. For undirected graphs, $\pi_i = d_i / \sum_j d_j$.

Ex 8.6

1. Show that the columns of $\mathbf{P} = \mathbf{A}\mathbf{D}^{-1}$ sum to one.
2. For the path graph $1 - 2 - 3$, compute $\mathbf{P}$.
3. Verify that $\pi_i = d_i / \sum_j d_j$ satisfies $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.

Def 8.9 (PageRank and Google Matrix) Models a random surfer on a directed web graph with teleportation:

$$\mathbf{G} = d\mathbf{S} + \frac{1-d}{N}\mathbf{J},$$

- d is the damping factor (standard: 0.85).
- $\mathbf{S}$ is column-stochastic and handles dangling nodes.
- $\mathbf{J}$ is the all-ones matrix (teleportation).

Computation: PageRank vector $\mathbf{r}$ is the dominant eigenvector of $\mathbf{G}$, computed via power iteration.

Remark 8.6 (Dangling nodes) A page with no outgoing links gives a zero column before normalization. The standard fix replaces that column by the uniform probability vector before forming $\mathbf{S}$.

8.2.4 Exercises

Ex 8.7

1. Construct $\mathbf{L}$ for a 3-vertex path graph. Find its eigenvalues by hand.
2. Show that $\mathbf{L}\mathbf{1} = \mathbf{0}$ for any graph.
3. Prove $\mathbf{L}$ is positive semidefinite using the quadratic form.
4. Implement PageRank power iteration for a 10-node random graph.

8.3 Linear Operators and the Poisson Equation

Differential equations become matrix problems after discretization. The finite matrix should preserve the main structure of the operator: sparsity, symmetry, definiteness, spectrum, and conditioning.

8.3.1 Differential Operators and Discretization

Def 8.10 (Linear Operator) Map $\mathcal{A} : X \rightarrow Y$ between function spaces satisfying:

$$\mathcal{A}(\alpha u + v) = \alpha \mathcal{A}u + \mathcal{A}v.$$

Matrices are finite-dimensional operators; $\frac{d}{dx}$ and ∇^2 are infinite-dimensional.

Def 8.11 (Dirichlet Poisson Problem) The one-dimensional Poisson problem with homogeneous Dirichlet boundary data is

$$-u''(x) = f(x), \quad 0 < x < 1, \quad u(0) = u(1) = 0.$$

The boundary conditions are part of the problem data.

Def 8.12 (Grid Function) For n interior grid points, let $h = 1/(n+1)$ and $x_i = ih$, $i = 1, \dots, n$. A grid function is the vector

$$\mathbf{u} = (u_1, \dots, u_n)^T, \quad u_i \approx u(x_i).$$

The boundary values are fixed separately by the boundary conditions.

Def 8.13 (Finite Difference Stencil) Approximating $u''(x)$ on a grid with spacing h :

$$u''(x_k) \approx \frac{u(x_{k-1}) - 2u(x_k) + u(x_{k+1}))}{h^2}.$$

Accuracy: Second-order $O(h^2)$ via Taylor expansion.

Ex 8.8

1. Taylor expand $u(x+h)$ and $u(x-h)$ about x .
2. Add the two expansions.
3. Solve for $u''(x)$.
4. Identify the leading error term and prove the $O(h^2)$ claim in Def 8.13.

Def 8.14 (1-D Discrete Laplacian) Discretizing $-d^2/dx^2$ on $[0, 1]$ with $u(0) = u(1) = 0$ yields the tridiagonal system $\mathbf{T}\mathbf{u} = \mathbf{f}$:

$$\mathbf{T} = \frac{1}{h^2} \text{tridiag}(-1, 2, -1) \in \mathbb{R}^{n \times n}.$$

Remark 8.7 (Sparsity) Each row of $\mathbf{T}$ uses only the grid point and its two neighbors. This locality is why finite difference matrices are sparse.

Thm 8.6 (Spectrum Convergence) Eigenvalues of $\mathbf{T}$ are $\lambda_k = \frac{4}{h^2} \sin^2(\frac{k\pi}{2(n+1)})$. As $n \rightarrow \infty$, $\lambda_k \rightarrow k^2\pi^2$ (eigenvalues of continuous $-d^2/dx^2$).

Ex 8.9

1. Let $(\mathbf{v}_k)_j = \sin(jk\pi h)$ with $h = 1/(n+1)$.
2. Compute the j -th entry of $\mathbf{T}\mathbf{v}_k$.
3. Use $\sin(A-B) + \sin(A+B) = 2\sin A \cos B$.
4. Use $1 - \cos \theta = 2\sin^2(\theta/2)$.
5. Derive the eigenvalue formula in Thm 8.6.
6. Let $h \rightarrow 0$ with fixed k and recover $k^2\pi^2$.

Thm 8.7 (Discrete Poisson Structure) The matrix $\mathbf{T}$ in Def 8.14 is sparse, symmetric positive definite, and has condition number

$$\kappa_2(\mathbf{T}) = O(h^{-2}).$$

Proof *Proof.* Sparsity and symmetry are immediate from the tridiagonal stencil. By Thm 8.6, all eigenvalues are positive because $1 \leq k \leq n$ and $0 < k\pi/(2(n+1)) < \pi/2$. Thus $\mathbf{T}$ is SPD. Also

$$\lambda_{\min} \sim \pi^2, \quad \lambda_{\max} \sim \frac{4}{h^2},$$

so $\kappa_2(\mathbf{T}) = \lambda_{\max}/\lambda_{\min} = O(h^{-2})$. □

Ex 8.10

1. Use the stencil to count the number of nonzeros in $\mathbf{T}$.
2. Use the quadratic form $\mathbf{u}^T \mathbf{T} \mathbf{u}$ to show positive definiteness for nonzero interior grid functions.
3. Use Thm 8.6 to estimate $\lambda_{\min}$ and $\lambda_{\max}$.
4. Derive the condition number estimate in Thm 8.7.

8.3.2 Operator Norms and Conditioning

Def 8.15 (**Boundedness**) An operator is **bounded** if $\|\mathcal{A}u\|_Y \leq C\|u\|_X$ for some $C < \infty$.

- Matrices are always bounded.
- Differential operators are **unbounded**.

Remark 8.8 **Numerical Consequence:** Because the continuous Laplacian is unbounded, its discretization $\mathbf{T}$ becomes increasingly ill-conditioned as $h \rightarrow 0$:

$$\kappa(\mathbf{T}) = \frac{\lambda_{\max}}{\lambda_{\min}} \approx \frac{4/h^2}{\pi^2} = O(h^{-2}).$$

Refining the grid $2\times$ quadruples the condition number.

Remark 8.9 (**Solver consequence**) Dense Gaussian elimination ignores sparsity. For Poisson matrices, sparse Cholesky, CG with preconditioning, multigrid, or FFT-based solvers are the natural algorithms, depending on geometry and boundary conditions.

8.3.3 Exercises

Ex 8.11

1. Build $\mathbf{T}$ for $n = 50$ and plot its eigenvalues vs. $k^2\pi^2$ using Thm 8.6.
2. Show that $\kappa(\mathbf{T}) \rightarrow \infty$ as $n \rightarrow \infty$ using Thm 8.7.
3. Implement a 2-D Poisson solver on a unit square. Verify $O(N^{3/2})$ cost for direct sparse solve.
4. Prove the differentiation operator is unbounded on $L^2[0, 1]$ using $u_k(x) = \sin(k\pi x)$.

8.4 Fourier Analysis as Change of Basis

Fourier analysis provides a framework for representing functions and data in terms of frequency components. In engineering, this transformation is critical for signal processing, image compression, and solving differential equations where the Laplacian operator is diagonalized by the Fourier basis.

8.4.1 Discrete Fourier Transform (DFT)

Def 8.16 (**Periodic Grid Signal**) A discrete signal $\mathbf{x} \in \mathbb{C}^n$ is interpreted as one period of a periodic sequence:

$$x_{j+n} = x_j.$$

The DFT represents this signal in the basis of discrete complex exponentials.

Def 8.17 (DFT Matrix) Unitary matrix $\mathbf{F} \in \mathbb{C}^{n \times n}$ with entries:

$$F_{jk} = \frac{1}{\sqrt{n}} \omega_n^{jk}, \quad \omega_n = e^{-2\pi i/n}.$$

- **Unitary Property:** $\mathbf{F}^* \mathbf{F} = \mathbf{I}$. Orthonormal columns.
- **Inverse DFT:** $\mathbf{F}^{-1} = \mathbf{F}^*$.

Thm 8.8 (Parseval's Theorem) The DFT is an isometry: $\|\mathbf{F}\mathbf{x}\|_2 = \|\mathbf{x}\|_2$. Total energy is preserved between time and frequency domains.

Proof *Proof.* Since $\mathbf{F}$ is unitary, $\mathbf{F}^* \mathbf{F} = \mathbf{I}$. Hence

$$\|\mathbf{F}\mathbf{x}\|_2^2 = (\mathbf{F}\mathbf{x})^* (\mathbf{F}\mathbf{x}) = \mathbf{x}^* \mathbf{F}^* \mathbf{F} \mathbf{x} = \mathbf{x}^* \mathbf{x} = \|\mathbf{x}\|_2^2.$$

Taking square roots gives the claim. □

Ex 8.12

1. Use Def 8.17 to compute the (j, ℓ) entry of $\mathbf{F}^* \mathbf{F}$.
2. Evaluate the finite geometric sum that appears.
3. Conclude $\mathbf{F}^* \mathbf{F} = \mathbf{I}$.
4. Explain why this is the finite Fourier analog of orthonormal coordinates.

Def 8.18 (Frequency Bins) For samples on $[0, 2\pi)$, the integer frequency attached to index k is usually ordered as

$$0, 1, 2, \dots, \lfloor n/2 \rfloor, -\lfloor (n-1)/2 \rfloor, \dots, -1.$$

This is the ordering returned by `np.fft.fftfreq` after scaling by n .

Remark 8.10 (Normalization convention) Some libraries put the factor $1/n$ in the inverse transform instead of using the unitary factor $1/\sqrt{n}$ in both directions. The formulas change by constants, but the frequency content is the same.

Def 8.19 (Nyquist Frequency and Aliasing) On n equally spaced samples of a 2π -periodic signal, frequencies that differ by multiples of n are indistinguishable:

$$e^{i(k+n)x_j} = e^{ikx_j}.$$

The largest resolvable unsigned frequency is the Nyquist frequency, approximately $n/2$ modes per period.

Ex 8.13

1. Let $x_j = 2\pi j/n$. Prove $e^{i(k+n)x_j} = e^{ikx_j}$ for every grid point.
2. For $n = 8$, list the frequency bin ordering from Def 8.18.
3. Sample $\sin(7x)$ on $n = 8$ points and identify its aliased frequency.

8.4.2 Convolution and Circulant Matrices

Def 8.20 (**Circulant Matrix**) Matrix $\mathbf{C}$ where each row is a cyclic shift of the first column $\mathbf{c}$.

- **Diagonalization:** Every circulant matrix is diagonalized by the DFT: $\mathbf{C} = \mathbf{F}^* \text{diag}(\mathbf{F}\mathbf{c})\mathbf{F}$.
- **Eigenvalues:** The eigenvalues of $\mathbf{C}$ are the DFT coefficients of its first column.

Thm 8.9 (**Convolution Theorem**) Circular convolution $\mathbf{z} = \mathbf{c} * \mathbf{x}$ is equivalent to pointwise multiplication in frequency:

$$\mathbf{F}(\mathbf{c} * \mathbf{x}) = (\mathbf{F}\mathbf{c}) \odot (\mathbf{F}\mathbf{x}).$$

Complexity: Reduced from $O(n^2)$ (direct) to $O(n \log n)$ (FFT).

Example (**Circular convolution for $n = 3$**) If $\mathbf{c} = (c_0, c_1, c_2)^T$, then

$$\mathbf{c} * \mathbf{x} = \begin{pmatrix} c_0 & c_2 & c_1 \\ c_1 & c_0 & c_2 \\ c_2 & c_1 & c_0 \end{pmatrix} \begin{pmatrix} x_0 \\ x_1 \\ x_2 \end{pmatrix}.$$

The wraparound terms are a consequence of the periodic convention in Def 8.16.

8.4.3 The Fast Fourier Transform (FFT)

Thm 8.10 (**Cooley-Tukey Algorithm**) Computes the DFT in $O(n \log n)$ flops for $n = 2^p$.

- **Mechanism:** Recursively splits n -point DFT into two $n/2$ -point DFTs (even/odd indices).
- **NumPy:** Use `np.fft.fft` and `np.fft.ifft`.

Ex 8.14

1. Split the DFT sum into even and odd input indices.
2. Show that each part is an $n/2$ -point DFT multiplied by phase factors.
3. Write the recurrence $T(n) = 2T(n/2) + O(n)$.
4. Solve the recurrence to obtain the cost in Thm 8.10.

Remark 8.11 **Spectral Differentiation:** For smooth periodic signals, differentiate by multiplying frequency components $\hat{x}_k$ by ik . Provides exponential accuracy compared to polynomial accuracy of finite differences.

Thm 8.11 **(Fourier Differentiation Rule)** For a smooth 2π -periodic function with Fourier series

$$f(x) = \sum_k \hat{f}_k e^{ikx},$$

the derivative has coefficients

$$\hat{f}'_k = ik \hat{f}_k.$$

8.4.4 Exercises

Ex 8.15

1. Write out the 4×4 DFT matrix explicitly. Verify it is unitary using Ex 8.12.
2. Solve the periodic Poisson equation $-u'' = f$ via FFT. (*Hint: Laplacian is circulant*).
3. Compare the speed of `np.fft.fft(x)` vs. `F @ x` for $n = 4096$.
4. Implement spectral differentiation for $f(x) = \sin(x)$ using Thm 8.11 and check error vs. n .

8.5 Markov Chains and Stochastic Matrices

Markov chains model random processes whose next state depends only on the current state. With the column convention used here, probability vectors are columns and the update is $\mathbf{x}^{(k+1)} = \mathbf{P}\mathbf{x}^{(k)}$.

8.5.1 Definitions

Def 8.21 **(Probability Vector)** A vector $\mathbf{x} \in \mathbb{R}^n$ is a probability vector if

$$x_i \geq 0, \quad \sum_{i=1}^n x_i = 1.$$

The entry x_i is the probability of state i .

Def 8.22 **(Column-Stochastic Matrix)** Matrix $\mathbf{P} \in \mathbb{R}^{n \times n}$ where:

1. $P_{ij} \geq 0$.

2. $\sum_i P_{ij} = 1$ (Columns sum to 1).

Remark 8.12 (**Column convention**) Column j of $\mathbf{P}$ contains the probabilities of moving from state j to all possible next states. Some texts use row-stochastic matrices and row probability vectors. The two conventions are transposes of each other.

Thm 8.12 (**Spectrum of a Stochastic Matrix**) If $\mathbf{P}$ is column-stochastic, then $\lambda = 1$ is an eigenvalue and every eigenvalue satisfies $|\lambda| \leq 1$.

Proof *Proof.* Since $\mathbf{1}^T \mathbf{P} = \mathbf{1}^T$, 1 is a left eigenvalue, hence an eigenvalue. To bound the spectrum, apply the row-stochastic argument to $\mathbf{P}^T$: if $\mathbf{P}^T \mathbf{v} = \lambda \mathbf{v}$ and $|v_j| = \|\mathbf{v}\|_\infty$, then

$$|\lambda| |v_j| = \left| \sum_i P_{ij} v_i \right| \leq \sum_i P_{ij} |v_i| \leq |v_j|.$$

Thus $|\lambda| \leq 1$. □

Ex 8.16

1. Show that $\mathbf{1}^T \mathbf{P} = \mathbf{1}^T$ for a column-stochastic matrix.
2. Conclude that 1 is a left eigenvalue of $\mathbf{P}$, hence an eigenvalue.
3. Prove $|\lambda| \leq 1$ first for a row-stochastic matrix by taking the largest-magnitude entry of an eigenvector.
4. Apply the previous step to $\mathbf{P}^T$ to prove Thm 8.12.

Def 8.23 (**Markov Chain**) Probability distributions $\mathbf{x}^{(k)}$ evolving by $\mathbf{x}^{(k+1)} = \mathbf{P} \mathbf{x}^{(k)}$.

- After k steps: $\mathbf{x}^{(k)} = \mathbf{P}^k \mathbf{x}^{(0)}$.
- Long-run behavior is governed by the eigendecomposition of $\mathbf{P}$.

Def 8.24 (**Stationary Distribution**) A probability vector $\boldsymbol{\pi}$ such that $\mathbf{P} \boldsymbol{\pi} = \boldsymbol{\pi}$.

- Right eigenvector of $\mathbf{P}$ for $\lambda = 1$.
- Represents the equilibrium state of the chain.

Example (**Two-state chain**) Let

$$\mathbf{P} = \begin{pmatrix} 1-a & b \\ a & 1-b \end{pmatrix}, \quad 0 < a, b < 1.$$

Solving $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$ with $\pi_1 + \pi_2 = 1$ gives

$$\boldsymbol{\pi} = \frac{1}{a+b} \begin{pmatrix} b \\ a \end{pmatrix}.$$

Def 8.25 (Reducible and Periodic Chains) A chain is reducible if some state cannot be reached from another state. A state has period $d > 1$ if returns to that state occur only at times divisible by d . A primitive chain is irreducible and aperiodic.

Example (Two failure modes) The identity matrix is reducible: every state is absorbing. The matrix

$$\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$$

is irreducible but periodic with period 2, so the distribution oscillates instead of converging.

8.5.2 Convergence and Mixing

Thm 8.13 (Perron-Frobenius for Chains) If $\mathbf{P}$ is **irreducible** (all states reachable) and **primitive** (no cycles), then:

1. $\lambda_1 = 1$ is simple and unique.
2. $|\lambda_i| < 1$ for all other eigenvalues.
3. $\lim_{k \rightarrow \infty} \mathbf{P}^k \mathbf{x}^{(0)} = \boldsymbol{\pi}$ for any initial state.

Def 8.26 (Spectral Gap) The gap $\delta = 1 - |\lambda_2|$ controls convergence speed.

Thm 8.14 (Spectral Mixing Heuristic) For a diagonalizable primitive chain, the distance to stationarity is dominated asymptotically by the second eigenvalue:

$$\|\mathbf{x}^{(k)} - \boldsymbol{\pi}\| \approx C|\lambda_2|^k.$$

Thus a larger spectral gap $\delta = 1 - |\lambda_2|$ means faster mixing, and the iteration count to reach tolerance ε scales like

$$k_\varepsilon \approx \frac{\log(1/\varepsilon)}{\delta}$$

when δ is small.

Ex 8.17

1. Write an initial distribution as $\mathbf{x}^{(0)} = \boldsymbol{\pi} + \sum_{i \geq 2} c_i \mathbf{v}_i$ using eigenvectors of $\mathbf{P}$.
2. Apply $\mathbf{P}^k$ and use $\mathbf{P}\boldsymbol{\pi} = \boldsymbol{\pi}$.
3. Identify the slowest-decaying term.
4. Use $\log(1 - \delta) \approx -\delta$ to derive the mixing-time estimate in Thm 8.14.

8.5.3 Properties and Design

Def 8.27 (Detailed Balance (Reversibility)) A chain is reversible if $\pi_j P_{ij} = \pi_i P_{ji}$ for all i, j .

Thm 8.15 (Reversible Chains Have Real Spectrum) If $\mathbf{P}$ satisfies detailed balance with stationary distribution $\boldsymbol{\pi}$ and $\mathbf{D}_{\boldsymbol{\pi}} = \text{diag}(\boldsymbol{\pi})$, then

$$\mathbf{D}_{\boldsymbol{\pi}}^{-1/2} \mathbf{P} \mathbf{D}_{\boldsymbol{\pi}}^{1/2}$$

is symmetric. Therefore reversible chains have real eigenvalues.

Ex 8.18

1. Compute the (i, j) entry of $\mathbf{D}_{\boldsymbol{\pi}}^{-1/2} \mathbf{P} \mathbf{D}_{\boldsymbol{\pi}}^{1/2}$.
2. Use detailed balance from Def 8.27 to show that this entry equals the (j, i) entry.
3. Explain why similarity preserves eigenvalues.
4. Conclude Thm 8.15.

Remark 8.13 (Design Tool: Metropolis-Hastings) Construct a reversible chain for a desired $\boldsymbol{\pi}$ by accepting moves with probability $a = \min(1, \frac{\pi_{\text{new}} Q(\text{old}|\text{new})}{\pi_{\text{old}} Q(\text{new}|\text{old})})$.

Thm 8.16 (Metropolis-Hastings Detailed Balance) Let Q_{ij} be a proposal probability from state j to state i . Define

$$A_{ij} = \min\left(1, \frac{\pi_i Q_{ji}}{\pi_j Q_{ij}}\right), \quad i \neq j.$$

The transition probabilities $P_{ij} = Q_{ij} A_{ij}$ for $i \neq j$, with the remaining probability kept at state j , satisfy detailed balance with $\boldsymbol{\pi}$.

Proof *Proof.* For $i \neq j$,

$$\pi_j P_{ij} = \pi_j Q_{ij} \min\left(1, \frac{\pi_i Q_{ji}}{\pi_j Q_{ij}}\right) = \min(\pi_j Q_{ij}, \pi_i Q_{ji}).$$

The same calculation with i and j interchanged gives

$$\pi_i P_{ji} = \min(\pi_i Q_{ji}, \pi_j Q_{ij}).$$

Thus $\pi_j P_{ij} = \pi_i P_{ji}$. □

8.5.4 Exercises

Ex 8.19

1. Construct $\mathbf{P}$ for a 2-state chain with transition probabilities $\{0.1, 0.2\}$. Find $\boldsymbol{\pi}$.
2. Prove that $|\lambda| \leq 1$ for any stochastic matrix using Ex 8.16.
3. Simulate 100 steps of PageRank on a 5-node graph. Check convergence rate vs. λ_2 using Thm 8.14.
4. Find $\boldsymbol{\pi}$ for a random walk on K_4 (complete graph).
5. Give one reducible chain and one periodic irreducible chain. Explain which part of Thm 8.13 fails.
6. Verify detailed balance for a three-state Metropolis-Hastings chain using Thm 8.16.

8.6 Koopman Theory, DMD, and SINDy

Koopman analysis studies nonlinear dynamics through linear evolution of observables. DMD fits a finite-dimensional linear model from snapshots. SINDy fits sparse governing equations from a chosen function library.

8.6.1 Koopman Operator Theory

Def 8.28 (Nonlinear Dynamics and Observables) $\mathbf{x}^{(k+1)} = \mathbf{F}(\mathbf{x}^{(k)})$ on $\mathcal{M} \subseteq \mathbb{R}^n$.

- **Observable:** Scalar function $g : \mathcal{M} \rightarrow \mathbb{R}$.
- **Koopman Operator $\mathcal{K}$:** Linear operator acting on observables: $(\mathcal{K}g)(\mathbf{x}) = g(\mathbf{F}(\mathbf{x}))$.

Lifting: $\mathcal{K}$ is linear even if $\mathbf{F}$ is nonlinear, at the cost of being infinite-dimensional.

Prop 8.17 (Koopman Linearity) The Koopman operator in Def 8.28 is linear on observables:

$$\mathcal{K}(\alpha g + \beta h) = \alpha \mathcal{K}g + \beta \mathcal{K}h.$$

Proof *Proof.* For every state $\mathbf{x}$,

$$\begin{aligned} [\mathcal{K}(\alpha g + \beta h)](\mathbf{x}) &= (\alpha g + \beta h)(\mathbf{F}(\mathbf{x})) \\ &= \alpha g(\mathbf{F}(\mathbf{x})) + \beta h(\mathbf{F}(\mathbf{x})) \\ &= [\alpha \mathcal{K}g + \beta \mathcal{K}h](\mathbf{x}). \end{aligned}$$

Since this holds for every $\mathbf{x}$, the two observables are equal. □

Example (Nonlinear map) Let $x_{k+1} = x_k^2$. For observables $g_1(x) = x$ and $g_2(x) = x^2$,

$$(\mathcal{K}g_1)(x) = x^2 = g_2(x), \quad (\mathcal{K}g_2)(x) = x^4.$$

The state map is nonlinear, but the operator $g \mapsto g \circ F$ is linear on the space of observables.

Def 8.29 (Koopman Eigenfunctions) Observable φ satisfying $\mathcal{K}\varphi = \mu\varphi$.

- **Temporal Evolution:** $\varphi(\mathbf{x}^{(k)}) = \mu^k \varphi(\mathbf{x}^{(0)})$.
- **Significance:** Provides a global linear coordinate system for nonlinear dynamics.

8.6.2 Dynamic Mode Decomposition (DMD)

Def 8.30 (DMD Algorithm) Finds the best linear fit $\mathbf{Y} \approx \mathbf{A}\mathbf{X}$ for snapshot matrices:

$$\mathbf{X} = [\mathbf{x}^{(0)} \dots \mathbf{x}^{(m-1)}], \quad \mathbf{Y} = [\mathbf{x}^{(1)} \dots \mathbf{x}^{(m)}].$$

1. Compute truncated SVD: $\mathbf{X} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T$.
2. Construct $r \times r$ reduced operator: $\tilde{\mathbf{A}} = \mathbf{U}_r^T \mathbf{Y} \mathbf{V}_r \mathbf{\Sigma}_r^{-1}$.
3. DMD eigenvalues $\{\lambda_i\}$ and modes $\{\phi_i\}$ are recovered from $\tilde{\mathbf{A}}$.

Thm 8.18 (DMD Least Squares Model) The best-fit linear map in Frobenius norm is

$$\mathbf{A}_* = \mathbf{Y}\mathbf{X}^\dagger.$$

The reduced operator in Def 8.30 is the projection of this map onto the rank- r left singular subspace of $\mathbf{X}$.

Ex 8.20

1. Use the least squares normal equations to derive $\mathbf{A}_* = \mathbf{Y}\mathbf{X}^\dagger$.
2. Substitute the truncated SVD $\mathbf{X} \approx \mathbf{U}_r \mathbf{\Sigma}_r \mathbf{V}_r^T$.
3. Show that $\mathbf{U}_r^T \mathbf{A}_* \mathbf{U}_r = \mathbf{U}_r^T \mathbf{Y} \mathbf{V}_r \mathbf{\Sigma}_r^{-1}$.
4. Interpret this as the small operator $\tilde{\mathbf{A}}$ in Def 8.30.

Remark 8.14 **Interpretation:** DMD decomposes data into spatiotemporal modes ϕ_i with associated frequencies and growth rates. It is the data-driven approximation of the Koopman operator using the identity dictionary.

8.6.3 SINDy: Sparse Identification

Def 8.31 (**Sparse Regression for Dynamics**) Identifies governing equations $\dot{\mathbf{x}} = \Xi^T \Theta(\mathbf{x})$ where Θ is a library of candidate functions (monomials, sin, etc.).

- **STLS Algorithm:** Sequential Thresholded Least Squares. Iteratively solves least squares and thresholds small coefficients to promote sparsity.
- **Outcome:** Parsimonious, interpretable models (e.g., recovering Lorenz equations).

Remark 8.15 (**DMD vs. SINDy**) DMD fits a linear time-advance map for snapshot data. SINDy fits a sparse differential equation in a chosen library. DMD returns modes and growth rates; SINDy returns equation coefficients.

Remark 8.16 (**Noisy derivatives**) SINDy is sensitive to derivative estimates. Smoothing, weak-form regression, or integral formulations are often needed when data are noisy.

8.6.4 Exercises

Ex 8.21

1. Prove Prop 8.17 without looking at the proof.
2. Implement DMD for a 2D damped oscillator. Verify the identified eigenvalues.
3. Build a polynomial library Θ for $\mathbf{x} \in \mathbb{R}^2$ up to degree 2.
4. Use SINDy (via STLS) to identify the Lotka-Volterra predator-prey model from noisy data.
5. Compare DMD and SINDy on the same two-dimensional linear system. Which quantities should agree?